

Principia Softwarica
Fundamental Literate System Programs
version 1.2

Yoann Padioleau
`yoann.padioleau@gmail.com`

with code from
Ken Thompson, Rob Pike, Dave Presotto, Phil Winterbottom,
Tom Duff, Andrew Hume, Sape Mullender, Russ Cox,
Richard Miller, Ori Bernstein, and Ron Minnich.

June 5, 2026

The text and figures are Copyright © 2014–2026 Yoann Padioleau.
All rights reserved.

The source code is Copyright © 2021 Plan 9 Foundation.
MIT license.

Contents

1	Introduction	5
1.1	Motivations	5
1.2	The ideal teaching operating system	7
1.3	The Plan 9 operating system	7
1.4	The C language and ARM architecture	9
1.5	Literate programming	9
1.6	Getting started	10
1.7	Requirements	11
1.8	Copyright	11
1.9	Acknowledgments	11
2	Overview	12
2.1	Code organization	12
2.2	Software architecture	12
3	The Literate Programs	16
3.1	The Core system	16
3.1.1	The kernel: <code>9pi</code>	16
3.1.2	The core libraries: <code>libc</code> , <code>libthread</code> , <code>libbio</code> , <code>libflate</code> , <code>libregexp</code>	16
3.1.3	The shell: <code>rc</code>	16
3.2	The development toolchain	17
3.2.1	The C compiler: <code>5c</code>	17
3.2.2	The assembler: <code>5a</code>	17
3.2.3	The linker: <code>5l</code>	17
3.2.4	The processor emulator: <code>5i</code>	17
3.3	The developer tools	18
3.3.1	The text editor: <code>ed</code>	18
3.3.2	The build system: <code>mk</code>	18
3.3.3	The version control system: <code>git9</code>	18
3.3.4	The debuggers: <code>acid</code> , <code>ratrace</code> , <code>db</code>	18
3.3.5	The profilers: <code>prof</code> , <code>tprof</code> , <code>stats</code> , <code>iostats</code>	19
3.4	Graphics	19
3.4.1	The graphics stack: <code>/dev/draw</code>	19
3.4.2	The windowing system: <code>rio</code>	19
3.4.3	The graphical user interface toolkit: <code>libpanel</code>	20
3.5	Networking	20
3.5.1	The network stack: <code>/net</code>	20
3.5.2	The web browser: <code>mothra</code>	20
3.6	Utilities: <code>cat</code> , <code>ls</code> , <code>grep</code> , <code>sed</code> , <code>cmp</code> , <code>tar</code> , <code>gzip</code> , <code>bc</code> , <code>dc</code> , <code>hoc</code> , <code>awk</code> , ...	20
3.7	Applications	22

4	Conclusion	23
A	Other Teaching Operating Systems	24
B	Bootstrapping from Scratch	26
B.1	A basic computer	27
B.2	A booting procedure	27
B.3	Physical programs	28
B.4	Phase 1: Machine code	28
B.5	Phase 2: Assembly	30
B.6	Phase 3: C	33
B.7	Summary of Plan 9 ancestor programs	34
C	Literate Program Example: ed	35
C.1	Introduction	35
C.1.1	A line editor	35
C.1.2	A scriptable editor	36
C.1.3	A Unicode editor	37
C.2	Core data structures	38
C.2.1	The backing store: <code>tfname</code> , <code>tfile</code> , and <code>tline</code>	38
C.2.2	The target file: <code>savedfile</code>	39
C.2.3	The lines as file offsets: <code>zero</code>	39
C.2.4	Cursors: <code>dot</code> and <code>dol</code>	40
C.2.5	Line input and output buffers: <code>line</code> and <code>linebuf</code>	41
C.2.6	Other globals: <code>count</code> , <code>col</code> , etc.	41
C.3	<code>main()</code>	41
C.3.1	<code>ed -</code> and <code>vflag</code>	42
C.3.2	<code>ed -o</code> and <code>oflag</code>	43
C.3.3	<code>mktemp()</code>	43
C.3.4	<code>init()</code>	44
C.3.5	<code>quit()</code>	44
C.4	Displaying and reading text	45
C.4.1	Displaying text	45
C.4.2	Reading text	47
C.5	<code>commands()</code> interpreter loop	49
C.6	reading a file: <code>r</code>	49
C.6.1	Reading a <code>filename()</code>	50
C.6.2	<code>setwide()</code> and <code>squeeze()</code>	51
C.6.3	<code>append()</code> and <code>getfile()</code>	52
C.6.4	<code>putline()</code> (simplified)	54
C.6.5	Concrete editing example	55
C.6.6	<code>exfile()</code>	56
C.7	writing a file: <code>w</code>	57
C.7.1	<code>putfile()</code>	57
C.7.2	<code>getline()</code> (simplified)	58
C.7.3	Write and quit: <code>wq</code>	58
C.8	Main commands	59
C.8.1	printing lines: <code>p</code>	59
C.8.2	printing the remembered file: <code>f</code>	60
C.8.3	printing line number: <code>=</code>	61

C.8.4	append and insert: <code>a, i</code>	61
C.8.5	quitting: <code>q</code>	62
C.8.6	deleting lines: <code>d</code>	62
C.8.7	changing lines: <code>c</code>	62
C.8.8	moving and copying lines: <code>m</code> and <code>t</code>	63
C.9	Command addresses	65
C.9.1	Reading <code>addr1</code> and <code>addr2</code>	66
C.9.2	<code>address()</code>	67
C.9.3	Basic addresses: <code>.</code> and <code>\$</code>	69
C.10	Search and replace	69
C.10.1	Search as addresses: <code>/re/</code> and <code>?re?</code>	70
C.10.2	Reading and compiling a regexp: <code>compile()</code>	71
C.10.3	<code>match()</code>	72
C.10.4	Reading and compiling a substitution: <code>compsub()</code>	73
C.10.5	substitute: <code>s</code>	74
C.10.6	Advanced substitutions	77
C.10.7	global commands: <code>g/re/cmd</code> and <code>v/re/cmd</code>	80
C.11	Advanced features	82
C.11.1	Running a shell command: <code>!</code>	83
C.11.2	Advanced listing: <code>l</code>	83
C.11.3	Advanced listing: <code>n</code>	84
C.11.4	Automatic listing: the <code>...pln</code> suffix flag	85
C.11.5	joining lines: <code>j</code>	86
C.11.6	browse: <code>b</code>	86
C.11.7	edit remembered file: <code>e</code>	87
C.11.8	Append-only files	88
C.12	Optimizations	88
C.12.1	Optimized <code>putline()</code>	88
C.12.2	<code>getblock()</code>	89
C.12.3	Optimized <code>getline()</code>	91
C.12.4	<code>gdelete()</code>	92
C.13	Error management	92
C.13.1	<code>error()</code>	93
C.13.2	Notes/signals management	94
C.14	Conclusion	94
C.15	Extra code	95
C.16	Index	98

References

100

Chapter 1

Introduction

Principia Softwarica is a series of books explaining how things work in a computer by describing with full details all the source code of all the essential programs used by a programmer. Among those essential programs are the kernel, the shell, the windowing system, the web browser, the compiler, the linker, the editor, or the debugger. Each program will be covered by a separate book.

The books not only describe the implementations of essential programs, *they are* the implementations of those programs. Indeed, each program in *Principia Softwarica* comes from a *literate program* [Knu92], which is a document containing both source code and documentation and where the code is organized and presented in a way to facilitate its comprehension. The actual code and the book are derived both automatically from this literate program. See Appendix C for a concrete example of literate program.

The goal of the report you are reading now is to introduce the series and to give a quick overview of the *Principia Softwarica* programs. They form together the foundation on top of which all applications can be built. Similar to *Principia Mathematica* [WR13], which is a series of books covering the foundations of mathematics, the goal of *Principia Softwarica* is to cover the fundamental programs. Those programs are mostly all *meta programs*, which are programs in which the input and/or output are other programs. For instance, the kernel is a program that *manages* other programs; the compiler is a program that *generates* other programs. Those programs are also sometimes referred as *system software*, in opposition to *application software* (e.g., spreadsheets, word processors, email clients), which I will not cover in *Principia Softwarica*.

1.1 Motivations

Why did I write those books? The main reason is that I have always been curious and always wanted to understand how things work under the hood, fully, to the smallest detail. Programs are now running the world; it is thus important to understand those programs, to be computer literate, and source code is what defines those programs.

There are already lots of books explaining how computers work, explaining the concepts, theories, and algorithms behind programs such as kernels or compilers. There are also a few books about debuggers. However, all those books rarely explain everything with full details, which is what source code is all about. There are a few books that include the whole source code of the program described, for instance, the books about Minix [Tan87], XINU [Com84], a toy Web Browser [PH24], or LCC [FH95]. However, those books cover only a few essential programs, and mostly always either the kernel or the compiler. Moreover, they do not form a coherent set.

Enter *Principia Softwarica*, a set of books covering all essential programs, in a coherent way. In addition to the kernel and compiler, *Principia Softwarica* covers also the graphics stack, the networking stack, the windowing system, the assembler, the linker, and many other fundamental programs that have never been fully explained before to the best of my knowledge.

I want to demystify those programs by showing their code, and by showing that they are actually not that complicated. I hope to remove some of the mental barriers people have that prevent them from extending the

tools they use every day: text editors, compilers, even kernels. As a side effect, it will also maybe help people imagine better systems. Indeed, it can be very intimidating to invent something completely new if you have no clue that it is actually possible to build from scratch a complete operating system.

Another motivation for those books, in addition to satisfy my curiosity, as well as your curiosity, is that I think you are a better programmer if you understand how things work under the hood. In my opinion, you become a better C programmer when you understand roughly what code generates the C compiler. A good way to write more efficient code, or to avoid writing really slow code, is to have some ideas of the assembly code generated for your code by the compiler. In the same way, you can better use resources, for instance, memory, if you have some ideas about how the kernel manages for you these resources; you can better fix latency issues if you understand how the networking stack works. The Principia Softwarica books can complement the excellent book *Computer systems: a programmer's perspective* [BO10] by illustrating the many concepts this book introduces with concrete code.

I also think it is easier to debug programs if you better understand the environment in which those programs evolve, if you understand the whole software stack, and how things interact with each other. Indeed, to fully understand certain error messages, e.g., from the kernel, from the networking stack, or from the linker, it is very useful to have some ideas about what those programs do. Moreover, even if in almost all situations the bug or the performance issue is in your program, it could also be sometimes in a core library, in the kernel, in the compiler, or in the linker. It is very rare, but when those situations happen, if you have no clue about the environment in which your program runs, you will never be able to fix your problem.

Finally, by showing code written by great programmers, I hope you will also learn how to write better programs. In other engineering fields it is quite common for the students to learn from the work of the masters of their fields.

Here are a few questions I hope the Principia Softwarica books will answer:

- What happens when the user turns on the computer? Which program gets executed first? What does this program?
- What exactly happens when the user types `ls` in a terminal? What is the set of programs involved in such a command? What is the trace of this command through the different layers of the software stack, from the keyboard interrupt to the display of text glyphs on the screen?
- How does source code get compiled, assembled, linked, and finally loaded in memory? What is the memory image of a program? How does it relate to the original source code? How can a debugger display debugging information from a binary?
- How does a debugger work? Does it rely on special services from the processor or from the kernel to implement breakpoints?
- Which program contains the memory allocator? The kernel or The C library? How `malloc()` is implemented?
- How certain graphical elements (e.g., rectangles, ellipses, characters) are rendered on the screen? How does the graphics card help? How does the kernel help? How things are intercepted by the windowing system to make sure applications can not draw in other windows?
- What happens when you open a connection to another machine, when you type a URL in your web browser? How can you achieve a reliable communication on an unreliable physical network?
- What does it take to port an entire operating system to a new machine, for instance, the Raspberry Pi¹?

¹<https://www.raspberrypi.org/>

1.2 The ideal teaching operating system

The question now is which actual source code to present? The code of the major operating systems (e.g., GNU/Linux, macOS, Windows) is gigantic with hundreds of millions lines of code (LOC). Here by operating system I mean not only the kernel but also the programs (the windowing system, the compiler, etc) that provide the platform for running application software. It is impossible to understand such large codebases.

I think there is hope though, that it is possible to understand fairly well everything, that it is possible to answer all of the questions mentioned in the previous section by focusing on the *essence* of those major operating systems. I think it is possible to design a teaching operating system with capabilities similar to the mainstream operating systems, but with a fraction of their code size. Here are the requirements for the ideal teaching operating system I want to use as the basis for the Principia Softwarica books:

- *Open source*: I want to show code, lots of code, and I want you to be able to play with such code, to modify it, so the ideal operating system must be open source.
- *Small*: I want programs that can be described in books of reasonable size. The program needs to be a bit minimalist.
- *As simple as possible, but not simpler*: I want small code, but I do not want to show toy code. For instance, ARM [Sea01] is a simpler architecture than x86 [Int86], so it makes sense to present the code of an ARM assembler rather than an x86 assembler, but I do not want to show a toy assembler for a toy architecture. In the same way I can present the code of a C compiler without certain advanced state-of-the-art optimizations, to simplify the presentation, but I do not want to show a toy compiler for a toy language.
- *Real*: It has to run on real machines (e.g., an x86 desktop, a Mac laptop, a Raspberry Pi), so you can play with it.
- *Complete*: It needs to not only have a kernel and a compiler but also a graphics stack, a networking stack, a windowing system, etc.
- *Coherent*: The whole set of programs must form a coherent set, so that the interactions between those programs can also be described, and described succinctly.
- *Self hosting*: I want to be able to improve, to recompile, and to run everything under the system itself.

Fortunately, this ideal teaching operating system already exists; it is Plan 9 (I discuss a few other candidates in Appendix A).

1.3 The Plan 9 operating system

Plan 9² is the successor of UNIX. It was designed from scratch by a small team of great programmers (Rob Pike, Dave Presotto, Phil Winterbottom), including the original creator of UNIX (Ken Thompson). Their goal was to redesign UNIX to better integrate graphics and networking, which both became popular after UNIX was originally invented. In some sense, Plan 9 is a kind of UNIX 2.0.

The choice of Plan 9 as the basis for the Principia Softwarica books may not be obvious, but it is in my opinion the simplest and at the same time fairly complete operating system. If you look at the screenshot of Plan 9 in action in Figure 1.1, you will see many features:

- A screen with basic graphics and multiple windows

²plan9.bell-labs.com/plan9/

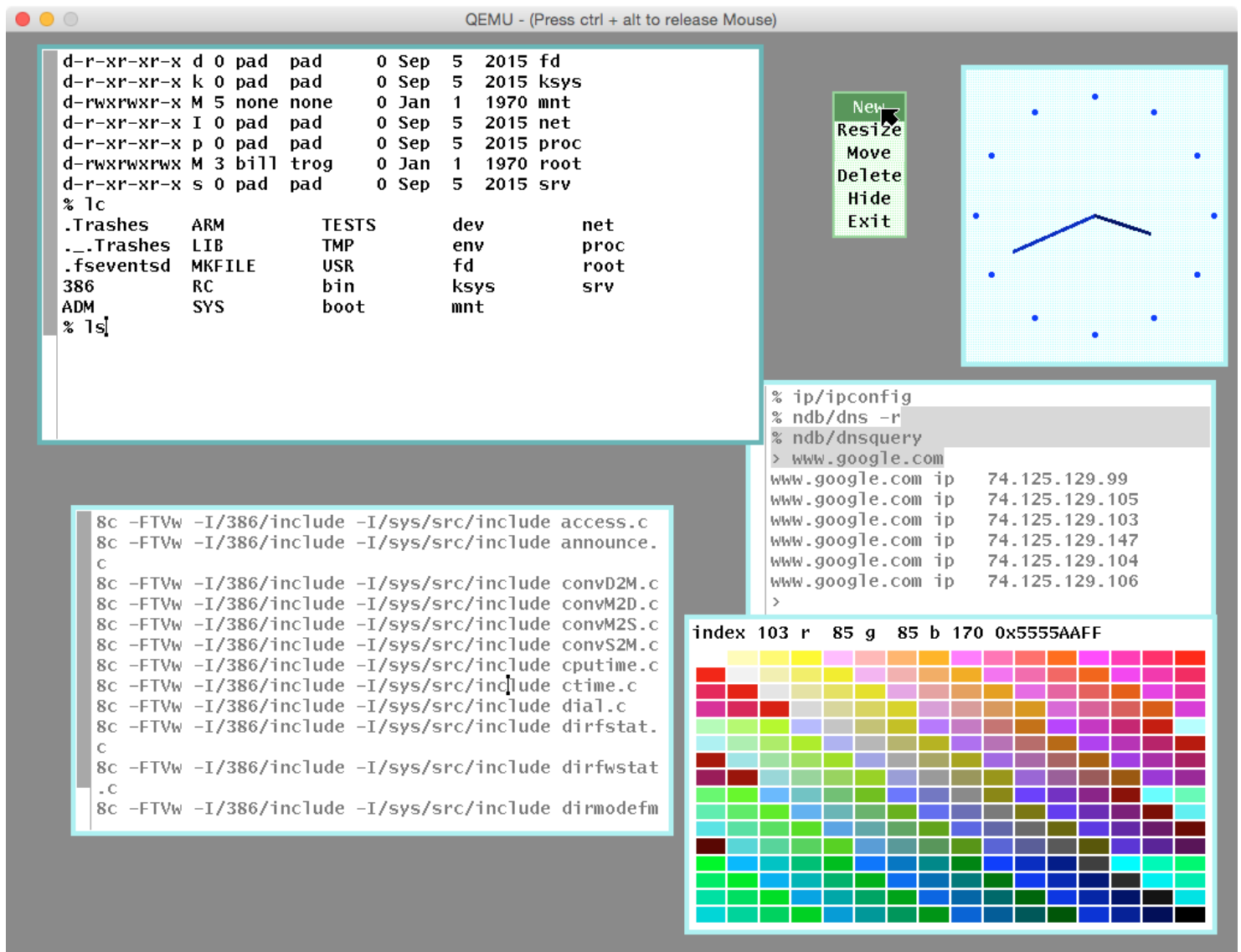


Figure 1.1: Plan 9 in action.

- Multiple shells running independent commands at the same time
- A simple clock graphical application
- A simple program communicating through the network

By comparison, if you look at other systems you will see that Plan 9 *in essence* provides the same core services than GNU/Linux/Xorg, macOS, or Microsoft Windows, without certain bells and whistles, and with a significantly smaller codebase. Indeed, my fork of Plan 9 (whose source code is available at <https://github.com/aryx/principia-softwarica/>), which includes all the essential programs described in all the Principia Softwarica books, and even a few more programs, is less than 365 000 LOC.

The Plan 9 programs are minimalist but powerful, and their source code is simple and small. The creators of Plan 9 were not afraid to rethink everything, not just the kernel, even if it meant not being backward compatible with UNIX. This led to programs with more elegant designs, to the removal of many ugly corners in UNIX, and ultimately to less source code while still providing more services. For instance, thanks to a few novel ideas in the kernel, namely per-process namespace, user-space filesystems, and union mount, every application who wish to interact with the console can just use the uniform `/dev/cons` device file, whether this console is attached directly to the physical terminal, to a remote machine, or whether it is one of the terminal window of the graphical user

interface. Thanks to this design, the Plan 9 windowing system `rio` could be implemented with only 8 800 LOC, including the code to emulate terminals in windows.

By comparison, Linux and X Window introduced the separate concepts of teletype `tty` devices and pseudo `pty` terminals. The code of `xterm`, which is just a terminal for X Window, not X Window itself, has already 88 000 LOC. This is partly because `xterm` carries the historical baggage of standards invented in the 70's (e.g., special control sequences for VT100 terminals).

Plan 9 contains all the essential programs used by a programmer. They form a coherent set because they were all designed from scratch by a small team of great programmers.

1.4 The C language and ARM architecture

The goal of books such as *The Elements* [EucBC] or *Principia Mathematica* [WR13] is to describe the foundation of a field starting from a very small basis, for instance, a logic language with a small set of axioms and inference rules, on top of which all the rest can be derived or built. The book *The Elements of Computing Systems* [NS05] does a similar thing for computers. Starting only from the `nand` logic gate, the book builds gradually the `and`, `or`, and `not` logic gates, a multiplexer, a flip-flop, memory banks, an adder, an arithmetic and logic unit, and finally a simple processor.

However, in Principia Softwarica we are interested in software, not hardware. Because most of the programs described in the Principia Softwarica books are written in C, our basis in some sense is the C programming language [KR88]. C is not a large language by modern standards, but it is larger than just a few set of axioms. Moreover, C has non trivial semantics, so our basis is unfortunately relatively large. Note that one of the Principia Softwarica book describes a C compiler that targets the ARM [Sea01] architecture, so in principle our basis could be reduced to the ARM machine language, which is fairly simple. Another Principia Softwarica book describes an ARM emulator. However, both the C compiler and the ARM emulator are written in C itself, which brings us back to C as our basis.

A more elegant alternative, avoiding self-reference, would be to start from a simple machine and build a tower of increasingly powerful languages. Starting from raw binary machine code, we could gradually build more sophisticated languages through a series of *bootstrapping* steps. In fact, such projects exist³, but with already six bootstrapping steps its author was still far away from a language like C. Appendix B outlines a similar (long) process to bootstrap Plan 9 from scratch.

Another alternative, chosen by Donald Knuth for his encyclopedic books *The Art of Computer Programming* [Knu73], was to pick as a basis a very simple computer he invented called MIX, and a very simple assembly language called MIXAL. Using assembly is maybe OK for describing algorithms, but I think it would not be productive for writing entire programs; this would lead to very long Principia Softwarica books.

I think that starting directly from the ARM machine, a real but fairly simple machine, and the C language, a higher level language than assembly, is maybe less elegant but more practical for Principia Softwarica.

1.5 Literate programming

I want to show source code because it is the ultimate explanation for what a program does. However, I think that showing pages and pages of listings in an appendix, as done for instance in the Minix book [Tan87], even when this appendix is preceded by documentation chapters, is not the best way to explain code. I think the code and its documentation should be mixed together, as done for instance in the Xinu book [Com84], so you do not have to switch back and forth between an appendix and multiple chapters.

Literate programming [Knu92] is a technique invented by Donald Knuth to make it easy to mix code and documentation in a document in order to better develop and better explain programs. Such documents are

³See <http://homepage.ntlworld.com/edmund.grimley-evans/bcompiler.html> or more recently the MES (Maxwell Equations of Software) project <https://www.gnu.org/software/mes/>.

called *literate programs*. All Principia Softwarica programs are literate programs.

Note that literate programming is different from using API documentation generators such as javadoc⁴ or doxygen⁵. Noweb⁶, the literate programming tool I used, does not provide the same kind of services. Indeed, literate programming allows programmers to explain their code in the order they think the flow of their thoughts and their code would be best understood, rather than the order imposed by the compiler.

Literate programming allows, among other things, to explain the code piece by piece, with the possibility to present a high-level view first of the code, to switch between top-down and bottom-up explanations, and to separate concerns. For instance, the `Proc` data structure — which you will see in the `KERNEL` book [Pad14] is a data structure that represents some information about a process — is a huge structure with more than 90 fields. Many of those fields are used only for advanced features of the kernel. The C compiler imposes to define this structure in one place. Noweb allows to present this structure piece by piece, gradually, in different chapters. I can show first the code of the structure with the most important fields, and delay the exposition of other fields to advanced chapters. This greatly facilitates the understanding of the code by not submerging the reader, you, with too much details first.

In the same way, the `main()` function in most programs is rather large and mixes together many concerns: command line processing, error management, debugging output, optimizations, and usually a call to the main algorithm. Showing in one listing the whole function would hide behind noise this call to the main algorithm. The main flow of the program though is arguably the most important thing to understand first. Using literate programming, I can show code where the most important parts are highlighted, and where other concerns are hidden and presented later.

In fact, I spent lots of time during the writing of the Principia Softwarica books in transforming the Plan 9 programs in literate programs, and in reorganizing again and again the Plan 9 code to find the best way, the best order, the best separation of concerns in which I think you would more easily understand the code.

Appendix C presents a concrete example of literate program, illustrating the advantages of literate programming mentioned in this section.

1.6 Getting started

To play with the different programs described in the Principia Softwarica books, I recommend to use my fork of Plan 9: <https://github.com/aryx/principia-softwarica>. Section 2.1 will explain the directory structure of this repository. To compile and install this fork of Plan 9, you will need a machine with a C compiler, a UNIX-like operating system, and a kernel that can write on a VFAT filesystem. So, GNU/Linux and macOS are possible host operating systems. You can also build Plan 9 using Docker⁷.

Because the source code of Plan 9 uses a special dialect of C and some non-standard assembly, you can not use directly popular C compilers and assemblers such as `gcc`⁸ and `gas`⁹. To compile Plan 9, you will also need to download my fork of Ken Thompson's C (cross) compilers called Goken at <https://github.com/aryx/goken9cc>. You can *compile* Goken using a regular C compiler (e.g., `gcc`) from your host operating system (e.g., from Linux or macOS). Then you can *cross-compile* the Plan 9 kernel, the Plan 9 C standard library, and then the whole Plan 9 operating system with all its libraries and programs, by using the Goken compiler installed in the previous step. This will build from scratch a Plan 9 distribution.

Finally, you can run this Plan 9 distribution either under Qemu¹⁰, which makes it easy to experiment, or by copying it on an SD Card and booting a physical machine such as a Raspberry Pi. See <https://www.principia-softwarica.org/getting-started.html> for the full procedure.

⁴<http://www.oracle.com/technetwork/articles/java/index-jsp-135444.html>

⁵<http://www.stack.nl/~dimitri/doxygen/>

⁶<http://www.cs.tufts.edu/~nr/noweb/>

⁷<https://www.docker.com/>

⁸<https://gcc.gnu.org/>

⁹<https://www.gnu.org/software/binutils/>

¹⁰<http://www.qemu.org>

1.7 Requirements

The Principia Softwarica books are not introductions to programming (read [FFFK18] instead), computer science (read [BB18] instead), or to any subfields of computer science. For instance, the book on the kernel is not an introduction to operating systems (read [SGG18] instead). Indeed, I will assume you have already a rough idea of how a kernel works and so that you are already familiar with concepts such as virtual memory, critical regions, interrupts, or system calls. I will present with full details the source code of different programs, but I assume you already know most of the concepts, theories, and algorithms behind those programs. The Principia Softwarica books are there to cover the practice. I assume the readers of Principia Softwarica are students in computer science or programmers who desire to consolidate their knowledge by reading the ultimate computer science explanations: source code.

As I said earlier, because most of the books are made of C source code, you will need a good knowledge of the C programming language [KR88]. Because the programs I describe are Plan 9 programs, and because Plan 9 has a lot in common with its ancestor UNIX, you will also need to be familiar with those systems. I recommend to read *The UNIX Programming Environment* [KP84] for UNIX, and to read the Plan 9 tutorials [PPD+95, PPT+93] available in my Plan 9 repository for Plan 9.

1.8 Copyright

All the programs in Principia Softwarica are from Plan 9, with copyrights from the Plan 9 Foundation. They are open source; permission is granted to copy, distribute and/or modify the source code according to the MIT license.

1.9 Acknowledgments

I would like to acknowledge of course Plan 9's authors who wrote in some sense most of the content of the Principia Softwarica books: Ken Thompson, Rob Pike, Dave Presotto, Phil Winterbottom, Russ Cox, and many other people from Bell Labs.

Finally, I would like to thank Donald Knuth for his early encouragement in this project. It meant everything.

Chapter 2

Overview

Before jumping in the description of the different Principia Softwarica programs in the next chapter, I first give here an overview of how the code is organized, and how the different programs depend on each other.

2.1 Code organization

Table 2.1 presents short descriptions of the main directories in my fork of Plan 9 (available at <https://github.com/aryx/principia-softwarica>), as well as the corresponding sections in this document in which the program associated with the directory is discussed.

A few Plan 9 programs have some architecture-specific parts, with support for x86 and ARM in my fork of Plan 9. The LOC column in Table 2.1 accounts for the architecture independent part of the code, as well as the x86 and ARM specific code. However, the Principia Softwarica books will present only ARM-specific code. Moreover, even if some directories have a plural form, e.g., `editors/`, the LOC column accounts only for one variant of this program category, e.g., one of the editor, the one I chose to present in Principia Softwarica (`ed`).

2.2 Software architecture

Many of the Principia Softwarica programs are mutually dependent on each other. Indeed, to run a compiler or an editor you need a kernel (and a shell), but to create this kernel in the first place you need an editor and a compiler. In a similar way the C compiler uses code from the core C library, but to create this library you need a C compiler. In fact, the C compiler is written in C itself, so there are even self-dependencies¹

It is possible though to *layer* things by looking at how things are organized in memory. A first separation to make is between code running in *kernel space* and code running in *user space*, as shown in the left of Figure 2.1. Most of the code running in kernel space is in `kernel/`, as well as some code in `lib_graphics/` and `lib_networking/`. Some of the code in the C library (the memory pool library² and a few utility functions and globals) are used both in the kernel and in user programs, but they are the exceptions. The rest of the codebase runs in user space.

The boundary between user programs and the kernel is provided by the *system calls* application programming interface (API). The functions of this API are declared in `include/core/syscall.h` included from `include/core/libc.h` (which contains also many utility functions). The user-space part of the system calls are implemented in `lib_core/libc/9syscall/`. This directory contains one assembly file per system call and each of those files contains mostly the software interrupt instruction with a special value in one of the argument in order to be dispatched to the appropriate code in the kernel. The dispatcher kernel code is in `kernel/syscalls/`. Some of those system calls are process related (e.g., `rfork()`, `exec()`, `exit()`³), some are memory related (e.g.,

¹Appendix B describes how to solve those mutual and self-dependencies issues.

²`lib_core/libc/port/pool.c`

³with implementation in `kernel/processes/`

Directory	Description	Section	LOC
kernel/	The Plan 9 basic kernel (for ARM and x86)	3.1.1	75 000
	+ parts of graphics stack (devices)	3.4.1	11 000
	+ network stack (protocols)	3.5.1	23 000
include/	The header files (e.g., <code>libc.h</code>)	3.1.2	7 500
lib_core/	The core C libraries (for ARM and x86)	3.1.2	33 000
shells/	The shell <code>rc</code>	3.1.3	7 500
compilers/	The C compiler (for ARM and x86)	3.2.1	39 000
assemblers/	The ARM and x86 assemblers	3.2.2	6 000
linkers/	The ARM and x86 linkers	3.2.3	17 000
machine/	The ARM emulator <code>5i</code>	3.2.4	4 500
generators/	The code generators <code>Lex</code> and <code>Yacc</code>	??	6 500
editors/	The editor <code>ed</code>	3.3.1	1 600
builders/	The build system <code>mk</code>	3.3.2	6 000
version_control/	The version control system <code>git9</code>	3.3.3	10 000
debuggers/	The debuggers (for ARM and x86) and tracers	3.3.4	22 000
profilers/	The profilers	3.3.5	5 000
lib_graphics/	Graphics stack libraries	3.4.1	29 000
windows/	The windowing system <code>rio</code>	3.4.2	8 800
lib_gui/	The widget library <code>libpanel</code>	3.4.3	4 000
lib_networking/	A small part of the network stack	3.5.1	4 000
browsers/	The web browser <code>mothra</code>	3.5.2	15 000
utilities/	Utilities such as <code>ls</code> , <code>cp</code> , <code>mv</code> <code>grep</code> , <code>gzip</code> , <code>tar</code> , <code>sed</code> , <code>cmp</code> , <code>xargs</code> , <code>ps</code> , <code>awk</code> , etc	3.6	28 000
Total			365 000

Table 2.1: Main source code directories of my Plan 9 repository.

`brk()` ⁴), and other are used for file input and output (IO) (e.g., `open()`, `close()`, `read()`, `write()` ⁵).

In fact, those IO system calls provide an extended way for programs to request services from the kernel by using the filesystem hierarchy. Indeed, one of the philosophy of UNIX is that *everything is a file*, including the devices. A process using the `/dev/cons` device file will trigger code in the kernel in `kernel/console/devcons.c`; this code will then trigger code that handles the keyboard device in `kernel/devices/keyboard/` (when reading from `/dev/cons`), or the screen device in `kernel/devices/screen/` (when writing to `/dev/cons`). This philosophy was pushed even further under Plan 9 where *everything is a file server*. The graphics API for instance is accessible through many files under `/dev/draw/`, and triggers code in `kernel/devices/screen/devdraw.c` and `lib_graphics/`. In a similar way, the networking API is accessible through files under `/net/` and triggers code in `kernel/network/`.

A second separation to be made is between *library code* and *application code*. All the user programs in Plan 9 rely first on the core C library in `lib_core/libc/`, exposed via the `include/core/libc.h` header file; all programs are linked with `libc.a`. The other `lib_xxx/` directories contain other general-purpose functions and data structures, which are used by different programs. By using libraries, source code can be reused more easily.

⁴with implementation in `kernel/memory`

⁵with implementation in `kernel/files/`

The shell is a regular program using also the C library. The assembler, linker, and compiler rely on the C library as well as other header files, for instance, `include/obj/5.out.h`, which declares the set of ARM opcodes.

All the graphical applications, for instance, the clock, but also the windowing system, rely also on the `lib_graphics/lib_draw/` library, which is exposed in the `include/graphics/draw.h` header file. The graphics library is essentially a thin wrapper over the protocol used by the `/dev/draw/` device files. In the same way, the `lib_networking/libip/` library, exposed in the `include/net/ip.h` header file, is a thin wrapper over the protocol used by the `/net/` device files.

Chapter 3

The Literate Programs

I now switch to quick descriptions of the 18 programs, and so 18 books, composing the Principia Softwarica series. I organized those programs in six different groups.

3.1 The Core system

The first group, which I call the *core system*, is made of the minimal set of programs that are needed to reach the point where the programmer can interactively launch other programs. In the case of Plan 9, this minimal set is made of the kernel, the shell, and the standard C library.

The C library is necessary because it is used by the shell. Indeed, the C library provides the necessary bridge to call the kernel from user-space programs. The library provides also memory allocation routines and a few general utility functions. In fact, a small part of the C library is also used internally by the kernel.

3.1.1 The kernel: 9pi

A *kernel* is a program that manages all the other programs. It is arguably the most important program; without the kernel no other program can run. The kernel provides the main abstractions of the computer and multiplex its resources (e.g., the processor, the memory, the input and output devices) to multiple programs at the same time.

The kernel is the biggest program and so the biggest book in the series. The Plan 9 kernel is called simply 9, but I will focus on a variant of the kernel for the Raspberry Pi called 9pi.

3.1.2 The core libraries: libc, libthread, libbio, libflate, libregexp

The Plan 9 C library `libc` defines data structures and functions that are used by all the other programs (including the kernel). In the case of Plan 9, this library contains the memory allocation routines (`malloc()`, `free()`), the bridge to call the kernel (the system calls), the Unicode functions, and many other utility functions. Note that some of the code in the C library is written in assembly.

In addition to `libc`, a few other core libraries are used by many programs: `libthread` for concurrency, `libbio` for buffered IO, `libflate` for compression, and `libregexp` for regular expressions, that I will also present.

3.1.3 The shell: rc

A *shell* is a program that allows a user to launch other programs. It is the primary user interface of Plan 9, the so called *command-line interface* (we will see later another interface).

In Plan 9, and also in its ancestor UNIX, the shell is a regular user-space program; it is not part of the kernel. The user can actually change shell without changing the kernel.

The shell will be the first user program I will describe in the Principia Softwarica series. It illustrates many features provided by the kernel, because the shell is using internally many system calls (e.g., `rfork()`, `exec()`, `chdir()`, `pipe()`).

The Plan 9 shell is called `rc` (for run command).

3.2 The development toolchain

Once you have a terminal where you can launch programs, you will need tools to produce those binary programs from source code. The *development toolchain* is a set of tools working together to produce executables. This toolchain is made essentially of a compiler, an assembler, and a linker.

3.2.1 The C compiler: 5c

A *compiler* is a program that transforms source code written in one high-level language (e.g., C), into another lower-level language (e.g., assembly). The C compiler is probably the most important program in Plan 9 after the kernel. Indeed, the Plan 9 kernel itself is written in C and so needs a C compiler to become executable by a machine. In fact, almost all programs in Principia Softwarica are written in C, including the compiler itself¹, so they all need the C compiler to become executables.

I will describe 5c, a C compiler targeting ARM assembly. The 5 in 5c comes from the Plan 9 convention to name architectures with a single character ('v' is MIPS, '5' is ARM, '8' is x86, etc). This is why the ARM assembler and linker I will describe later are called respectively 5a and 5l.

I chose the ARM architecture because it is one of the simplest architecture while being also one of the most used architecture in the world. Indeed, almost every phone contains an ARM processor. It is also the processor of the extremely cheap Raspberry Pi a machine used by many electronic hobbyists. Thus, ARM is a great candidate for my teaching purpose.

3.2.2 The assembler: 5a

An *assembler* is a kind of compiler; it translates source code written in an assembly programming language into machine code, or into an object code close to machine code. Some of the kernel code, as well as code in the C library, are written in assembly.

To be consistent with the compiler, I will describe 5a, the Plan 9 ARM assembler.

3.2.3 The linker: 5l

A *linker* is a program taking as input multiple files called the *object files*, which contain object code generated either by the assembler or compiler, and creates the final executable. In theory, compiling, assembling, and linking could be done by a single program; the C compiler could take as input multiple C source files and produce directly an executable². However, from a software engineering point of view, it is better to separate concerns and have three separate tools for those three separate tasks.

I will describe 5l, the Plan 9 ARM linker.

3.2.4 The processor emulator: 5i

An *emulator* is a program that acts like a computer and can interpret another (binary) program. An emulator is not really a part of the development toolchain. However, because the assembler, linker, and compiler target an architecture, it is useful to describe the instruction set of an architecture (ISA) through its emulator. Moreover,

¹Appendix B describes how to solve this self-reference issue.

²Actually `gcc` can be used that way.

an emulator can be extremely useful when transforming a compiler in a *cross compiler*, to test quickly the generated machine code.

I will describe `5i`, an ARM emulator, which we can view as a kind of interpreter for a low-level language, hence the `i`. In fact, `5i` can also be used as a (slow) debugger and profiler.

3.3 The developer tools

The *developer tools* are a set of tools that are not strictly necessary to produce programs, like the software development toolchain I have described in the previous section, but which are useful in the software development process. Those tools are the text editor, the build system, the version control system, the debugger, and finally the profiler.

3.3.1 The text editor: `ed`

A *text editor* is a program to help read and write text, including source code. It is perhaps the most important tool for a programmer. Indeed, even if the kernel and compiler are essentials to run and produce software, their source code was first entered in a computer by a programmer using a text editor. In fact, text editors are used to enter the code of the text editors themselves, leading to self-reference issues just like for a compiler³. Plan 9 has a few text editors: `ed`, `sam`, and `acme`. I chose to present `ed`, which was originally written by Ken Thompson in the 1970s for UNIX. `ed` is a line editor, as opposed to screen editors like `sam`, `acme`, `vi`, or Emacs.

The main reason I chose `ed` is that it is small enough (under 2000 lines of C) to fit in one appendix. This makes `ed` a perfect candidate to illustrate literate programming in a concrete way, complementing the abstract presentation in Section 1.5. The complete source code of `ed` is presented in Appendix C.

3.3.2 The build system: `mk`

A *build system* is a program to help automate the compilation of programs by calling the appropriate tools from the development toolchain. The whole Plan 9 operating system can be built from scratch with one simple command, `mk all`, thanks to the build system called `mk` and a few configuration files (the `mkfiles`).

3.3.3 The version control system: `git9`

A *version control system* (VCS) is a program to store and retrieve past versions of a project, and to record who made each change, when, where, and why.

The Plan 9 authors didn't use originally a VCS to store the code of Plan 9 and so Plan 9 didn't have any VCS for a long time. They relied instead on powerful snapshot-based filesystems allowing to retrieve easily past versions of a file. However, in the modern age VCSs are useful not only to retrieve past versions of a file but also to help people collaborate with each other and to support concurrent and parallel development. Fortunately, in 2021 Ori Bernstein decided to rewrite Git, the most popular open source VCS, for Plan 9 and redesigned the program to use far less code. I will present his Git port called `git9`, as well as the code of simplified versions of `diff` and `patch` he wrote.

Note that the fork of Plan 9 used for Principia Softwarica is using Git and can be cloned from [git://github.com/aryx/principia-softwarica/](https://github.com/aryx/principia-softwarica/).

3.3.4 The debuggers: `acid`, `ratrace`, `db`

A *debugger* is a program that controls and inspects another program. Programming is so difficult that invariably we make mistakes when writing code. Having tools to help find those mistakes is essential, and the debugger is

³See Appendix B for discussions on how to bootstrap an editor.

the most important of those tools. Even if many programmers, including great programmers [Sei09], use just `printf()` tracing instructions to debug their programs, I think a debugger can greatly accelerate the time it takes to find bugs. Debuggers are also great tools to help understand programs, especially programs written by other people.

Plan 9 has a few tools to help debug programs: `db` the original debugger coming from UNIX, `ratrace` to trace other programs (similar to `strace` in Linux), and `acid` a *programmable debugger*. I will describe all of them but mostly focus on `acid` as well as the `libmach` library that abstracts over the executable file format and the architecture-specific machine details (registers, stack frames, binary instructions), and on which all those debuggers rely.

3.3.5 The profilers: `prof`, `tprof`, `stats`, `iostats`

A *profiler* is a program that generates statistics about another program. It is mainly used to optimize code by spotting where the program spends most of its time. It can be useful also to find bugs because unexpected statistics can sometimes be good hints to fix code where the programmer was expecting different results [Ben88]

Profiling is not a single problem but a spectrum of needs: timing a command, identifying hot functions, or monitoring system-wide resource usage. Plan 9 addresses each with a small focused tool: `prof` and `tprof` for function-level profiling, `stats` for live system monitoring, and `iostats` for I/O cost analysis. I will describe all of them.

3.4 Graphics

Up until now, I have mostly described command-line tools interacting with the programmer through simple text-based terminals. However, one of the most important inventions in computer science was the *graphical user interface* (GUI), introduced in the 1970's with the Xerox Alto [TML⁺79]. Even command-line text-based programs benefit from a graphical user interface as you can run multiple programs in different windows at the same time like in Figure 1.1 on page 8.

3.4.1 The graphics stack: `/dev/draw`

A *graphics stack* provides the basis on top of which graphical applications can be built. GUI elements such as menus, windows, cursors, or texts are ultimately rendered on the screen using simple graphic operations provided by the graphics stack: lines, circles, rectangles, arcs, etc.

Under Plan 9, the graphics services are accessible through the `/dev/draw/` device directory, which is connected to the screen device driver in the kernel. On the user side, applications do not talk to `/dev/draw/` directly but through `libdraw`, which provides drawing primitives and event handling, and `libimg`, which reads and writes standard image file formats (PNG, JPEG, GIF). On the kernel side, the `/dev/draw/` driver itself relies on `libmemdraw` for in-memory bitmap operations and `libmemlayer` for layer stacking and clipping.

The Principia Softwarica book for the graphics stack will contain both the kernel-side and user-side code, with all the libraries mentioned above. It will complement the KERNEL book [Pad14].

3.4.2 The windowing system: `rio`

One of the most important graphical applications is the *windowing system*. In some sense, a windowing system is an extension of the kernel and the shell; it is a program that also manages and launches other programs, which are represented visually by separate windows.

The Plan 9 windowing system is called `rio`. It is essentially a multiplexer of the `/dev/cons`, `/dev/mouse`, and `/dev/draw` device files. `rio` is a file server that internally uses the keyboard, mouse, and screen physical devices, and provides a virtual keyboard, virtual mouse, and virtual screen to its windows using views of those

same device files (thanks to the per-process namespace feature in the kernel). In fact, an unusual feature of `rio` is that it can even be run inside itself.

3.4.3 The graphical user interface toolkit: `libpanel`

A *GUI toolkit* provides the high-level *widgets* from which graphical applications are built: buttons, menus, sliders, scroll bars, and the layout managers that arrange them on screen. A toolkit turns drawing primitives (lines, rectangles, text) and raw input events (mouse clicks, key presses) into reusable interactive elements.

There are a few Plan 9 libraries to create widgets but I will focus on the most complete widget library: `libpanel`.

3.5 Networking

Up until now, I have described programs that can run on isolated machines. I will now switch to programs that can communicate with each other on different machines.

Just like the graphical user interface, *networking* (and internetworking, also known as the Internet) was one of the most important inventions in computer science, pioneered also by the Xerox Alto [TML⁺79]. It has led ultimately to the creation of the Web where networking programs (Web browsers and Web servers) made possible great things, for instance, the worldwide collaborative project Wikipedia.

3.5.1 The network stack: `/net`

A *network stack* is a part of the kernel, usually fairly large, that provides the necessary abstractions for programs to communicate with each other on different machines, by using different *protocols* (TCP, UDP, etc.).

Under Plan 9, the network services are accessible through the `/net/` device directory. Similar to the book about the graphics stack, the Principia Softwarica book for the network stack will contain both kernel-side and user-side code and will complement the KERNEL book [Pad14]. The book will also present the Plan 9 specific protocol 9P which allows to implement file servers.

3.5.2 The web browser: `mothra`

There are far too many networking applications to present in a single book—Email, Usenet news, FTP, telnet, and IRC, all played a role in the early Internet. I will focus on the one that has eclipsed all the others: the Web browser.

A *Web browser* is a program that fetches *hypertext* documents over the network and renders them on the screen. A modern Web browser can read mail, transfer files, chat, play video, and much more; it has grown so versatile that some treat it as an operating system on its own.

Plan 9 has a few experimental Web browsers but I will present only `mothra`, the simplest one. It is using the `libpanel` GUI library I mentioned earlier. `mothra` is much more limited than Firefox or Chrome—no JavaScript, no CSS—but its few thousand lines of C are small enough to read end-to-end and to understand how a browser is structured. In fact, `mothra` is powerful enough to fetch and render (mostly correctly) the Principia Softwarica website at: <https://www.principia-softwarica.org/>

3.6 Utilities: `cat`, `ls`, `grep`, `sed`, `cmp`, `tar`, `gzip`, `bc`, `dc`, `hoc`, `awk`, ...

In addition to the programs I mentioned previously, a programmer very often uses small utilities to perform or automate certain tasks. The code of those utilities is usually pretty small because those utilities have often a single and simple function. One of the Principia Softwarica books will be dedicated to describe the code of a few of those utilities, the most important ones for the programmer.

File and directory utilities

The shell has very few *builtins* (e.g., `cd`); to create, delete, or modify files or directories, a programmer needs to use special programs. Those programs are essentially small wrappers around the file and directory system calls provided by the kernel.

Here are the file and directory utilities I will describe:

- `touch` and `mkdir`
- `cat` and `ls`
- `rm`, `cp`, and `mv`
- `chmod` and `chgrp`

Text processing utilities

Source code is stored in text files, which are made of a set of lines, which are a set of strings (words). It is thus normal that string-processing utilities are very often used by programmers to search, modify, or compare source code. Here are the string-processing utilities I will describe:

- **grep**: It is one of the most versatile and useful tool for a programmer. It can be used to find code using regular expressions.
- **sed**: It can be used to help refactor code.
- **awk**: It generalizes the regexp-driven idea of **grep** and **sed** to a small pattern-action language that handles not just strings but also numeric computations, making it a very versatile and useful tool for a programmer (as well as for Coding agents).

Process utilities

Source code ultimately is transformed into binary programs, which ultimately become processes when run. A programmer needs a set of utilities to manipulate those processes. Under Plan 9, the `/proc` directory can be used to inspect and manipulate those processes. An alternative is to use command-line programs that are often small wrappers over `/proc`.

Here are a few process utilities I will describe:

- `ps` and `pstree`
- `kill`

Archive utilities

Once a program has been written, a programmer often wants to share it with the world. In this case, he can count on a few utilities to *package* and *compress* code.

Here are a few utilities used to help disseminate programs:

- `tar`
- `gzip`

3.7 Applications

I decided to limit Principia Softwarica to system programs, and to the system programs that are the most relevant to the programmer. I do not cover applications such as spreadsheets, word processors, calendars, email clients, or video games. However, I encourage other people to find applications with small codebases (using minimalist approaches), and to write and publish their literate programs.

Chapter 4

Conclusion

I hope the Principia Softwarica books will greatly consolidate your computer science knowledge, and give you a better and more complete picture of what is going on in your computer.

I think the Principia Softwarica programs form together the minimal foundation on top of which all applications can be built. Even though there are 18 books in the series, I still think it is the minimal foundation. Indeed, it is hard to remove any of those programs because they depend on each other. First, you need to rely on a kernel (hence the name), but the shell and the C library are also essential. However, because those programs are coded in C and assembly, you also need a C compiler and an assembler, and because source code is usually split in many files you also need a linker. To write all this code in the first place you need an editor. Then, with so many source files you need a build system to automate and optimize the compilation process, and a version control system to keep track of changes to those files. Because the programs I just mentioned inevitably have bugs or non optimal parts, you will also need a debugger and a profiler. Finally, nowadays it is inconceivable to not use a graphical user interface and to not work with multiple windows opened at the same time. In the same way it is also not conceivable to work in isolation; programmers collaborate with each other, especially via the Web. This means you need a graphical and networking stack as well as a windowing system and a Web browser. As I said earlier, it is hard to remove any of the programs from the series.

I hope those books will answer many of your questions, even those that seem very simple at first such as “What happens when the user type `ls` in a terminal window?”. The answer to this question involves many software layers (the shell, the C library, the kernel, the graphics stack, and the windowing system) and lots of code.

The books in the series can be read mostly in any order. You do not have to read them all. I recommend to pick the program you are the most interested in, for instance, the one you are the most curious about because you have only a vague idea of how they are implemented, and read the corresponding book in the series.

Enjoy!

Appendix A

Other Teaching Operating Systems

Here are a few operating systems that I considered for Principia Softwarica, but which I ultimately discarded:

- UNIX V6¹ (Ken Thompson et al.), fully commented in the classic book by John Lions [Lio77], or its modern incarnation xv6², are great resources to fully understand a UNIX kernel. However, this kernel is too simple; there is no support for graphics or networking for instance.
- XINU³ (Douglas Comer), fully documented in two books [Com84, Com87], has a network stack, but the kernel is still too simple with no virtual memory for instance.
- Minix⁴ (Andrew Tannenbaum et al.), also fully documented [Tan87], is fairly small, but it is just a kernel. Minix does not provide for instance its own windowing system; it relies instead on X Window, which is far more complicated than the Plan 9 windowing system.
- Hack⁵ (Noam Nisan and Shimon Shoken) is a toy computer introduced in the excellent book *The Elements of Computing Systems* [NS05]. This book is great for understanding processors, assemblers, and even compilers, but the kernel part is really too simple.
- MMIX⁶ (Donald Knuth) and its ancestor MIX are computers designed by Donald Knuth and used in his classic book series *The Art of Computer Programming* [Knu73]. Donald Knuth also wrote a book using literate programming, *MMIXware* [Knu99], to explain the full code of the MMIX simulator and assembler. However, similar to Hack, very few programs have been written for this machine. For instance, the book assumes the presence of a kernel called NNIX, but nobody has ever written it.
- STEPS⁷ (Alan Kay et al.) is a project to reinvent from scratch programming. It has a far more ambitious goal than Principia Softwarica: write a full operating system in 20 000 LOC. It was unfortunately never finished.
- Oberon⁸ (Niklaus Wirth et al.) is a kernel, compiler, and windowing system designed from scratch. It is a great operating system, very compact, and fully documented in a single book [WG92]. However, it imposes strong restrictions on the programmer: only applications written in the Oberon programming language can be run. This simplifies many things, but operating systems like UNIX (and Plan 9) are more universal; they can run any program in any language, as long as the program can be interpreted or compiled into a binary.

¹<http://minnie.tuhs.org/cgi-bin/utree.pl>

²<http://pdos.csail.mit.edu/6.828/2014/xv6.html>

³<http://minnie.tuhs.org/cgi-bin/utree.pl?file=Xinu7>

⁴<http://minnie.tuhs.org/cgi-bin/utree.pl?file=Minix1.1>

⁵<http://www.nand2tetris.org/>

⁶<http://www-cs-faculty.stanford.edu/~uno/mmix-news.html>

⁷<http://vpri.org/html/writings.php>

⁸<http://www.projectoberon.com/>

- TempleOS⁹ (Terry A. Davis) is an operating system single handedly created over a decade. It contains a kernel, a windowing system, a compiler for a dialect of C, and even some games. It has graphics capabilities but there is no network support.
- Inferno¹⁰ (Winterbottom et al.), introduced in [DPP⁺97] and heavily documented in [Stu08], was the successor of Plan 9, designed and written mostly by the same team. It introduced many innovations: the ability to also run on a hosted OS, a virtual machine (VM) [WP97], a new concurrent language Limbo [SM03] (similar to Go that came later) using garbage collection, a new Web browser, etc. Many of the Plan 9 C programs were also ported to Limbo. Inferno is in some sense better than Plan 9 but it is also more complicated. For example, basing Principia Softwarica on Inferno would now require to present Limbo, its compiler, virtual machine, etc.

Another candidate for Principia Softwarica was the combination of the GNU system¹¹, the Linux kernel¹², and the X Window graphical user interface Xorg¹³. However, GNU/Linux/Xorg together are far bigger than Plan 9. If you take the source code of the Linux kernel, the GNU C library (`glibc`), the `bash` shell, the GNU C compiler (`gcc`), the GNU assembler (`gas`) and linker (`ld`) part of the `binutils` package, the GNU Lex and Yacc clones (`flex` and `bison`), the Emacs editor, GNU `make`, the VCS `git`, the GNU debugger (`gdb`), the GNU profiler (`gprof`), and the X Window system (`Xorg`), you will get orders of magnitude more source code than Plan 9, even though Plan 9 provides in essence the same core services. In fact, almost all of the programs above use individually more source code than the whole Plan 9 system.

Of course, the Linux kernel contains thousands of specific device drivers, `gcc` handles a multitude of different architectures, and `Xorg` supports lots of graphic cards. All of those things could be discarded when presenting the core of those programs. However, their core is still far bigger than the equivalent core in Plan 9 programs.

⁹<http://www.templeos.org/>

¹⁰<https://www.inferno-os.org/>

¹¹<http://www.gnu.org/>

¹²<http://www.kernel.org/>

¹³<http://www.freedesktop.org>

Appendix B

Bootstrapping from Scratch

It is currently fairly easy to install Plan 9 on a new machine, as explained in Section 1.6. This is because there are already in the world computers running operating systems that include executable C compilers and text editors, with standard formats for data as well as compatible storage devices. This is also because the source code of Plan 9 can easily be downloaded on the storage device of those computers through the network. All of this makes it easy to compile (or cross compile) Plan 9 on one machine and install it on the disk of a new machine that can be booted on. In fact, you can even use an emulator like Qemu and run Plan 9 from the same machine.

However, what if we had to start from scratch?

- What if there was no executable C compiler? As mentioned in Section 3.2.1, the C compiler I describe in Principia Softwarica is itself written in C. This self-reference leads to a *chicken and egg* problem: How was compiled the code of the first compiler?
- What if there was no digital text of Plan 9 and no text editor? If the only representation of Plan 9 was the printed Principia Softwarica books, the source code of Plan 9 would have to be entered first in a computer. But without a text editor, how to enter and save text in a computer? How was entered the source code of the first text editor?
- What if there was no kernel? How to interact then with a new machine? If most programs are loaded into memory by the kernel, which can be seen as an interactive program loader, how was loaded in memory the first loader?

In fact, the answer to all those questions is a technique called *bootstrapping*. According to Wikipedia, “bootstrapping in general refers to the starting of a self-sustaining process that is supposed to proceed without external input”. In our case, bootstrapping is a process where you start from something simple and gradually build something self-sustaining and more complex.

In the book *The Knowledge: How to Rebuild our World from Scratch* [Dar14], its author Lewis Dartnell imagines our world after an apocalypse and describes the key knowledge one needs to start rebuilding civilization from scratch. He does not reach though the computer and software age. What if after this apocalypse there was no more software? What if the only programs we had was the printed Principia Softwarica books retrieved from a time capsule? Would that be enough to bootstrap Plan 9? The kernel, compiler, and editor being mutually dependent on each other, it is difficult to imagine where to start. Which program to write first? For which machine? In which language? And how to enter this program in the machine?

In the following sections, I outline a bootstrapping process for Plan 9 where I start from scratch. I think it is an interesting intellectual exercise. By scratch I mean with no existing software, but because in Principia Softwarica we are interested in software and not hardware, I assume the existence of a basic computer though.

B.1 A basic computer

Software and hardware need each other in order to be useful, but hardware is the concrete starting point. Before executing programs, we first need a physical machine, a computer.

The main components of a basic computer are as follows:

- A processor: This is an interpreter for a simple low-level language where instructions are encoded in a binary format.
- Some memory: This contains data but also the code of programs. The *stored-program* concept was one of the major inventions of computer science.
- Input and output devices: This is used for interactivity with the user.

To learn how to build a simple processor and memory, I recommend to read [NS05] in which starting only from the **nand** logic gate, you learn how to build gradually the **and**, **or**, and **not** logic gates, a multiplexer, flip-flops, memory banks, an adder, an arithmetic and logic unit (ALU), and finally a simple central processing unit (CPU).

Regarding the external devices, a basic computer typically has three:

- An input device, e.g., a keyboard, for reading data. The press of a key can trigger an interrupt in the processor that will trigger the execution of a special program. The value of the key could be read at a certain memory location (memory-mapped IO) or via a special instruction. Before keyboards, *switches* on *panels* or *punched cards* with card readers connected to the computer were the main ways to enter data in a computer.
- An output device, e.g., a screen, for displaying data. Again the device could be memory-mapped, in which case the writing of data at certain memory locations would trigger the display of special characters on the screen. Before screens, diodes on panels or printers connected to the computer were the main ways to display data from the computer.
- A storage device, e.g., a magnetic tape with its tape drive, to permanently store data, for instance programs in executable forms and source forms. Again, access to this device could be provided by special instructions or by writing and reading certain codes at certain memory locations.

All those things can be built from scratch. They are not trivial to build, but they do not require any software; they are physical artifacts.

B.2 A booting procedure

Once you have a basic computer, an important question is what happens when a human presses the On/Off button of the computer? The initialization of a computer system is called *booting*, which is the shortened form of the word bootstrapping. The machine needs to initialize itself and execute the first program from memory. If most programs are loaded into memory by other programs already running on the computer, what is the mechanism to load the very first program?

There are multiple ways to load an initial program in memory. The very first computers used switches on panels where the initial configuration of memory could be manually toggled, or at least part of the memory could be toggled. Then the computer when turned on, by construction, would start to execute the code located at a specific memory address, e.g., `0x10`. This initial program, usually small, is called the *boot loader*. Its job is just to load a larger program stored on the main storage device: the kernel (which itself will load other programs).

Later, computers used punched cards and a card reader connected to the computer to load automatically their first program in memory. Modern computers use special read-only memory (ROM) chips to store the initial program. Such programs are also called *firmware*, because they are in the middle between the hard and soft.

In any case, all those techniques are similar: a program is hardwired at a persistent location (panel, punched card, tape, ROM) and loaded in memory at a specific address by the computer when the human presses a special button. The computer then jumps to this memory address and starts executing the program.

B.3 Physical programs

We have just seen different ways to enter programs in a computer without a text editor; switches and punched cards are ways to write data, and so also programs, in the physical world. Thanks to panels and card readers connected to the computer, those physical programs are made machine readable.

Different kinds of punched cards and card readers can be used to write different kinds of information:

- Different holes for different bits can be used to represent binary data such as machine instructions.
- Different holes for different letters or numbers can be used for textual data used by programs.
- Different holes for different instructions of specific languages can be used for the source code of languages such as assembly.

Note that a card reader (or a keypunch machine) could be built in such a way to behave like a primitive mechanical assembler. The holes in the punched card could correspond to different assembly mnemonic instructions and be transmitted to the computer as the assembled binary machine instructions.

Once those physical programs are loaded into memory, they can be stored back on magnetic tapes thanks to special computer instructions.

We now have everything we need to start producing software:

- A basic computer, which can be seen as a machine-language *physical interpreter*. This will allow us to bootstrap the compiler.
- A way to enter programs in the computer, with punched cards, a keypunch machine, and a card reader, which can be seen together as a *physical text (and binary) editor*. This will allow us to bootstrap the text editor.
- A simple booting procedure which will load the boot loader from a fixed *physical location*. This will allow us to bootstrap the kernel.

The following sections describe the process to bootstrap Plan 9 from scratch by writing a series of programs. The process is decomposed in different phases depending on the programming language used.

B.4 Phase 1: Machine code

BOOT-M-CARD

The very first program to write is a boot loader, which I call **BOOT-M-CARD**. **M** stands for machine code and **CARD** for punched card. The first program has to be written directly in machine code and entered in the computer via a punched card as there is no assembler and no text editor yet. The bits of the instructions of this program can be encoded as holes in the punched card. This card can then be inserted in the first position of the card reader and loaded at boot time by the computer according to its booting procedure.

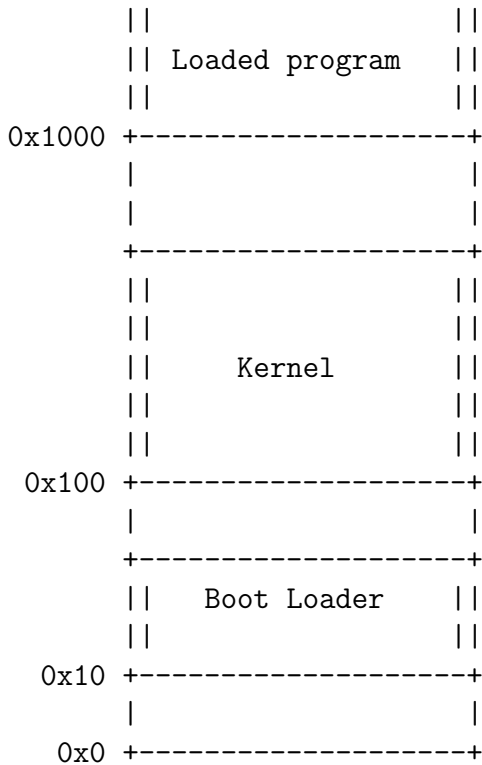
One card should be enough to encode a very simple boot loader. With just a few instructions, this program could load in memory the rest of the punched cards from the card deck (or data from the tape), and so bootstrap an hypothetical kernel. It must load this code after its own code though, to not overwrite itself, e.g., at address `0x100`, and then jump to this address.

To create **BOOT-M-CARD**, we could have created first **BOOT-ASM-PAPER**, the boot loader program written in assembly on paper. Then, thanks to a very powerful (but error-prone) computer and assembler, the human¹, we could have translated this program in binary and then generate **BOOT-M-CARD**.

KERNEL0-M-CARD

The next program to write is a simple kernel, which I call **KERNEL0-M-CARD**. It is essentially an interactive program loader. With this loader, you could type on the keyboard after a prompt the location of a program in order to get this program loaded in memory and jumped to.

The organization of different programs on the tape or in the card reader can be trivial: the different programs could be stored one after the other. The location of a program could then be specified simply by a letter and two numbers: the letter to select the card reader or tape drive, the two numbers for the start and end locations of the program on the tape or in the card deck. Moreover, by convention programs could be loaded at the memory address **0x1000** and executed from there. The last instruction of a program could be to jump to the address **0x100** to give back control to the kernel. The following diagram describes the memory layout after a program was loaded by the kernel:



There is no need yet for a real filesystem or an executable format. The code of this kernel can be very simple; it should require only a few punched cards. Those cards can then be placed just after the punched card of the boot loader in the cards deck.

ASSEMBLERO-M-CARD

The final program to write in machine code is a rudimentary assembler, which I call **ASSEMBLERO-M-CARD**. Again, we could write first **ASSEMBLERO-ASM-PAPER** and then translate this program in machine code by using a human assembler.

Because we do not have yet a text editor, this assembler when loaded could ask first a series of questions to the programmer, and then ask interactively to the programmer to enter via the keyboard the full content of

¹Not just its brain, but also its powerful IO devices: the hands that can type on a keypunch machine.

the assembly program (without mistakes). The assembler could also take as input the content of an assembly program from cards or the tape, depending on the answer to the initial questions. Another question could be used to specify where to save the generated (assembled) machine-code program.

The punched cards of this assembler program can be put just after the cards of the kernel in the card deck. You can then count the number of cards in the deck to know the location numbers to enter in the prompt of the kernel to load this assembler program in memory.

B.5 Phase 2: Assembly

Thanks to `BOOT-M-CARD`, `KERNEL0-M-CARD`, and `ASSEMBLER0-M-CARD`, we can now enter via the keyboard programs in assembly, a major productivity improvement over punched cards and binary machine-code.

`EDITOR0-ASM0-KBD`

For improving even more productivity, the first assembly program to write is an editor, which I call `EDITOR0-ASM0-KBD`. It is written in the rudimentary assembly `ASM0` supported by the rudimentary assembler `ASSEMBLER0-M-CARD`, and bootstrapped via the keyboard input of the assembler. The assembler can then generate `EDITOR0-M-TAPE`, an executable editor, which can be loaded from tape.

Thanks to this editor, further programs can be entered via the keyboard, edited, and saved on tape. We can make mistake and fix the mistake easily. From now on I assume every programs will be entered via the text editor and saved on tape, so there is no need anymore for the `CARD`, `TAPE`, or `KBD` suffixes in the program names.

`ASSEMBLER1-ASM0`, `ASSEMBLER2-ASM1`, etc

Thanks to `ASSEMBLER0-M` (previously called `ASSEMBLER0-M-CARD`), we can now bootstrap an assembler written in itself: `ASSEMBLER1-ASM0`. In fact, we can write a series of increasingly powerful assemblers. Each assembler can add features, be assembled by the previous generation assembler, which will generate a new binary assembler, e.g., `ASSEMBLER1-M`, enabling now programmers to write assembly programs using those new features, including the assembler program itself. Hopefully at some point there will be no need for more features or optimizations and we will reach a fixpoint with an assembly language I call `ASM` from now on.

`EDITOR1-ASM`

Given this feature-rich assembly language `ASM` and assembler, we could rewrite the editor that was originally using the rudimentary assembly `ASM0`. I call `EDITOR1-ASM` this new editor. Again, we could write a series of increasingly powerful editors where each new feature added could be used to edit more quickly the code of the next version of the editor.

`KERNEL1-ASM`

We can now use a nice editor and program in a powerful assembly language. However, until this point we are still using the basic program loader `KERNEL0-M`. With this kernel, you need to enter the start and end locations on tape of a program to get it executed, which is inconvenient. Indeed, with the multitude of programs we previously created, you need to maintain a physical map of the tape to remember where the programs (in source and binary forms) are located. Moreover, the programs have to be stored contiguously on the tape. Finally, you must take care when using the editor to not create data that would overwrite another program.

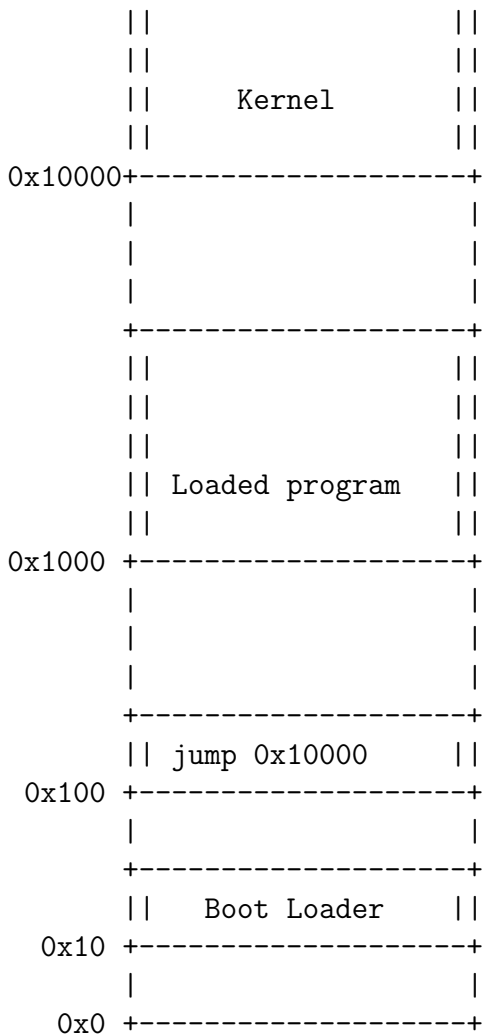
All of this management of data, currently manual and error-prone, can be done instead by the computer. I call `KERNEL1-ASM` a new kernel written in assembly with one important new feature: a *filesystem*. Transitioning to this new kernel can be facilitated by having two tape drives connected to the computer and two programs

FSINIT-ASM and CP-ASM, which can respectively initialize a new filesystem on a tape, and transfer data from one “raw” tape to another tape managed by a filesystem, and to give a filename to this data.

The start of a tape managed by a filesystem can still be reserved to contain the boot loader and the kernel. By preparing carefully the second tape with the boot loader, the new kernel, the filesystem with the previous programs (assembler, editor, kernel) in source and executable forms appropriately named, you could then switch the first tape with the second and boot the new kernel.

The interactive program loader in KERNEL1-ASM would now take after its prompt symbolic filenames instead of tape locations to execute commands. Note though that the copied text editor and assembler programs (EDITOR1-M and ASSEMBLER2-M) would still use tape locations to operate so you must take care when executing those commands to not corrupt the filesystem. One solution is to now use the second tape as a scratch and use CP-ASM to export and import data from this tape; the old editor and assembler could read and write data on this second tape without risking to corrupt the filesystem of the first tape.

Because the code required to manage a filesystem can be large, the kernel might not fit anymore in the original 0x100-0x1000 memory interval. After being loaded by the boot loader at 0x100, the kernel could copy itself in higher memory and just put at 0x100 a few instructions to jump back to higher memory. That way programs could still be loaded at 0x1000 without overwriting code from the kernel, as in the following diagram:



Note though that very large programs could still overwrite the kernel if their code size would reach the higher memory area where the kernel resides in. We will see later mechanisms to protect the kernel from such programs.

LINKER0-ASM, ASSEMBLER3-ASM-OBJ

Now that we have a filesystem, we can easily create many files. We can now split big programs in multiple files to separate concerns and also to factorize code among those programs in library files.

To create an executable from multiple files, we could create first a program **CAT-ASM** that would concatenate multiple files into one. We could then use the old program **ASSEMBLER2-M** on this single file (after it has been exported by **CP-ASM** on the scratch tape). An alternative is to create intermediate machine-code files, *object files*, and to create a *linker* I call **LINKER0-ASM** to produce the final executable from those object files. This program is more complicated than **CAT-ASM** but it enables *separate compilation*, which in the long term will save time.

To create those new object files we need to modify the assembler though and create **ASSEMBLER3-ASM-OBJ**. Moreover, we can use the opportunity to update also the interface of the assembler to now take filenames as arguments for input and output instead of tape locations.

LIB0-ASMs

For **ASSEMBLER3-ASM-OBJ** to work with filenames, code in the kernel related to the filesystem had to be copied and integrated in the assembler. Moreover, lots of code dealing with tapes, punched cards, and other devices had to be duplicated in different programs. This is unfortunate. Thanks to the linker we can now factorize code in one library source file and get the resulting object file linked in different programs.

I call **LIB0-ASMs** a core library written in assembly using multiple files (hence the **s** in **ASMs**). This library contains reusable code among all the previous programs. This library can provide functions to interact with devices, the filesystem, files, memory, etc. In some sense this library can also play the role of a kernel as it can abstract hardware and provide low-level functions to other programs. A system call is then simply a library call.

LINKER1-ASMs, ASSEMBLER4-ASMs, EDITOR2-ASMs

We can now reorganize the code of the previous assembly programs by splitting their code in multiple assembly files, to better separate concerns, and by removing the code that was factorized in the **LIB0-ASMs** library. I call those new programs **LINKER1-ASMs**, **ASSEMBLER4-ASMs**, and **EDITOR2-ASMs**. The interface of all those programs can also be changed to operate on filenames instead of tape locations.

KERNEL2-ASMs, LIB1-ASMs

KERNEL1-ASM has a convenient filesystem but its program loader is still rudimentary. You can load only one program at a time in memory (at **0x1000** by convention). You can load the editor, edit a file, quit, then load the assembler, wait it finishes, then run back the editor again, and so on. It would be nice to run the assembler command from the editor, or at least being able to switch between the editor and assembler without losing the state of the editor, or even better being able to edit files while a time-consuming assembling or linking job is running.

To do so requires a *multi-tasking* kernel where multiple programs loaded in memory at the same time called *processes* can execute concurrently. There are multiple ways to implement multi-tasking but the now dominant way is to write a *preemptive scheduler* using a *timer* interrupt and to leverage *virtual memory*. I call **KERNEL2-ASMs** a new kernel written in assembly using this technique. The first edition of UNIX was actually written in assembly².

Note that the timer and virtual memory requires hardware extensions to the basic computer I described in Section **B.1**. Thanks to virtual memory, multiple programs can still be loaded at the same (virtual) address, **0x1000**, as long as their physical addresses are different. This helps for the transition to the new kernel as old programs would still work with the new kernel. Those programs could also still call code from **LIB0-ASMs** to interact with devices, the filesystem, etc.

²<https://github.com/c3x04/Unix-1st-Edition-jun72>

The **LIB0-ASMs** library could be gradually extended to offer process related services, e.g., `fork()`, `exec()`, which would allow to program the scenario I mentioned before with the concurrent use of the editor and assembler. The program loader in the kernel could also be extended to provide advanced shell features such as pipes on the command line, job control, etc.

Protection rings are another hardware extension, often associated with virtual memory, which is very useful to have in a computer. They provide the mean to protect the kernel from regular programs and also to protect programs from each other when combined with virtual memory. In practice two rings are enough. The processor then can operate in two different modes:

- *Kernel mode.* In this mode all instructions are allowed, including the ones used to modify the virtual memory mapping, or the ones to interact with devices. Those instructions are called *privileged instructions*. Moreover, in kernel mode all memory accesses are allowed.
- *User mode.* In this mode many privileged instructions and access to certain memory area are disallowed. Such accesses cause *traps* in the kernel when those operations happens.

The only way to go from user mode to kernel mode (other than by causing a trap) is through a special instruction: the *software interrupt*. Calling this instruction is also known as performing a *system call* or *syscall*.

Thanks to protection rings and virtual memory, certain memory area can be marked as protected, e.g., the high memory area where resides the kernel, in which case programs can not mess anymore with the code of the kernel by overwriting its code. Moreover, we can gradually forbid programs to access directly devices such as the screen or the keyboard so that the kernel instead can mediate and control the use of those devices.

To transition to this safer model, we can gradually modify **LIB0-ASMs** and move routines using privileged instructions (dealing with memory, devices, the filesystem, processes, etc) to the kernel and give access to those routines via a system call (instead of a library call) to user programs. I call **LIB1-ASMs** this new library, which should be significantly smaller than **LIB0-ASMs**. Once all the privileged code has been migrated to the kernel, we can turn on the protection rings and forbid the execution of any privileged instructions in user programs.

COMPILERC0-ASMs

The final program to write in assembly will enable the transition to a higher-level language: C. It is a compiler written in assembly for a simple subset of C. I call this program **COMPILERC0-ASMs**.

B.6 Phase 3: C

Thanks to the previous programs, we now have a basic UNIX environment with a simple C compiler, all written in assembly. It is now time to rewrite this environment in a more succinct and readable way using C.

COMPILERC1-C0, COMPILERC2-C1, etc

Thanks to **COMPILERC0-M** we can now bootstrap a C compiler written in itself: **COMPILERC1-C0**. In fact, just like for the assembler and editor, we can write a series of increasingly powerful compilers: each compiler can add features, be compiled by the previous generation compiler generating a new binary compiler, e.g., **COMPILERC1-M**, enabling now programmers to write C programs using those new features, including the compiler program itself. Hopefully at some point there will be no need for more features and we will reach a fixpoint with a C language I call **C** from now on.

KERNEL3-C/ASM, LIB2-C/ASM

We can rewrite lots of the core library in C. Indeed, the C language being very flexible and powerful, many of the assembly code idioms can be expressed in C, and usually in a clearer and shorter way. However, some code has

to be kept in assembly, which is why I call this new library **LIB2-C/ASM**. Some low-level code, such as the system call instructions that allows to jump in the kernel, can not be expressed in C. Moreover, it can be preferable sometimes for heavily used routines to keep code written in highly-optimized assembly. Those routines though can be exposed to other programs with a C interface, e.g., in a `libc.h` header file. Those other programs can then be written entirely in C.

The kernel itself can also be rewritten in C and assembly in a program I call **KERNEL3-C/ASM**. The fourth edition of UNIX³ was the edition where most programs, including the kernel, were rewritten in C. At the time, writing a kernel in a high-level language such a C was controversial.

EDITOR3-C, LINKER2-C, ASSEMBLER5-C

We can rewrite most of the previous assembly programs entirely in C: the editor (**EDITOR3-C**), the linker (**LINKER2-C**), and even the assembler can be rewritten in C (**ASSEMBLER5-C**).

B.7 Summary of Plan 9 ancestor programs

I summarize here the list of programs that can be viewed as ancestors to programs explained in the Principia Softwarica series. Those programs would be necessary to bootstrap Plan 9 from scratch:

- Boot loader: **BOOT-ASM** (translated in **BOOT-M-CARD** by a human assembler). The Plan 9 kernel 9 described in Section 3.1.1 includes a boot loader fairly similar to **BOOT-ASM**.
- Kernels: **KERNEL0-ASM** (translated in **KERNEL0-M-CARD** by a human assembler), **KERNEL1-ASM**, **KERNEL2-ASMs**. This led finally to **KERNEL3-C/ASM**, which can be a close ancestor to 9 described in Section 3.1.1. We could derive **KERNEL3-C/ASM** from the source code of 9 by just removing many features.
- Core library: **LIB0-ASMs**, **LIB1-ASMs**. This led to **LIB2-C/ASM**, which can be a close ancestor to `libc` described in Section 3.1.2.
- C compilers: **COMPILERC0-ASMs**, **COMPILERC1-C0**, **COMPILERC2-C1**. This led finally to **COMPILERC3-C/LY**, which can be a close ancestor to 5c described in Section 3.2.1.
- Assemblers: **ASSEMBLER0-ASM** (translated in **ASSEMBLER0-M-CARD** by a human assembler), **ASSEMBLER1-ASM0**, **ASSEMBLER2-ASM1**, **ASSEMBLER3-ASM-OBJ**, **ASSEMBLER4-ASMs**, **ASSEMBLER5-C**. This led finally to **ASSEMBLER6-C** which can be a close ancestor to 5a described in Section 3.2.2.
- Linkers: **LINKER0-ASM**, **LINKER1-ASMs**, This led finally to **LINKER2-C**, which can be a close ancestor to 5l described in Section 3.2.3.
- Editors: **EDITOR0-ASM0**, **EDITOR1-ASM**, **EDITOR2-ASMs**, **EDITOR3-C**.

This led finally to **EDITOR3-C**, which can be a close ancestor to `ed` described in Section 3.3.1.

³<http://minnie.tuhs.org/cgi-bin/utree.pl?file=V4>

Appendix C

Literate Program Example: `ed`

This appendix has two purposes. First, it is a concrete example of a literate program—a complement to Section 1.5, which presented literate programming in the abstract. Reading the source of a real program presented in literate form is the best way to see what literate programming actually buys you. Second, it is a full explanation of a real Plan 9 program: `ed`, which is small enough (under 1700 lines of C) to fit in one appendix, yet old enough and famous enough that almost every modern editors and string-processing tools descend from it in some way. Reading its source code is a quick way to see how a non-trivial UNIX tool was structured in the 1970s.

Note that the original source code of `ed` has been slightly modified to be arguably easier to read. Moreover, in addition to my explanations, I have also sometimes added a few comments in the code itself. The original comments from Ken Thompson are using the standard C syntax `/**/` while my comments are using the `//` syntax, so they are easy to differentiate.

C.1 Introduction

C.1.1 A line editor

`ed`, short for “editor”¹, is one of the oldest UNIX programs, originally written by Ken Thompson in the 1970s. It is a *line editor*: rather than always displaying a file content on screen like `vi`, Emacs, or Plan 9’s `sam` and `acme`, `ed` works mostly one line at a time, accepting commands from standard input and printing results to standard output (possibly displaying lines of the edited file depending on the input commands). This makes it look primitive compared to modern *screen editors*, but it has a crucial advantage: it can be scripted. You can pipe a sequence of editing commands into `ed` from a shell script, something that is awkward or impossible with most editors.

While you can already create and modify files using `echo` and shell redirection (`>`, `>>`), `ed` offers far more power: searching, substituting, moving lines, and operating on ranges of lines. For a quick hands-on feel, here is a complete `ed` session:

```
$ ed
i
hello
world
.
1,2p
hello
world
```

¹Plan 9 and UNIX authors did like short command names: `ed`, `mk`, `rc`, `db`, `ls`, etc.

```

f hello.txt
hello.txt
w
12
q
$ cat hello.txt
hello
world
$

```

The session above launches `ed`, starts *insert mode* with `i`, types two lines, ends the insertion with a single `'.'` on its own line, prints lines 1 to 2 with `1,2p` made of an *address range* and the `p` command (`ed` echoes the 2 lines), sets the *remembered filename* with `f hello.txt` (`ed` echoes it back), saves the *buffer* with `w` (`ed` echoes the character count written), and finally quits with `q`.

`ed` is famously silent: it prints no prompt, and every line you type is either a command or part of a text-entry session opened by `i` (or `a` for *append mode*). For a full tutorial introduction to `ed`, see Appendix 1 of “The UNIX Programming Environment” [KP84], or Kernighan’s two tutorials from the Bell Labs technical reports. See also the man page of `ed` at `docs/man/1/ed` in my Plan 9 repository.

Note that many ideas that seem to “belong” to other tools actually originated in `ed`: the `s/re/replacement/` syntax used by `sed`, the `g/re/p` command that gave `grep` its name (“global regular expression print”), and commands like `w`, `q`, `a`, `i` that `vi` inherited directly. It is thus interesting to see the code of the program that started it all.

C.1.2 A scriptable editor

Here is a real example of `ed` used as a code generator during the build of Plan 9. The `mkenam` script below transforms a header file (`5.out.h`) containing assembler opcode definitions into a C source file (`enam.c`) containing a string array of opcode names:

```

⟨mkenam 36⟩≡
ed - ../51/5.out.h <<'!'
v/^ A/d
1,$s/^ A/ "/"
g/ .*$/s///
,s/,*$/"/,/
1i
char* anames[] =
{
.
$a
};
.
w enam.c
Q
!

```

To understand what this script does, it helps to see a concrete example of input and output. Here is what a snippet of the input file `5.out.h` looks like (left) and what the output `enam.c` is supposed to look like (right):

5.out.h (input)	enam.c (generated output)
-----	-----
enum As {	char* anames[] =
AXXX,	{
ANOP,	"XXX",
AAND,	"NOP",
AORR,	"AND",
AEOR,	"ORR",
AADD,	"EOR",
ASUB,	"ADD",
...	"SUB",
};	...
	};

The transformations from `5.out.h` to `enam.c` are: keep only lines starting with a tab followed by `A` (the opcode names), remove that prefix `A`, wrap each name in double quotes followed by a comma, and add the `char* anames[]` array header and enclosing braces. Each time someone modifies `5.out.h`, the `enam.c` can be regenerated automatically thanks to the `mkenam` script called from `5c/mkfile`.

The `mkenam` script uses `ed` - (quiet mode, suppressing character counts) and a shell here-document (`<<'!'`, see the SHELL book [Pad18]) to feed commands. It first deletes lines not starting with `A` (`v/^ A/d`), then uses `s///` substitutions to reshape each remaining line into a quoted C string, and finally uses `i` (`1i`, meaning insert back at line 1) and `a` (`$a`, meaning appending at the end) to wrap the result with the array declaration. By the end of this appendix, you will fully understand every command in this script (and the code behind those commands).

The key advantage of `ed` here is that the editing commands are plain text, easily embedded in a shell script or a `mkfile` rule. Try doing the same in Emacs or Visual Studio Code—you would need an Emacs Lisp program or complex key-sequence macro recording which would not be so easy and fast to replay than `mkenam`.

C.1.3 A Unicode editor

Note that the Plan 9 version of `ed` is Unicode-ready. `ed` can read, edit, and write text that contains non-ASCII characters (e.g., accented letters, chinese characters, emoji), so before we dive into its code it is worth spending a few paragraphs on how Plan 9 represents text. A full treatment lives in the LIBCORE book [Pad16b]; this primer covers just what you need to follow the rest of `ed`.

Unicode is an abstract catalog of characters. A *codepoint* is the numeric ID of a character in that catalog: `U+0041` is “A” (65 in decimal), `U+00E9` is “é” (233), and so on up to `U+10FFFF` for the rarest emoji. *UTF-8* is one way to store/transmit those code points as *bytes*; It is an *encoding*.

Plan 9 can store codepoints in two complementary formats. A *Rune* is a *fixed-width* unsigned integer (usually 32 bits) holding one codepoint; Rune arrays such as `Rune linebuf[LBSIZE]` are the in-memory format for text manipulation, the Unicode equivalent of `char linebuf[LBSIZE]` in classical old (pre-Unicode) UNIX programs. UTF-8 is a *variable-length* byte encoding where ASCII codepoints take 1 byte, Latin supplements take 2, most other scripts 3, and rare codepoints (including emoji) 4; UTF-8 lives in `char` arrays and is the format for files and terminal I/O in Plan 9, which is using UTF-8 encoding everywhere text I/O is involved.

The pros and cons of each format is discussed at length in the LIBCORE book [Pad16b], but to summarize UTF-8 is *compact*, which is why it is used when storing text on disk, while Runes are *regular*, which is why they are usually used when storing text in memory; it is easy to access the 3rd character in an array of Runes while more tricky in an array of `chars` containing UTF-8.

The bridge between the two formats is two routines from `libc`:

```
int chartorune(Rune *r/*OUT*/, char *s); // UTF-8 -> Rune; returns # bytes consumed
```

```
int runetochar(char *s/*OUT*/, Rune *r); // Rune -> UTF-8; returns # bytes written
```

Both functions copy at most `UTFmax` (usually set to 4) bytes per codepoint.

Here is the three-character string “aéb” in both formats. “é” is codepoint U+00E9, which UTF-8 encodes as the two bytes 0xc3 0xa9:

```
Rune array (in memory, e.g. linebuf[]):
+-----+-----+-----+-----+
| 0x00000061 | 0x000000e9 | 0x00000062 | 0x00000000 |   each cell = sizeof(Rune)
+-----+-----+-----+-----+
   'a'         'é'         'b'         term

      |
      | runetochar          chartorune |
      | (Rune -> UTF-8)      (UTF-8 -> Rune)
      v                        |

UTF-8 char/byte array (on disk / on the wire, e.g. iobuf):
+-----+-----+-----+-----+
| 0x61 | 0xc3 | 0xa9 | 0x62 |   4 bytes total
+-----+-----+-----+-----+
   'a'   \---- 'é' ----/   'b'
   (1B)      (2B)      (1B)
```

`chartorune()` inspects the top bits of the leading byte to determine how many continuation bytes follow—`0xxxxxxx` is a 1-byte sequence, `110xxxxx` introduces a 2-byte sequence, `1110xxxx` introduces 3 bytes, `11110xxx` introduces 4—and assembles the codepoint. `runetochar()` does the inverse, selecting the encoding by the codepoint’s value range.

The modern UNIX equivalent of Plan 9’s `Rune` is `wchar_t`, a C standard type since 1995 (declared in `<wchar.h>`). On Linux and macOS `wchar_t` is 32 bits; on Windows it is only 16 bits—a legacy of Microsoft adopting UCS-2 in the early 1990s, before it became clear that 16 bits would not fit all of Unicode.

C.2 Core data structures

Show me your code and conceal your data structures, and I shall continue to be mystified. Show me your data structures, and I won’t usually need your code; it’ll be obvious.

Fred Brooks

The data structures in `ed` are radically different from the one in modern text editors. There is no linked list of lines, no rope, no gap buffer or other fancy data structures to represent a set of lines. In fact, the content of the file being edited is not even kept in memory. Instead, all text is stored in a temporary file on disk (`tfile`^{39b}), and the only in-memory data structure is `zero`^{39f}, an array of integer offsets into that temporary file—one offset per line. Editing operations like delete and move merely shuffle entries in `zero`; the temporary file itself only grows, never shrinks.

C.2.1 The backing store: `tfname`, `tfile`, and `tline`

The temporary file is `ed`’s backing store. `tfname`^{39a} holds its path (e.g., `/tmp/e000042`), `tfile`^{39b} is the open file descriptor, and `tline`^{39c} tracks the current write offset—where the next line will be appended. Every operation

that adds text (reading a file, appending user input) writes to `tfile` and records the offset in `zero`^{39f}.

```
<globals ed.c 39a>≡ (95) 39b>
char*  tfname; // temporary filename ("/tmp/eXXXXXX")

<globals ed.c 39b>+≡ (95) <39a 39c>
    fdt tfile  = -1;

Uses tfile 39b.

<globals ed.c 39c>+≡ (95) <39b 39e>
    // current write file offset in tfile (in Rune-unit)
    int tline;
```

C.2.2 The target file: savedfile

At some point, lines from the temporary file, also called the *buffer* in `ed`’s man page, will be copied in a specific file. `savedfile`^{39e} remembers the name of the file being edited—the one specified on the command line or set by the `f` command. When you type `w`, this is the file that gets written.

```
<constants ed.c 39d>≡ (95) 41b>
    FNSIZE  = 128,      /* file name */

<globals ed.c 39e>+≡ (95) <39c 39f>
    // for w, r, f
    char    savedfile[FNSIZE];

Uses FNSIZE-1 39d.
```

C.2.3 The lines as file offsets: zero

`zero`^{39f} is the central data structure: a dynamically-growing array of `int` file offsets into `tfile`^{39b}, one per line. Lines are 1-indexed: `zero[1]` is the file offset for line 1, `zero[2]` for line 2, etc. The entry `zero[0]` is unused (always 0), which is why probably the array is called “zero”—it starts at the zero-th slot which represents “before line 1.”

```
<globals ed.c 39f>+≡ (95) <39e 40>
    ulong   nlall = 128;
    // growing_array<int(even)|int&mark|0>, initial size = (nlall+2+margin)*sizeof(int)
    // where the ints are file offsets (Rune-unit) in tfname, one per line
    int*     zero;

Uses nlall 39f.
```

The crucial insight is that many editing operations only modify this array of offsets, never the temporary file itself. When you delete a line with `d`, `ed` shifts entries in `zero` down but the text remains in `tfile`. When you read a file with `r`, `ed` appends text to `tfile` and inserts new offsets into `zero`. The temporary file only grows; it is never compacted.

Concretely, suppose the user has the following text in the buffer:

tfile (the temporary file on disk)	zero (in memory)
+-----+	+-----+
alpha\n	unused 0 6 11
beta\n	+-----+
gamma\n	index: 0 1 2 3
a deleted line still here\n	line 1 line 2 line 3
+-----+	

Three lines are visible to the user, even though the temporary file on disk also contains a fourth line at offset 17 that was previously deleted. Editing operations simply shift entries within `zero`: deleting line 2 (`beta\n`) would move `zero[3]` (the offset 11) into `zero[2]`, leaving the actual text at offset 6 in `tfile` orphaned but harmless.

The actual on-disk layout

The diagram above uses byte offsets and `\n`-terminated text because that is how most people picture a file on disk. The real layout for `tfile` differs in three ways. First, `tfile` is a stream of fixed-width Rune cells (see Section C.1.3), not UTF-8 bytes: every codepoint occupies `sizeof(Rune)` bytes (usually 4), so a 5-character line like “alpha” takes 24 bytes on disk—5 character cells plus a terminator. Second, lines are terminated by a null Rune (value 0), not by `\n`. Third, `tline`^{39c} starts at 2 (in Rune units), so the first two Rune cells of `tfile` are reserved and remain zero-filled. We will see soon the reason for it.

To make the layout concrete, here is a small two-line buffer with lines “aéb” and “c” (reusing the example from Section C.1.3) drawn at both byte and Rune granularity, with `sizeof(Rune) = 4` and little-endian Rune storage:

tfile (disk), byte granularity:							
0	4	8	12	16	20	24	28
+-----+-----+-----+-----+-----+-----+-----+-----+							
00 00 00 00	00 00 00 00	61 00 00 00	e9 00 00 00	62 00 00 00	00 00 00 00	63 00 00 00	00 00 00 00
+-----+-----+-----+-----+-----+-----+-----+-----+							
slot 0	slot 1	slot 2='a'	slot 3='é'	slot 4='b'	slot 5=\0	slot 6='c'	slot 7=\0
reserved	reserved	\----- 'aéb\0' -----/				\----- 'c\0' -----/	

Same tfile, Rune granularity:							
0	1	2	3	4	5	6	7
+---+---+---+---+---+---+---+---+							
0	0	'a'	'é'	'b'	0	'c'	0
+---+---+---+---+---+---+---+---+							
\reserv/		\-- 'aéb\0'		--/\-'c\0'-/			
		^		^			
		zero[1]=2		zero[2]=6			

zero (memory) holds Rune-slot indexes: [_, 2, 6]

C.2.4 Cursors: dot and dol

`dot`⁴⁰ and `dol`⁴⁰ are pointers into the `zero`^{39f} array. Their names match the corresponding `ed` command syntax: `dot` is the *current line* (what the user refers to as `.`), and `dol` is the *last line* (what the user refers to as `$`, “dollar”). Both are pointers, not indices: `dot - zero` gives the current line number (thanks to pointer arithmetic), and `dol - zero` gives the total number of lines. When the buffer is empty, `dol == zero`, which is why you will see often code like `dol!=zero` to test for non-empty buffer.

```
(globals ed.c 40)+≡
// ref<int> in zero[], current line pointer
int* dot;
// ref<int> in zero[], last line pointer
int* dol;
```

This is where the `1,$ address range` from the `mkenam` script in Section C.1.2 comes from. The substitute command in that script means “from line 1 to `$` (the last line), substitute ...”—and now we know that `$` is just the user spelling for `dol`.

C.2.5 Line input and output buffers: line and linebuf

There are two distinct line buffers, which can be confusing. `line`^{41a} (70 bytes of `char`) is a small output buffer used by `putchr()`^{46b} to batch `write()` system calls— UTF-8 encoded characters accumulate in `line` and are flushed when a newline is encountered or the buffer is nearly full. `linebuf`^{41c} (LBSIZE entries of `Rune`) is the main working buffer where a single line of edited text is held in `Rune` form. Functions like `getfile()`^{53c} fill `linebuf` from the input file, `putline()`^{54c} writes it to `tfile`, and `getline()`⁵⁸ reads it back. The distinction between `char` for UTF-8 I/O and `Rune` for text manipulation runs throughout the code.

```
<globals ed.c 41a>+≡ (95) <40 41c>
char    line[70];
// ref<char> in line (mark end of line usually)
char*    linp    = line;
```

Uses `line` 41a and `linp` 41a.

```
<constants ed.c 41b>+≡ (95) <39d 48a>
LBSIZE  = 4096,      /* max line size */
```

```
<globals ed.c 41c>+≡ (95) <41a 41d>
Rune    linebuf[LBSIZE];
```

Uses `LBSIZE-2` 41b.

C.2.6 Other globals: count, col, etc.

`count`^{41d} is a general-purpose counter, most often used to track the number of characters read or written (displayed by `exfile()`^{56a} in verbose mode). `col`^{41e} tracks the current output column, used by the `l` (list) command to wrap long lines (see Section C.11.2).

```
<globals ed.c 41d>+≡ (95) <41c 41e>
long count;
```

```
<globals ed.c 41e>+≡ (95) <41d 41f>
int col;
```

C.3 main()

I now switch from the bottom-up approach of the previous section to a top-down approach, starting with `main()`, the entry point of `ed`. The `main()` function initializes buffered console I/O with `Binit()` (see the `LIBCORE` book [Pad16b] with the `libbio` library and its `Biobuf` data structure), installs a signal handler (called notes in Plan 9) with `notify()`, processes command-line flags, allocates `zero`^{39f}, and finally executes the main sequence: `init()`^{44a}; `commands()`^{49b}; `quit()`^{45b}.

```
<globals ed.c 41f>+≡ (95) <41e 42a>
// console buffered input
Biobuf bcons;
```

```
<function main(ed.c) 41g>≡ (95)
void
main(int argc, char *argv[])
{
    char *p1, *p2;

    Binit(&bcons, STDIN, OREAD);
    notify(notifyf);

    ARGBEGIN {
```

```

⟨main() (ed.c) flags processing cases 43b⟩
} ARGEND

USED(argc);
⟨main() (ed.c) if - flag 42b⟩
⟨main() (ed.c) if oflag 43c⟩
else if(*argv) {
    p1 = *argv;
    p2 = savedfile;
    while(*p2++ = *p1++)
        if(p2 >= &savedfile[sizeof(savedfile)])
            p2--;
    globp = L"r";
}
zero = malloc((nlall+5)*sizeof(int*)); // BUG should be sizeof(int)
tfname = mktemp(template);

init();
⟨main() (ed.c) before commands() 93a⟩
commands();
quit();
}

```

Uses `bcons` 41f, `commands()` 49b, `globp` 47c, `init()` 44a, `mktemp()` 43e, `nlall` 39f, `notifyf()` 94b, `quit()` 45b, `savedfile` 39e, `template` 43d, `tfname` 39a, and `zero` 39f.

The most tricky part in `main()` above is the `globp = L"r"` line. When a filename argument is given (e.g., `ed foo.txt`), the filename is copied into `savedfile`^{39e} and `globp`^{47c} is set to the Unicode string `"r"`. The `L""` prefix is standard C syntax for a *wide-string literal* (see the COMPILER book [Pad16a]): where the bare `"r"` would be a `char*` (one byte plus a null terminator), `L"r"` is a `Rune*`—one `Rune` holding the codepoint for `'r'` followed by a null `Rune` terminator. Later, when `commands()` calls `getchr()`^{47d} to read user input, it will read from `globp` before reading from the console, so the first command executed is automatically `r`—as if the user had typed `r foo.txt`. This elegant trick avoids duplicating file-reading logic in `main()`.

The `sizeof(int*)` in the `zero = malloc(...)` line above is actually a bug—it should be `sizeof(int)`, since `zero` holds `int` file offsets, not pointers. The bug is harmless on architectures where pointers and integers have the same size (32-bit systems); on 64-bit systems, pointers are 8 bytes while `int` is still 4, so the allocation simply over-reserves memory without causing any incorrect behavior. Even Ken Thompson wrote slightly buggy C code.

C.3.1 ed - and vflag

By default `ed` is verbose² (`vflag = true`): it prints character counts after `r` and `w` operations (as shown in the `ed` session in Section C.1.1). The `-` flag (as in `ed - foo.txt`) suppresses this, which is what you want when `ed` is driven by a script (as in the `mkenam` example in Section C.1.2).

```

⟨globals ed.c 42a⟩+≡ (95) <41f 43a>
// verbose (a.k.a. interactive) mode
bool vflag = true;

```

Uses `vflag` 42a.

```

⟨main() (ed.c) if - flag 42b⟩≡ (41g)
if(*argv && (strcmp(*argv, "-") == 0)) {
    argv++;
    vflag = false;
}

```

Uses `vflag` 42a.

²verbose is a strong word for `ed`; even in verbose mode, `ed` remains very terse (to not say cryptic).

C.3.2 ed -o and oflag

The `-o` flag turns `ed` into a *filter* (see UTILITIES book [Pad25]): it sets `savedfile`^{39e} to `/fd/1` (Plan 9's path for stdout), so that `w` writes to standard output instead of a file. It also sets `globp = L"a"` to start this time in *append mode*, and disables verbose output (since character counts would corrupt the output stream).

```
<globals ed.c 43a>+≡ (95) <42a 43d>
// output to standard output (instead of editing a file).
// Useful when ed is used as a filter.
bool oflag;
```

```
<main() (ed.c) flags processing cases 43b>≡ (41g)
case 'o':
    oflag = true;
    vflag = false;
    break;
```

Uses oflag 43a and vflag 42a.

```
<main() (ed.c) if oflag 43c>≡ (41g)
if(oflag) {
    p1 = "/fd/1";
    p2 = savedfile;
    while(*p2++ = *p1++)
        ;
    globp = L"a";
}
```

Uses globp 47c, oflag 43a, and savedfile 39e.

C.3.3 mktemp()

`mktemp()`^{43e} called early in `main()` generates a unique temporary filename by replacing the trailing X's in the template parameter with the process ID digits. If the resulting file already exists, it tries replacing the first digit with a, b, etc.

```
<globals ed.c 43d>+≡ (95) <43a 44c>
// in Linux pid can be very long, so better to have at least 6 X (was 5 before)
// the mkstemp Linux man page recommends 6 X
char template[] = "/tmp/eXXXXXX";
```

Uses template 43d.

```
<function mktemp(ed.c) 43e>≡ (95)
/// main -> <>
char*
mktemp(char *as)
{
    char *s;
    unsigned pid;
    int i;

    pid = getpid();
    s = as;
    while(*s++)
        ;
    s--;

    while(*--s == 'X') {
        *s = pid % 10 + '0';
        pid /= 10;
    }
}
```

```

}
s++;

i = 'a';
while(access(as, 0) != -1) {
    if(i == 'z')
        return "/";
    *s = i++;
}
return as;
}

```

C.3.4 init()

`init()`^{44a} creates (or re-creates) the temporary file and resets `dot`⁴⁰ and `dol`⁴⁰ to `zero`^{39f} (empty buffer). It is called both from `main()` at startup and from the `e` command (see Section C.11.7), which is why it closes the previous `tfile`^{39b} first.

```

⟨function init(ed.c) 44a⟩≡ (95)
/// main | commands('e') -> <>
void
init(void)
{
    ⟨init() (ed.c) locals 80c⟩

    close(tfile);
    ⟨init() (ed.c) initializing globals 44b⟩
    tfile = create(tfname, ORDWR, 0600);
    if(tfile < 0){
        error_1(T);
        exits(nil);
    }
    dot = dol = zero;
}

```

Uses T 44c, dol 40, dot 40, error_1() 93c, tfile 39b, tfname 39a, and zero 39f.

```

⟨init() (ed.c) initializing globals 44b⟩≡ (44a) 75b▷
    tline = 2;

```

Uses tline 39c.

Recall that `tline` is a Rune-unit offset into `tfile` (the actual byte file offset on disk is `tline*sizeof(Rune)`; see the on-disk layout in Section C.2.3). It starts at 2, not 0, to satisfy two invariants every line offset must maintain: it must be non-zero so 0 stays the “no line” *sentinel*, and it must be even so the low bit of each `zero` entry stays free for `global()`^{81a} and the `k` command to use as a “matched” flag (see Section C.10.7).

In most of this appendix I will not comment much the error management code, which is often trivial (but important). See Section C.13 for a full discussion about error management in `ed` and for the code of `error_1()`^{93c} called in `init()` above.

```

⟨globals ed.c 44c⟩+≡ (95) <43d 45a▷
    char    T[] = "TMP";

```

Uses T 44c.

C.3.5 quit()

`quit()`^{45b} implements the “are you sure?” safety net. It is called both from `main()` after `commands()`^{49b} finished, or from `commands()` itself when the user types the command `q` or `Q` (see Section C.8.5). If there are unsaved

changes (`fchange == true`) and the buffer is non-empty (using the cryptic `dol!=zero`), the first `q` prints `?` and resets `fchange`, so a second `q` will actually exit. The `Q` command bypasses this by clearing `fchange` before calling `quit()` (see Section C.8.5).

```
<globals ed.c 45a>+=
    bool fchange;                                     (95) <44c 45c>
```

```
<function quit(ed.c) 45b>=                                     (95)
    /// main | commands('q') | commands('Q') | ... -> <>
    void
    quit(void)
    {
        if(vflag && fchange && dol!=zero) {
            fchange = false;
            error(Q);
        }
        // else
        remove(tfname);
        exits(nil);
    }
```

Uses `Q` 45c, `dol` 40, `error()` 93b, `fchange` 45a, `tfname` 39a, `vflag` 42a, and `zero` 39f.

Note that `error()`^{93b} internally uses `longjmp()` to emulate exceptions in C (see Section C.13), which is why the control leaves `quit()` after the call to `error()`.

```
<globals ed.c 45c>+=
    char    Q[] = "";                                (95) <45a 47a>
```

Uses `Q` 45c.

C.4 Displaying and reading text

We have now seen the code of the `init()`^{44a} and `quit()`^{45b} functions. It remains to see the most important one, `commands()`^{49b} called also directly from `main()`. However, it is useful to see before the routines to display and read text (which are different from the routines to read and write files, which we will see later).

C.4.1 Displaying text

All diagnostic text output in `ed` (e.g., error messages, number of bytes written with the command `w`) goes through a small buffering layer: `putchr()`^{46b} is supposed to print a single Rune but actually accumulates the UTF-8 encoded bytes in the `line`^{41a} buffer and flushes with a single `write()` call when it sees a newline (or the buffer is nearly full). `putst()`^{45d} and `putshst()`^{46a} below are wrappers for printing `char` (UTF-8) strings and Rune strings respectively and printing a final newline.

```
<function putstr(ed.c) 45d>=                                     (95)
    /// commands | getfile | error_1 | ... -> <>
    void
    putst(char *sp)
    {
        Rune r;

        col = 0;
        for(;;) {
            sp += chartorune(&r, sp);
            if(r == 0)
                break;
            putchr(r);
        }
```

```
    putchar(L'\n');
}
```

Uses col 41e and putchar() 46b.

```
<function putshst(ed.c) 46a>≡ (95)
void
putshst(Rune *sp)
{
    col = 0;
    while(*sp)
        putchar(*sp++);
    putchar(L'\n');
}
```

Uses col 41e and putchar() 46b.

```
<function putchar(ed.c) 46b>≡ (95)
/// error | putst | putshst | ... -> <>
void
putchar(int ac)
{
    char *lp;
    int c;
    Rune rune;

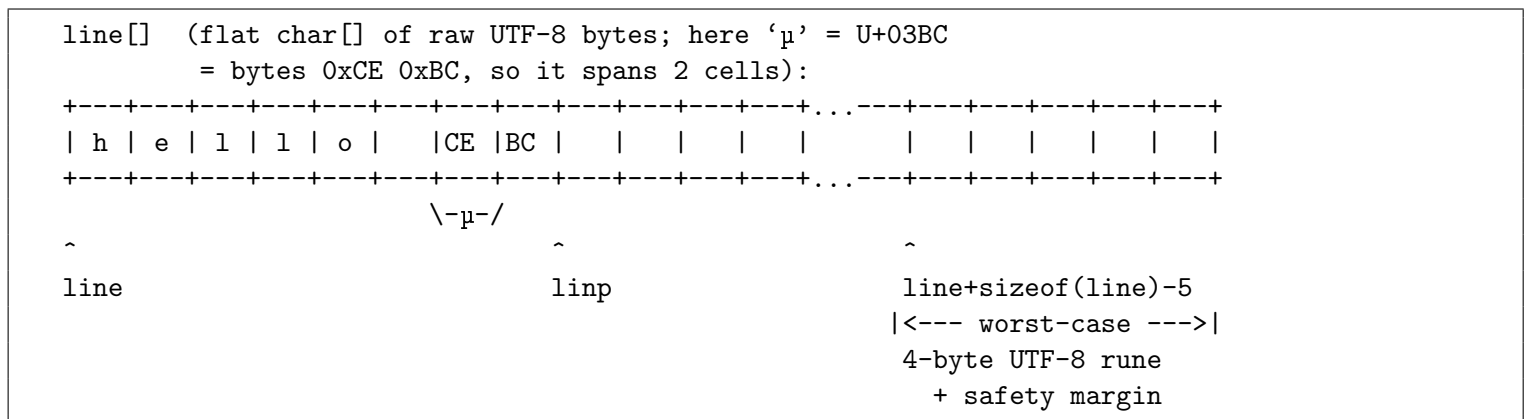
    lp = linp;
    c = ac;
    <putchar() if listf 84c>

    rune = c;
    lp += runetochar(lp, &rune);

    if(c == '\n' || lp >= &line[sizeof(line)-5]) {
        linp = line;
        // the actual output syscall!
        write(oflag? STDERR: STDOUT, line, lp-line);
        return;
    }
    // else
    linp = lp;
}
```

Uses line 41a, linp 41a, and oflag 43a.

The picture below shows line mid-fill during a call to putshst(L"hello μs"). The key points: line is a flat char array holding raw UTF-8 bytes (not Rune cells, unlike tfile^{39b} or linebuf^{41c}), so a single codepoint may occupy one to four cells; linp^{41a} is the persistent fill pointer carried across calls (the local lp above is initialized from it and advanced by runetochar() before the bounds check fires).



`linp` tracks how full the buffer is. The flush check `lp >= &line[sizeof(line)-5]` reserves 5 bytes at the end of the buffer because: (1) runes converted to UTF-8 can be up to 4 bytes each, and (2) one extra byte for safety. Without this margin, a multi-byte rune arriving when the buffer is nearly full could overrun the array.

Note the `oflag` check: when `-o` is active, the edited content goes to `STDOUT`, so diagnostic output from `putchr()` is redirected to `STDERR` to avoid corrupting the data stream.

C.4.2 Reading text

Character input from the user is layered through `getchr()`^{47d}, which checks three sources in order: (1) `peekc`^{47a}, a one-character pushback used when a character needs to be “unread” (a classic technique used in many lexers in Plan 9); (2) `globp`^{47c}, a programmed command string (e.g., `L"r"` or `L"a"` injected by `main()`); (3) actual console input via `Bgetrune()` (from `libbio`, see `LIBCORE` book [Pad16b]). This layering is what makes the `globp` trick work—`commands()`^{49b} does not need to know whether input comes from the user or from a synthetic command.

```
<globals ed.c 47a>+≡ (95) <45c 47b>
//option<Rune> (0 = None)
int peekc;
```

```
<globals ed.c 47b>+≡ (95) <47a 47c>
//option<Rune> (0 = None)
int lastc;
```

```
<globals ed.c 47c>+≡ (95) <47b 49c>
// optional programmed command (e.g., L"r", L"a" or longer string with global())
//option<array<Rune>>
Rune* globp;
```

```
<function getchr(ed.c) 47d>≡ (95)
int
getchr(void)
{
    lastc = peekc;
    if(lastc) {
        peekc = 0;
        return lastc;
    }
    // else
    if(globp) {
        lastc=*globp++;
        if(lastc != 0)
            return lastc;
        // else
        globp = nil;
        return EOF;
    }
    // else
    lastc = Bgetrune(&bcons);
    return lastc;
}
```

Uses `EOF-3` 48a, `bcons` 41f, `globp` 47c, `lastc` 47b, and `peekc` 47a.

`getty()`^{47e3}, reads a line of user input (via `gety()`^{48b}) and checks for the special `.\n` terminator that ends text-entry mode in commands like `a` and `i` (see Section C.1.1).

```
<function getty(ed.c) 47e>≡ (95)
```

³“get from teletype”, an old UNIX name for the console


```

/// main -> commands('a'|'i') -> add -> append(<>,...) -> <>
int
gettty(void)
{
    int rc;

    rc = gety();
    if(rc)
        return rc;
    // else
    if(linebuf[0] == '.' && linebuf[1] == 0)
        return EOF;
    // else
    return OK_0; // OK_0 for caller to read more lines
}

```

Uses EOF-3 48a, gety() 48b, and linebuf 41c.

```

<constants ed.c 48a>+=
    EOF = -1,

```

(95) <41b 71b>

Note that there is no easy way in `ed` to create a line containing just a `'.'` followed by a newline since this special sequence is recognized by `getty()` to mark the end of a user input. One needs instead to enter some fake text like `'.A'` and then use a substitute command to remove the `'A'`. Fortunately, it is rare to need text with a single dot by itself in a line (which is probably why this special sequence was chosen).

`gety()` does the actual character-by-character reading into `linebuf`^{41c}, stopping at a newline.

```

<function gety(ed.c) 48b>≡
    /// getty -> <>
    int
    gety(void)
    {
        int c;
        Rune *p = linebuf;
        <gety() other locals 48c>

        for(;;) {
            c = getchr();
            if(c == '\n') {
                *p = 0;
                return OK_0;
            }
            // else
            if(c == EOF) {
                <get() if gf 49a>
                return c;
            }
            // else
            if(c == 0)
                continue;
            // else
            *p++ = c;
            if(p >= &linebuf[LBSIZE-sizeof(Rune)])
                error(Q);
        }
    }
}

```

(95)

Uses EOF-3 48a, LBSIZE-2 41b, Q 45c, error() 93b, getchr() 47d, and linebuf 41c.

```

<gety() other locals 48c>≡
    Rune *gf = globp;

```

(48b)

```

⟨get() if gf 49a⟩≡
    if(gf)
        peekc = c;
Uses peekc 47a.

```

(48b)

C.5 commands() interpreter loop

We are now finally ready to see the code of `commands()`^{49b}, the main command interpreter loop. In the infinite for loop below, `commands()` reads first an optional *address range* into `addr1`^{49c}, `addr2`^{49c} and then dispatches on the command character `c` via a `switch`. As we will see soon, most command `switch` cases end with a `continue` to loop back; falling through to `error(Q)` below after the switch deals with unknown commands.

```

⟨function commands(ed.c) 49b⟩≡
    void
    commands(void)
    {
        int c;
        ⟨commands() other locals 59a⟩

        for(;;) {
            ⟨commands() in for loop, if pflag 85e⟩
            ⟨commands() read addr1 and c via getch 67a⟩
            switch(c) {
                ⟨commands() switch c cases (ed.c) 49d⟩
            }
            error(Q);
        }
    }
Uses Q 45c and error() 93b.

```

(95)

I will defer the explanations about address ranges to Section C.9 but for now it is enough to understand that some commands can operate on ranges (as in 1,3p) and that the two globals below will store the starting and ending part of the command address range. They both point to `zero`^{39f} entries, similar to `dot`⁴⁰ and `dol`⁴⁰ we saw before.

```

⟨globals ed.c 49c⟩+≡
    // ref<int> in zero[]
    int*   addr1;
    // ref<int> in zero[]
    int*   addr2;

```

(95) <47c 50a>

One of the command `switch` cases handles the EOF character returned by `getchr()`^{47d} when there is no more input (e.g., when the user typed Ctrl-D). In that case `commands()` simply returns to the caller `main()` which will then call `quit()`.

```

⟨commands() switch c cases (ed.c) 49d⟩≡
    case EOF:
        return;
Uses EOF-3 48a.

```

(49b) 50c>

C.6 reading a file: r

Reading a file (as in `r foo.txt`) is the first substantial command, and it involves several cooperating functions: `filename()`^{50e} parses the filename argument in `file`^{50a}, `setwide()`^{51d} sets the address range to the whole buffer (which will have for effect to add the content of `foo.txt` to the end of `zero`, after `dol`⁴⁰), `append()`^{52b} inserts

lines by calling a callback in a loop, `getfile()`^{53c} (the callback) reads one line from the input file (e.g., `foo.txt`) into `linebuf`^{41c}, and `putline()`^{54c} writes `linebuf` to `tfile`^{39b} and returns the offset to store in `zero`^{39f}.

```
<globals ed.c 50a>+≡ (95) <49c 50b>
char file[FNSIZE];
Uses FNSIZE-1 39d.
```

```
<globals ed.c 50b>+≡ (95) <50a 57a>
fdt io;
Biobuf iobuf;
```

```
<commands() switch c cases (ed.c) 50c>+≡ (49b) <49d 57b>
case 'r':
    // will set file[]
    filename(c);
caseread:
    io=open(file, OREAD);
    <commands() in r case, sanity check io 50d>
    <commands() in r case, if append only file 88b>
    Binit(&iobuf, io, OREAD);
    // will set addr2 to dol
    setwide();
    squeeze(0);
    c = (zero != dol);
    // getfile() will use iobuf
    append(getfile, addr2);
    exfile(OREAD); // will close io

    fchange = c;
    continue;
```

Uses `addr2` 49c, `append()` 52b, `dol` 40, `exfile()` 56a, `fchange` 45a, `file` 50a, `filename()` 50e, `getfile()` 53c, `io` 50b, `iobuf` 50b, `setwide()` 51d, `squeeze()` 52a, and `zero` 39f.

The sections below will detail each the code of one of the helper functions used above.

The `fchange = c` at the end is subtle: `c` was saved as `(zero != dol)` before the `append`, so `fchange` is set to true only if the buffer already had content. Reading the first file into an empty buffer does not count as a “change”—this is what lets you `q` without being warned after just opening a file.

```
<commands() in r case, sanity check io 50d>≡ (50c)
if(io < 0) {
    lastc = '\n';
    error(file);
}
```

Uses `error()` 93b, `file` 50a, `io` 50b, and `lastc` 47b.

C.6.1 Reading a filename()

`filename()`^{50e} reads a filename from user input and stores it in `file`^{50a}. If no filename is given (just a newline), it falls back to `savedfile`^{39e}—this is how `r` with no argument re-reads the current file and how setting `globp` to just `L"r"` and setting `savedfile` in `main()` was enough. `filename()` also updates `savedfile` when used from `e` or `f`, establishing the “remembered filename” for future operations.

```
<function filename(ed.c) 50e>≡ (95)
/// main -> commands('r' | 'e' | 'f') -> <>
void
filename(int comm)
{
    char *p1, *p2;
```

```

Rune rune;
int c;

count = 0;
c = getchr();
⟨filename() if c is newline or EOF, use savedfile 51b⟩
// else
⟨filename() read a space and skip extra spaces 51a⟩
p1 = file;
do {
    if(p1 >= &file[sizeof(file)-6] || c == ' ' || c == EOF)
        error(Q);
    rune = c;
    p1 += runetochar(p1, &rune);
} while((c=getchr()) != '\n');
*p1 = '\0';

⟨filename() set savedfile depending on commands 51c⟩
}

```

Uses EOF-3 48a, Q 45c, count 41d, error() 93b, file 50a, and getchr() 47d.

```

⟨filename() read a space and skip extra spaces 51a⟩≡ (50e)
if(c != ' ')
    error(Q);
while((c=getchr()) == ' ')
    ;
if(c == '\n')
    error(Q);

```

Uses Q 45c, error() 93b, and getchr() 47d.

```

⟨filename() if c is newline or EOF, use savedfile 51b⟩≡ (50e)
if(c == '\n' || c == EOF) {
    p1 = savedfile;
    if(*p1 == '\0' && comm != 'f')
        error(Q);
    p2 = file;
    while(*p2++ = *p1++)
        ;
    return;
}

```

Uses EOF-3 48a, Q 45c, error() 93b, file 50a, and savedfile 39e.

```

⟨filename() set savedfile depending on commands 51c⟩≡ (50e)
if(savedfile[0] == '\0' || comm == 'e' || comm == 'f') {
    p1 = savedfile;
    p2 = file;
    while(*p1++ = *p2++)
        ;
}

```

Uses file 50a and savedfile 39e.

C.6.2 setwide() and squeeze()

setwide()^{51d} sets addr1,addr2 to span the whole buffer (line 1 to dol⁴⁰) when the user did not supply explicit addresses (which can be checked by inspecting the global given^{66a}).

```

⟨function setwide(ed.c) 51d⟩≡ (95)
/// main -> commands('r') -> <>

```

```

void
setwide(void)
{
    if(!given) {
        addr1 = zero + (dol>zero);
        addr2 = dol;
    }
}

```

Uses `addr1` 49c, `addr2` 49c, `dol` 40, `given` 66a, and `zero` 39f.

`squeeze()` 52a validates that the address range is sane—called with 0 when an empty buffer is acceptable (e.g., for the `r` command) or 1 when at least one line is required (e.g., for the `p` command).

```

⟨function squeeze(ed.c) 52a⟩≡ (95)
/// main -> commands('r') -> <>
void
squeeze(int i)
{
    if(addr1 < zero+i || addr2 > dol || addr1 > addr2)
        error(Q);
}

```

Uses `Q` 45c, `addr1` 49c, `addr2` 49c, `dol` 40, `error()` 93b, and `zero` 39f.

C.6.3 `append()` and `getfile()`

`append()` 52b is the core insertion function. It takes a callback `f` (either `getfile()` 53c for reading files or `gettty()` 47e for user input) and an address `a` (where to insert after, for instance `dol` 40). For each line that `f` returns via `linebuf` 41c, it calls `putline()` 54c to write it to `tfile` 39b, then shifts all entries in `zero` 39f above the insertion point up by one to make room for the new offset. This shift is the `while(a1 > rdot)` loop.

```

⟨function append(ed.c) 52b⟩≡ (95)
/// main -> commands('r'|'i'|'a') -> add -> <> -> (getfile | gettty)
int
append(int (*f)(void), int *a)
{
    //ref<int> in zero[]
    int *a1, *a2, *rdot;
    int nline = 0;
    // file offset in tfile for temporary line just added by putline() (Rune-unit)
    int tl;
    ⟨append() other locals 53a⟩

    dot = a;
    // f() (e.g., getfile()) will modify linebuf[]
    while((*f)() == OK_0) {
        ⟨append() grow zero if zero too small 53b⟩
        // putline() will use linebuf[]
        tl = putline();
        nline++;

        // set zero[] indices
        a1 = ++dol;
        a2 = a1+1;
        rdot = ++dot;
        // shift existing line references up by one
        while(a1 > rdot)
            *--a2 = *--a1;
        // insert the new line reference in zero
        *rdot = tl;
    }
}

```

```

    }
    return nline;
}

```

Uses [dol 40](#), [dot 40](#), and [putline\(\) 54c](#).

When `zero` is full, `realloc()` may move it to a new address. Every pointer into the old array—`addr1`, `addr2`, `dol`, `dot`—must then be adjusted by the same delta (`a1 - zero`, i.e., new base minus old base). This is a classic C `realloc` pattern: compute the displacement once, then apply it to all pointers that referred to the old allocation.

```

⟨append() other locals 53a⟩≡ (52b)
/* claude: pointer-relocation delta when realloc() moves the zero[] block.
 * It is a 64-bit ptrdiff (in int-element units), so it must NOT be an int:
 * the original code reused 'int tl' here and silently truncated it on LP64
 * systems. glibc's realloc keeps the regrown block nearby, so the delta
 * stayed within INT_MAX and Linux never tripped this. macOS's malloc can
 * return a block in a far-away zone, making (a1-zero) overflow a 32-bit
 * int; the truncated delta then corrupts zero/addr1/addr2/dol/dot and the
 * next '*rdot = tl' store segfaults. */
intptr delta;

```

```

⟨append() grow zero if zero too small 53b⟩≡ (52b)
if((dol-zero) >= nlall) {

    nlall += 512;
    a1 = realloc(zero, (nlall+5)*sizeof(int*)); // BUG: sizeof(int)
    if(a1 == nil) {
        error("MEM?");
        rescue();
    }
    // old: tl = a1 - zero;
    delta = a1 - zero; /* relocate pointers */

    zero += delta;
    addr1 += delta;
    addr2 += delta;
    dol += delta;
    dot += delta;
}

```

Uses [addr1 49c](#), [addr2 49c](#), [dol 40](#), [dot 40](#), [error\(\) 93b](#), [nlall 39f](#), [rescue\(\) 94c](#), and [zero 39f](#).

`getfile()` reads one line from `iobuf`^{50b} (the `Biobuf` initialized from the file to read from the `r` command) into `linebuf`, counting characters in `count`^{41d}. It replaces the terminating `\n` with a null rune (lines in `tfile` are null-terminated, not newline-terminated as explained in [Section C.2.3](#)). It returns 0 on success or EOF when the file is exhausted.

```

⟨function getfile(ed.c) 53c⟩≡ (95)
/// main -> commands(c = 'r') -> append(<>, ...) -> <>
int
getfile(void)
{
    int c;
    Rune *lp;

    lp = linebuf;
    do {
        c = Bgetrune(&iobuf);
        ⟨getfile() if c negative, possibly return EOF 54a⟩
        // else
        ⟨getfile() if overflow linebuf 54b⟩
    }
}

```

```

        // else
        *lp++ = c;
        count++;
    } while(c != '\n');
    lp[-1] = 0;
    return OK_0;
}

```

Uses count 41d, iobuf 50b, and linebuf 41c.

```

⟨getfile() if c negative, possibly return EOF 54a⟩≡ (53c)
    if(c < 0) {
        if(lp > linebuf) {
            putst("'\\n' appended");
            c = '\\n';
        } else
            return EOF;
    }

```

Uses EOF-3 48a, linebuf 41c, and putst() 45d.

```

⟨getfile() if overflow linebuf 54b⟩≡ (53c)
    if(lp >= &linebuf[LBSIZE]) {
        lastc = '\\n';
        error(Q);
    }

```

Uses LBSIZE-2 41b, Q 45c, error() 93b, lastc 47b, and linebuf 41c.

C.6.4 putline() (simplified)

Here is a simplified version of `putline()`^{54c} that shows the essential logic: seek to the current write position in `tfile`^{39b}, write `linebuf`^{41c} in `tfile` as a run of fixed-width Rune cells terminated by a null Rune, advance `tline`^{39c} by the number of Runes written (rounded up to an even count), and return the old offset (which `append()`^{52b} stores in an adjusted slot in `zero`^{39f}).

```

⟨function putline 54c⟩≡ (95)
    /// append -> <>
    int
    putline(void)
    {
        Rune *lp;
        int n, tl;

        ⟨putline() if opti 88f⟩
        // else

        fchange = true;
        tl = tline;                                     // current write offset (Rune units)

        // walk linebuf[]; stop at the null Rune or an embedded '\\n'
        for(lp = linebuf; *lp; lp++) {
            ⟨putline() if newline char, adjust and break 55⟩
            // else, nothing, continue loop until *lp == 0 == null Rune
        }
        n = lp - linebuf + 1;                            // # Runes including terminator

        if(tline + n >= NBLK * BLKSIZE / (int)sizeof(Rune))
            error(T);

        // tfile stores fixed-width Runes (not UTF-8);
    }

```

```

// convert Rune offset/count to byte offsets for seek/write
seek(tfile, tline * sizeof(Rune), SEEK__START);
// syscall!
write(tfile, linebuf, n * sizeof(Rune));

// advance past the line just written, rounding up to an even Rune
// count so tline stays even. zero[] entries are Rune offsets, and
// global() and 'k' steal the low bit of each entry as a "matched"
// flag (see global() and anymarks). Padding costs at most one Rune.
tline += (n + 1) & ~1;
return tl;
}

```

Uses BLKSIZE-8 88e, NBLK-9 90, T 44c, error() 93b, fchange 45a, linebuf 41c, tfile 39b, and tline 39c.

The crucial detail: `tfile` is a stream of Rune-sized cells, not bytes, and `tline` counts cells, not bytes—which is why every seek and write needs a `*sizeof(Rune)` conversion.

The other subtlety is the even-padding of `tline`. Each `zero` entry holds a Rune offset into `tfile` (the value returned here), and `global()`^{81a} together with the `k` command steal the low bit of that entry as a “matched” flag. Keeping every offset even costs at most one extra Rune cell per line and frees the bit for the mark.

```

<putline() if newline char, adjust and break 55>≡ (54c)
// (the latter happens after dosub() injects \n in a multi-line subst)
if(*lp == '\n') {
    *lp = 0; // turn '\n' into null terminator
    linebp = lp + 1; // stash remainder for getsub()
    break;
}

```

Uses linebp 88d.

C.6.5 Concrete editing example

Here is a short example of how `zero`^{39f} and `tfile`^{39b} evolve. The diagrams below use byte offsets and `\n`-terminated text for readability, as in the earlier picture of `zero` in Section C.2.3; the actual on-disk layout is fixed-width Rune cells, but that detail does not affect the editing operations shown here. We do keep one detail from the real layout: offsets start at 2, because `tline` starts at 2 and so the first two cells of `tfile` are reserved (kept clear for `global()`^{81a}’s mark trick). Suppose the user types:

```

r foo.txt      (file contains: "alpha\nbeta\ngamma\n")
2d             (delete line 2)
1a            (append after line 1)
delta         (the new text)
.             (end of new text)

```

After `r` reads the three lines in `foo.txt`:

```

tfile (offsets):  0  2      8      13      19
                  +--+-----+-----+-----+
                  |..|alpha\|beta\|gamma\|
                  +--+-----+-----+-----+
                  ^^ reserved (bytes 0-1, kept clear)
zero:  [_ , 2, 8, 13]    (3 lines, dot=3)

```

After `2d` (delete line 2: shift entries down in `zero`, do nothing on disk):


```
tfile:                unchanged (offset 8 'beta' is orphaned)
zero:  [_ , 2, 13]      (2 lines, dot=2)
```

After 1a then `delta` then `.` (append “delta” after line 1, `putline()` writes `delta\n` to the end of `tfile` at offset 19, `append()`^{52b} inserts the new offset into `zero`):

```
tfile:                0   2       8       13       19       25
                    +---+-----+-----+-----+-----+
                    |..|alpha\|beta\|gamma\|delta\|
                    +---+-----+-----+-----+-----+
                                                ^new

zero:  [_ , 2, 19, 13]  (3 lines, dot=2)
```

Notice that `tfile` grew by one line (`delta`), but the “deleted” `beta` is still on disk—it just is no longer referenced by `zero`. This is why `ed` can do bounded edits in constant time but the temp file can become arbitrarily large during a long session.

C.6.6 `exfile()`

`exfile()`^{56a} (“exit file”) finalizes an I/O operation: it flushes the buffer if writing, closes `io`^{50b}, and in verbose mode prints the character count via `putd()`^{56b}.

```
<function exfile(ed.c) 56a>≡ (95)
  /// main -> commands('r'|'w') -> <>
  void
  exfile(int om)
  {
      if(om == OWRITE)
          if(Bflush(&iobuf) < 0)
              error(Q);
      close(io);
      io = -1;
      if(vflag) {
          putd();
          putchar(L'\n');
      }
  }
```

Uses `Q` 45c, `error()` 93b, `io` 50b, `iobuf` 50b, `putchr()` 46b, `putd()` 56b, and `vflag` 42a.

```
<function putd(ed.c) 56b>≡ (95)
  void
  putd(void)
  {
      int r;

      r = count%10;
      count /= 10;
      if(count)
          // recurse
          putd();
      putchar(r + L'0');
  }
```

Uses `count` 41d, `putchr()` 46b, and `putd()` 56b.

C.7 writing a file: w

Writing (as in `w foo.txt`) is the dual of reading. `putfile()`^{57c} called below iterates over the address range, fetches each line from `tfile` via `getline()`⁵⁸, and writes it to the output file via `Bputrune()` (from `libbio`). The `W` (uppercase) variant opens the file in append mode.

```
<globals ed.c 57a>+≡ (95) <50b 66a>
// write append
bool wrapp;

<commands() switch c cases (ed.c) 57b>+≡ (49b) <50c 59d>
case 'W':
    wrapp = true;
    // Fallthrough
case 'w':
    setwide();
    squeeze(dol>zero);

<commands() when 'w' check for 'q' part1 59b>
filename(c);

if(!wrapp ||
    ((io = open(file, OWRITE)) == -1) ||
    ((seek(io, OL, SEEK__END)) == -1))

    io = create(file, OWRITE, 0666);
    if(io < 0)
        error(file);
// else and fallthrough

Binit(&iobuf, io, OWRITE);
wrapp = false;
if(dol > zero)
    putfile();

exfile(OWRITE);

if(addr1<=zero+1 && addr2==dol)
    fchange = false;
<commands() when 'w' check for 'q' part2 59c>
continue;
```

Uses `addr1` 49c, `addr2` 49c, `dol` 40, `error()` 93b, `exfile()` 56a, `fchange` 45a, `file` 50a, `filename()` 50e, `io` 50b, `iobuf` 50b, `putfile()` 57c, `setwide()` 51d, `squeeze()` 52a, `wrapp` 57a, and `zero` 39f.

C.7.1 putfile()

`putfile()`^{57c} is the dual of `getfile()`^{53c}: it writes in `iobuf`^{50b}.

```
<function putfile(ed.c) 57c>≡ (95)
/// main -> commands('w') -> <>
void
putfile(void)
{
    int *a1;
    Rune *lp;
    long c;

    a1 = addr1;
    do {
```

```

    // modifies (and returns) linebuf[]
    lp = getline(*a1++);
    for(;;) {
        count++;
        c = *lp++;
        if(c == 0) {
            // a null Rune in tfile converts to \n in the written file
            if(Bputrune(&iobuf, '\n') < 0)
                error(Q);
            break;
        }
        // else
        if(Bputrune(&iobuf, c) < 0)
            error(Q);
    }
} while(a1 <= addr2);
if(Bflush(&iobuf) < 0)
    error(Q);
}

```

Uses Q 45c, addr1 49c, addr2 49c, count 41d, error() 93b, getline() 58, and iobuf 50b.

C.7.2 getline() (simplified)

getline()⁵⁸ is the dual of putline()^{54c}: given a line offset `tl` (in Rune units), it seeks to that position in `tfile` and reads Rune cells into `linebuf`^{41c} until it hits the null-Rune terminator. As in `putline()`, the `*sizeof(Rune)` factor in the seek and the read size is what converts the cell-unit offset into a byte file offset on disk.

```

⟨function getline 58⟩≡
    /// putfile -> <>
    Rune*
    getline(int tl)
    {
        int n;

        ⟨getline() if opti 91b⟩
        // else

        // tfile stores fixed-width Runes; convert Rune offset to bytes
        seek(tfile, tl * sizeof(Rune), SEEK__START);

        // read one Rune at a time until we hit the null-Rune terminator
        n = 0;
        // syscall
        while(read(tfile, &linebuf[n], sizeof(Rune)) == sizeof(Rune)
            && linebuf[n] != 0)
            n++;

        linebuf[n] = 0;
        return linebuf;
    }

```

Uses linebuf 41c and tfile 39b.

C.7.3 Write and quit: wq

The `w` command peeks at the next character to detect `wq` (write and quit) or `wQ` (write and force-quit). If the next character is neither `q` nor `Q`, it is pushed back (“unread”) with `peekc`^{47a} so `filename()`^{50e} called after can

consume it.

```
<commands() other locals 59a>≡ (49b) 66b▷  
    int temp;
```

```
<commands() when 'w' check for 'q' part1 59b>≡ (57b)  
    temp = getch();  
    if(temp != 'q' && temp != 'Q') {  
        peekc = temp;  
        temp = 0;  
    }
```

Uses `getchr()` 47d and `peekc` 47a.

```
<commands() when 'w' check for 'q' part2 59c>≡ (57b)  
    if(temp == 'Q')  
        fchange = false;  
    if(temp)  
        quit();
```

Uses `fchange` 45a and `quit()` 45b.

C.8 Main commands

With the reading and writing machinery in place, we can now look at the basic editing commands. Most of them are short—often just a few lines calling the helper functions we have already seen.

C.8.1 printing lines: p

The `p` command prints lines in the address range. `printcom()`^{59e} iterates from `addr1` to `addr2`, calling `getline()`⁵⁸ to fetch each line from `tfile` and `putshst()`^{46a} to display it.

```
<commands() switch c cases (ed.c) 59d>+≡ (49b) <57b 60c▷  
<commands before 'p' case 84b>  
    case 'p':  
    case 'P':  
        newline();  
        printcom();  
        continue;
```

Uses `newline()` 60a and `printcom()` 59e.

```
<function printcom(ed.c) 59e>≡ (95)  
    void  
    printcom(void)  
    {  
        int *a1;  
  
        nonzero();  
        a1 = addr1;  
        do {  
            <printcom() if listn 85c>  
            putshst(getline(*a1++));  
        } while(a1 <= addr2);  
        dot = addr2;  
        <printcom() reset flags at the end 85g>  
    }
```

Uses `addr1` 49c, `addr2` 49c, `dot` 40, `getline()` 58, `nonzero()` 60b, and `putshst()` 46a.

The command `p` does not accept further arguments beyond the address range (as in `1,3p`), hence the function below to enforce it finishes with a newline (as opposed to `r` which can accept a filename as in `r hello.txt`).

```
⟨function newline(ed.c) 60a⟩≡ (95)
void
newline(void)
{
    int c;

    c = getchar();
    if(c == '\n' || c == EOF)
        return;
    ⟨newline() if special chars pln 85f⟩
    // else
    error(Q);
}
```

Uses `EOF-3` [48a](#), `Q` [45c](#), `error()` [93b](#), and `getchr()` [47d](#).

```
⟨function nonzero(ed.c) 60b⟩≡ (95)
void
nonzero(void)
{
    squeeze(1);
}
```

Uses `squeeze()` [52a](#).

Pressing just Enter (newline with no command) prints the next line—the `'\n'` case below advances `dot`^{[40](#)} by one and prints.

```
⟨commands() switch c cases (ed.c) 60c⟩+≡ (49b) <59d 60d>
case '\n':
    if(a1==nil) {
        a1 = dot+1;
        addr2 = a1;
        addr1 = a1;
    }
    if(lastsep=='')
        addr1 = a1;

    printcom();
    continue;
```

Uses `addr1` [49c](#), `addr2` [49c](#), `dot` [40](#), and `printcom()` [59e](#).

C.8.2 printing the remembered file: f

```
⟨commands() switch c cases (ed.c) 60d⟩+≡ (49b) <60c 61a>
case 'f':
    setnoaddr();
    filename(c);
    putst(savedfile);
    continue;
```

Uses `filename()` [50e](#), `putst()` [45d](#), `savedfile` [39e](#), and `setnoaddr()` [60e](#).

This command does not accept any explicit address range before, hence the call to `setnoaddr()`^{[60e](#)}.

```
⟨function setnoaddr(ed.c) 60e⟩≡ (95)
void
setnoaddr(void)
{
    if(given)
```

```
    error(Q);
}
```

Uses Q 45c, error() 93b, and given 66a.

C.8.3 printing line number: =

```
<commands() switch c cases (ed.c) 61a>+≡ (49b) <60d 61b>
case '=':
    setwide();
    squeeze(0);
    newline();
    count = addr2 - zero;
    putd();
    putchar(L'\n');
    continue;
```

Uses addr2 49c, count 41d, newline() 60a, putchar() 46b, putd() 56b, setwide() 51d, squeeze() 52a, and zero 39f.

C.8.4 append and insert: a, i

Both a (append) and i (insert) use add()^{61d}, which calls append(gettty, addr2). The difference is that i decrements addr1/addr2 by one, so the new text goes *before* the current line instead of after it.

```
<commands() switch c cases (ed.c) 61b>+≡ (49b) <61a 61c>
case 'a':
    add(0);
    continue;
```

Uses add() 61d.

```
<commands() switch c cases (ed.c) 61c>+≡ (49b) <61b 62a>
case 'i':
    add(-1);
    continue;
```

Uses add() 61d.

```
<function add(ed.c) 61d>≡ (95)
void
add(int i)
{
    if(i && (given || dol > zero)) {
        addr1--;
        addr2--;
    }
    squeeze(0);
    newline();
    append(gettty, addr2);
}
```

Uses addr1 49c, addr2 49c, append() 52b, dol 40, gettty() 47e, given 66a, newline() 60a, squeeze() 52a, and zero 39f.

We saw before a call to append(getfile, ...) for the r command in Section C.6. This time add() passes instead gettty()^{47e} (that we also saw before) for the callback to append()^{52b}.

C.8.5 quitting: q

```
<commands() switch c cases (ed.c) 62a>+≡ (49b) <61c 62b>
case 'Q':
    fchange = false;
    // fallthrough:
case 'q':
    setnoaddr();
    newline();
    quit();
```

Uses `fchange` 45a, `newline()` 60a, `quit()` 45b, and `setnoaddr()` 60e.

C.8.6 deleting lines: d

`rdelete()` 62c (“range delete”) removes lines by shifting the `zero` 39f entries above `addr2` 49c down to overwrite the deleted range. The text remains in `tfile` 39b (it is never reclaimed), only the offsets in `zero` change.

```
<commands() switch c cases (ed.c) 62b>+≡ (49b) <62a 62d>
case 'd':
    nonzero();
    newline();
    rdelete(addr1, addr2);
    continue;
```

Uses `addr1` 49c, `addr2` 49c, `newline()` 60a, `nonzero()` 60b, and `rdelete()` 62c.

```
<function rdelete(ed.c) 62c>≡ (95)
void
rdelete(int *ad1, int *ad2)
{
    int *a1, *a2, *a3;

    a1 = ad1;
    a2 = ad2+1;
    a3 = dol;
    dol -= a2 - a1;
    do {
        *a1++ = *a2++;
    } while(a2 <= a3);
    a1 = ad1;
    if(a1 > dol)
        a1 = dol;
    dot = a1;
    fchange = true;
}
```

Uses `dol` 40, `dot` 40, and `fchange` 45a.

C.8.7 changing lines: c

The `c` (change) command is simply delete followed by append: it calls `rdelete()` 62c then `append(gettty, addr1-1)`.

```
<commands() switch c cases (ed.c) 62d>+≡ (49b) <62b 63a>
case 'c':
    nonzero();
    newline();
    rdelete(addr1, addr2);
    append(gettty, addr1-1);
    continue;
```

Uses `addr1` 49c, `addr2` 49c, `append()` 52b, `gettty()` 47e, `newline()` 60a, `nonzero()` 60b, and `rdelete()` 62c.

C.8.8 moving and copying lines: m and t

The **m** (move) and **t** (transfer, but really copy but **c** was already taken) commands share the `move()`^{63c} function below. For **t**, the lines are first duplicated to the end of the buffer via `append(getcopy, ...)`, then moved.

The two commands require an additional *destination address*, for instance `3,5t1` will copy lines 3 to 5 after the first line. For example, on the following buffer:

```
one
two
three
four
five
six
```

you would then get after `3,5t1`:

```
one
three
four
five
two
three
four
five
six
```

```
<commands() switch c cases (ed.c) 63a>+≡ (49b) <62d 63b>
case 'm':
    move(0);
    continue;
```

Uses `move()` 63c.

```
<commands() switch c cases (ed.c) 63b>+≡ (49b) <63a 74c>
case 't':
    move(1);
    continue;
```

Uses `move()` 63c.

```
<function move(ed.c) 63c>≡ (95)
void
move(int cflag)
{
    int *adt, *ad1, *ad2;

    nonzero();
    adt = address();
    <move() sanity check adt 65a>
    newline();

    <move() (ed.c) if cflag 65b>
    else {
        ad2 = addr2;
        <move() (ed.c) clear the low bit for the range 65d>
        ad1 = addr1;
    }
    ad2++;
```



```

if(ad1<ad2) {
    dot = ad1 + (ad2-ad1);
    if(++ad1==ad2)
        return;
    reverse(ad1, ad2);
    reverse(ad2, dot);
    reverse(ad1, dot);
} else
if(ad1 >= ad2) {
    dot = ad1++;
    reverse(ad1, ad2);
    reverse(ad2, dot);
    reverse(ad1, dot);
} else
    error(Q);
fchange = true;
}

```

Uses Q 45c, addr1 49c, addr2 49c, address() 67d, dot 40, error() 93b, fchange 45a, newline() 60a, nonzero() 60b, and reverse() 64.

The actual move uses a clever triple-reverse algorithm: to move a block from position A to position B, reverse the elements from A to the block, reverse the block, then reverse the whole range. This rearranges **zero** entries in-place without any temporary storage. Here is how it rotates a block to the left in three reversal passes. Suppose **zero** looks like this and we want to move lines 4–5 (the block “DE”) after line 1:

```

start:      A B C D E F
            1 2 3 4 5 6
            ^   ^^^
            adt++ ad1..ad2

reverse(adt, ad1):    reverse [B C] -> [C B]
                    A C B D E F

reverse(ad1, ad2):    reverse [D E] -> [E D]
                    A C B E D F

reverse(adt, ad2):    reverse [C B E D] -> [D E B C]
                    A D E B C F
                    ^^^
                    moved!

```

This is the same algorithm Jon Bentley attributes to Doug McIlroy in *Programming Pearls* [Ben99]: rotating an array can be done in-place in $O(n)$ time and $O(1)$ extra space using three reversals, no temporary buffer required. Compare with the naive approach (allocate a temp array, copy, shift, copy back) which needs $O(n)$ extra memory. The opposite case (moving a block forward instead of backward) is symmetric: **move()** picks one of the two branches based on whether the destination is before or after the source.

```

⟨function reverse(ed.c) 64⟩≡ (95)
void
reverse(int *a1, int *a2)
{
    int t;

    for(;;) {
        t = *--a2;
        if(a2 <= a1)

```

```

        return;
    *a2 = *a1;
    *a1++ = t;
}
}

```

`<move() sanity check adt 65a>≡` (63c)

```

    if(adt==0)    /* address() guarantees addr is in range */
        error(Q);

```

Uses Q 45c and error() 93b.

`<move() (ed.c) if cflag 65b>≡` (63c)

```

    if(cflag) {
        int *ozero, delta;

        ad1 = dol;
        ozero = zero;
        append(getcopy, ad1++);
        ad2 = dol;
        delta = zero - ozero;
        ad1 += delta;
        adt += delta;
    }

```

Uses append() 52b, dol 40, getcopy() 65c, and zero 39f.

`<function getcopy(ed.c) 65c>≡` (95)

```

    int
    getcopy(void)
    {
        if(addr1 > addr2)
            return EOF;
        getline(*addr1++);
        return OK_0;
    }

```

Uses EOF-3 48a, addr1 49c, addr2 49c, and getline() 58.

`<move() (ed.c) clear the low bit for the range 65d>≡` (63c)

```

    for(ad1 = addr1; ad1 <= ad2;)
        *ad1++ &= ~01; // tricky

```

Uses addr1 49c.

C.9 Command addresses

Until now I have glossed over how `addr1`^{49c} and `addr2`^{49c} are set. In `ed`, most commands can be preceded by one or two addresses: a line number (`3p`), a range (`1,5d`), the current line (`.p`), the last line (`$a`), or even a search (`/foo/p`). The address-parsing code runs before the command character is read, so every command benefits from the same rich addressing syntax.

Here is a cheat sheet of the address primitives and how they eventually show up in the code:

syntax -----	meaning -----	handled by -----
N	absolute line N	getnum() in address()
.	dot (current line)	case '.' in address()
\$	dol (last line)	case '\$' in address()
'x	line previously marked x	case '\'' (see 'k')
/re/	next line matching re	case '/' in address()
?re?	previous line matching re	case '?' in address()
A+N A-N	A offset by N	+ and - operators
A,B	range (addr1=A, addr2=B)	top-level in commands()
A;B	range with side effect dot=A	';' case in commands()

All these forms feed a single recursive-descent parser. The two address separators , and ; are parsed in `commands()`^{49b} itself; everything to the left or right of a separator is parsed by `address()`^{67d}.

A worked example makes this concrete. Suppose the user types `.+2,/foo/p` when `dot` points to line 5 and the next line matching `foo` is line 11. The parse proceeds as follows:

```

outer loop in commands():
  a1 = address()      -> parses ".+2"
    step 1: c='.'     -> a=dot (line 5), opcnt=1
    step 2: c='+'     -> sign=+, nextopand=2, loop
    step 3: c='2'     -> a += 2 (line 7), opcnt=2
    step 4: c=','     -> push back, return a (line 7)
  c = ','             -> separator, loop again
  addr1 = 7
  a1 = address()      -> parses "/foo/"
    step 1: c='/'     -> compile(/foo/); scan from dot
                        until match -> a=line 11
    step 2: c='p'     -> push back, return a (line 11)
  c = 'p'             -> not a separator, exit loop
  addr2 = 11; given = true
  switch(c='p') -> print lines 7..11

```

Two things are worth noticing. First, the address parser is strictly left-to-right with no precedence—every operator is applied in sequence, so `1+2-3` means `((1+2)-3)`, never `1+(2-3)`. Second, the search forms `/re/` and `?re?` are resolved *during* parsing: `address()` actually scans the buffer and returns a concrete line pointer, which is why compiling the regexp happens inline in `address()` rather than later at execution time.

Returning to the `mkenam` script in Section C.1.2, you can now read the address parts of each line. The substitution line is two addresses (1 and \$) followed by the `s` command; the `v` line is a global-not-match command (no address part, matches operate on the whole buffer); `1i` is a single address (line 1) followed by `i` (insert before); `$a` is a single address (last line with \$) followed by `a` (append after).

C.9.1 Reading addr1 and addr2

```

<globals ed.c 66a>+≡                                     (95) <57a 71a>
  bool given;

<commands() other locals 66b>+≡                           (49b) <59a 66c>
  int *a1;

<commands() other locals 66c>+≡                           (49b) <66b 88a>
  char lastsep; // '\n' or ',' or ';'

```

The address-parsing loop reads addresses separated by , or ;. If no addresses are given, `addr1 = addr2 = dot` (the current line). If only one address is given, both `addr1` and `addr2` point to it. The `given` flag records whether the user typed an explicit address, so commands like `setwide()`^{51d} can choose appropriate defaults.

```

⟨commands() read addr1 and c via getch() 67a⟩≡ (49b)
    c = '\n';
    addr1 = nil;

    for(;;) {
        lastsep = c;
        a1 = address();
        c = getch();

        if(c != ',' && c != ';')
            break;

        // else
        if(lastsep == ',')
            error(Q);
        if(a1 == nil) {
            a1 = zero+1; // line 1
            if(a1 > dol)
                a1--;
        }
        addr1 = a1;
        ⟨commands() in address parsing, if separator is ',' 67b⟩
    }
    ⟨commands() after address parsing, use defaults if missing addresses 67c⟩

```

Uses `Q` 45c, `addr1` 49c, `address()` 67d, `dol` 40, `error()` 93b, `getch()` 47d, and `zero` 39f.

```

⟨commands() in address parsing, if separator is ';' 67b⟩≡ (67a)
    if(c == ';')
        dot = a1;

```

Uses `dot` 40.

```

⟨commands() after address parsing, use defaults if missing addresses 67c⟩≡ (67a)
    if(lastsep != '\n' && a1 == nil)
        a1 = dol;

    if((addr2=a1) == nil) {
        given = false;
        addr2 = dot;
    } else
        given = true;

    if(addr1 == nil)
        addr1 = addr2;

```

Uses `addr1` 49c, `addr2` 49c, `dol` 40, `dot` 40, and `given` 66a.

C.9.2 address()

`address()`^{67d} parses a single address expression. It starts from `dot` and applies offsets: a bare number is relative to line 0 (i.e., absolute), `.` and `$` set the base, `+` and `-` adjust the sign for subsequent operands, and `/re/` or `?re?` search forward or backward.

```

⟨function address(ed.c) 67d⟩≡ (95)
    int*
    address(void)

```

```

{
    int sign, *a, opcnt, nextopand, *b, c;

    nextopand = -1;
    sign = 1;
    opcnt = 0;
    a = dot;
    do {
        do {
            c = getchrc();
        } while(c == ' ' || c == '\t');

        if(c >= '0' && c <= '9') {
            peekc = c;
            if(!opcnt)
                a = zero;
            a += sign*getnum();
        } else

            switch(c) {
                // will return in default: case
                <address() (ed.c) switch c cases 68>
            }

            sign = 1;
            opcnt++;
        } while(zero <= a && a <= dol);

        error(Q);
        return nil;
    }
}

```

Uses Q 45c, dol 40, dot 40, error() 93b, getchrc() 47d, getnum() 69a, peekc 47a, and zero 39f.

The nested do-do structure above is subtle. The inner loop just skips whitespace. The outer loop chains operators: each iteration consumes one operand (., \$, a number, or a search), and then loops back to see if another + or - follows. The variable nextopand tracks whether a trailing operator like + or - was seen without a following operand, in which case the default: branch adds an implicit 1 (so 3+ means 3+1, i.e., line 4).

The default: case below handles the return: if no operator is pending, push the character back and return the computed address.

```

<address() (ed.c) switch c cases 68>≡ (67d) 69b▷
default:
    if(nextopand == opcnt) {
        a += sign;
        if(a < zero || dol < a)
            continue; /* error(Q); */
    }

    if(c != '+' && c != '-' && c != '^') {
        peekc = c;
        if(opcnt == 0)
            a = nil;

        // finally returning!
        return a;
    }
    sign = 1;
    if(c != '+')
        sign = -sign;
    nextopand = ++opcnt;

```

```
continue;
```

Uses `dol` 40, `peekc` 47a, and `zero` 39f.

```
<function getnum(ed.c) 69a>≡ (95)
int
getnum(void)
{
    int r = 0;
    int c;

    for(;;) {
        c = getchar();
        if(c < '0' || c > '9')
            break;
        r = r*10 + (c-'0');
    }
    peekc = c;
    return r;
}
```

Uses `getchr()` 47d and `peekc` 47a.

C.9.3 Basic addresses: `.` and `$`

```
<address() (ed.c) switch c cases 69b>+≡ (67d) <68 70>
case '$':
    a = dol;
    // Fallthrough
case '.':
    if(opcnt)
        error(Q);
    break;
```

Uses `Q` 45c, `dol` 40, and `error()` 93b.

C.10 Search and replace

Search and replace is where `ed`'s real power lies. The `/re/` and `?re?` address forms search forward and backward, the `s` command performs substitutions, and the `g` command applies commands to all matching lines. The actual regular expression engine is in `libregex` (see `LIBCORE` book [Pad16b]); `ed` just reads and compiles patterns via `compile()`^{71c} and tests matches via `match()`^{72d}.

Before diving in, it helps to see the buffers and pointers that the substitute machinery juggles together. A single `s/re/rhs/` command touches five distinct pieces of state:

pattern -->		libregex Reprog (NFA)		compiled RE
+-----+				
rhsbuf :		replacement with ESCFLG		parsed by compsub()
		markers for \1..\9 and &		
+-----+				
linebuf :		current line from tfile		
+-----+				
		loc1 (match start)	loc2 (match end)	
		set by match() via subexp[0]		
+-----+				
genbuf :		scratch: being assembled		
		[linebuf..loc1) + rhs		
		expanded + [loc2..end)		
+-----+				

The flow of a substitution is always the same: (1) `getline()`⁵⁸ fills `linebuf`^{41c} from `tfile`; (2) `match()` runs `rregexexec()` on `linebuf` and records the hit as `loc1/loc2` pointers *into* `linebuf`; (3) `dosub()`^{75d} walks `linebuf` and `rhsbuf` together, emitting the result into `genbuf`, then copies `genbuf` back into `linebuf`; (4) `putline()`^{54c} writes `linebuf` to `tfile` and the new offset replaces the old one in `zero`. The two-buffer dance is what makes the `g` flag work: after `dosub()` the new `loc2` points into the updated `linebuf`, so a fresh `match(nil)` can resume scanning right where the previous hit ended.

C.10.1 Search as addresses: `/re/` and `?re?`

When `/re/` or `?re?` appears as an address, `address()`^{67d} compiles the pattern and scans through the buffer line by line, *wrapping around* from `dol` to `zero` (or vice versa for `?re?` backward search), until a match is found or we return to where we started.

```

<address() (ed.c) switch c cases 70>+≡ (67d) <69b 80a>
case '?:':
    sign = -sign;
    // Fallthrough
case '/':
    // read the regexp and compile it in Reprog pattern[]
    compile(c);
    b = a;
    for(;;) {
        // direction
        a += sign;
        // wrap around
        if(a <= zero)
            a = dol;
        if(a > dol)
            a = zero;

        if(match(a))
            break;
    }

```

```

    // reached back where we were without finding anything
    if(a == b)
        error(Q);
}
break;

```

Uses Q 45c, compile() 71c, dol 40, error() 93b, match() 72d, and zero 39f.

C.10.2 Reading and compiling a regexp: compile()

compile() 71c reads a regular expression delimited by the eof parameter (typically '/' or '?'), handles backslash escapes, and passes the result to regcomp() (from libregexp, see LIBCORE book [Pad16b]). If the delimiter is given with no pattern in between, the previously compiled pattern is reused—this is how // means “repeat the last search.”

```

⟨globals ed.c 71a⟩+≡ (95) <66a 72c>
    Reprog *pattern;

```

```

⟨constants ed.c 71b⟩+≡ (95) <48a 72b>
    ESIZE    = 256,        /* max size of reg exp */

```

```

⟨function compile(ed.c) 71c⟩≡ (95)
    /// (main -> address) | (commands('g' | 'v') -> global) -> <>
    void
    compile(int eof)
    {
        Rune c;
        char *ep;
        // UTF8 string
        char expbuf[ESIZE];

        c = getch();
        if(c == '\n') {
            peekc = c;
            c = eof;
        }
        if(c == eof) {
            if(!pattern)
                error(Q);
            return;
        }
        // else
        if(pattern) {
            free(pattern);
            pattern = nil;
        }
        ep = expbuf;
        do {
            if(c == '\\') {
                ⟨compile() (ed.c) sanity check ep inside expbuf 72a⟩
                // else
                ep += runetochar(ep, &c);
                c = getch();
                if(c == '\n') {
                    error(Q);
                    return;
                }
            }
        }
        ⟨compile() (ed.c) sanity check ep inside expbuf 72a⟩
    }

```



```

        // else
        ep += runetochar(ep, &c);
    } while((c = getchar()) != eof && c != '\n');
    if(c == '\n')
        peekc = c;
    *ep = 0;

    // lib_regexp call
    pattern = regcomp(expbuf);
}

```

Uses Q 45c, error() 93b, getchar() 47d, pattern 71a, and peekc 47a.

```

⟨compile() (ed.c) sanity check ep inside expbuf 72a⟩≡ (71)
    if(ep >= expbuf+sizeof(expbuf)) {
        error(Q);
        return;
    }

```

Uses Q 45c and error() 93b.

C.10.3 match()

match()^{72d} tests whether the given line matches pattern^{71a}. When addr is non-null, it calls getline()⁵⁸ to load the line into linebuf first; when addr is null, it retries the match from loc2 (the end of the previous match on the same line), which is how s///g finds successive matches. On success, loc1 and loc2 bracket the matched region in linebuf.

```

⟨constants ed.c 72b⟩+≡ (95) <71b 73g>
    MAXSUB = 9, /* max number of sub reg exp */

```

```

⟨globals ed.c 72c⟩+≡ (95) <71a 73b>
    Resub subexp[MAXSUB];

```

Uses MAXSUB-5 72b.

```

⟨function match(ed.c) 72d⟩≡ (95)
    bool
    match(int *addr)
    {
        ⟨match() (ed.c) return if no pattern 72e⟩
        if(addr){
            ⟨match() (ed.c) return if addr is zero 73a⟩
            subexp[0].s.rsp = getline(*addr);
        } else
            ⟨match() (ed.c) when null addr use loc2 not getline 73e⟩
            subexp[0].e.rep = nil;

        if(rregexexec(pattern, linebuf, subexp, MAXSUB)) {
            ⟨match() (ed.c) set loc1 and loc2 with matched string 73c⟩
            return true;
        }
        // else
        ⟨match() (ed.c) reset loc1 and loc2 73d⟩
        return false;
    }

```

Uses MAXSUB-5 72b, getline() 58, linebuf 41c, pattern 71a, and subexp 72c.

```

⟨match() (ed.c) return if no pattern 72e⟩≡ (72d)
    if(!pattern)
        return false;

```

Uses pattern 71a.

`<match() (ed.c) return if addr is zero 73a>≡` (72d)

```
    if(addr == zero)
        return false;
```

Uses zero 39f.

`<globals ed.c 73b>+≡` (95) `<72c 73f>`

```
Rune*   loc1;
Rune*   loc2;
```

`<match() (ed.c) set loc1 and loc2 with matched string 73c>≡` (72d)

```
    loc1 = subexp[0].s.rsp;
    loc2 = subexp[0].e.rep;
```

Uses loc1 73b, loc2 73b, and subexp 72c.

`<match() (ed.c) reset loc1 and loc2 73d>≡` (72d)

```
    loc1 = loc2 = nil;
```

Uses loc1 73b and loc2 73b.

`<match() (ed.c) when null addr use loc2 not getline 73e>≡` (72d)

```
    subexp[0].s.rsp = loc2;
```

Uses loc2 73b and subexp 72c.

C.10.4 Reading and compiling a substitution: compsub()

`compsub()` ^{73h} reads the full `s/re/replacement/` command. It first calls `compile()` ^{71c} to handle the left-hand side (the pattern), then reads the right-hand side into `rhsbuf` ^{73f}, handling backslash escapes and the `ESCFLG` marker for group references like `\1`. It returns true if the trailing `g` flag is present (global substitution on the line).

`<globals ed.c 73f>+≡` (95) `<73b 75a>`

```
Rune    rhsbuf[LBSIZE/sizeof(Rune)];
```

Uses `LBSIZE-2` 41b.

`<constants ed.c 73g>+≡` (95) `<72b 80g>`

```
ESCFLG  = Runemax, /* escape Rune - user defined code */
```

`<function compsub(ed.c) 73h>≡` (95)

```
bool
compsub(void)
{
    int seof, c;
    Rune *p;

    seof = getch();
    if(seof == '\n' || seof == ' ')
        error(Q);

    // read (and compile) the regexp (left hand side)
    compile(seof);

    // read the subst (right hand side)
    p = rhsbuf;
    for(;;) {
        c = getch();
        <compsub() (ed.c) if match group 77c>
        else
        <compsub() (ed.c) if newline and no globp 74b>
        else
```

```

        if(c == seof)
            break;
    // else
    *p++ = c;
    <compsub() (ed.c) sanity check p inside rhsbuf 74a>
}
*p = 0;
<compsub() (ed.c) peek to check for 'g' 78e>
// else
newline();
return false;
}

```

Uses Q 45c, compile() 71c, error() 93b, getch() 47d, newline() 60a, and rhsbuf 73f.

```

<compsub() (ed.c) sanity check p inside rhsbuf 74a>≡ (77c 73h)
    if(p >= &rhsbuf[LBSIZE/sizeof(Rune)])
        error(Q);

```

Uses LBSIZE-2 41b, Q 45c, error() 93b, and rhsbuf 73f.

```

<compsub() (ed.c) if newline and no globp 74b>≡ (73h)
    if(c == '\n' && (!globp || !globp[0])) {
        peekc = c;
        pflag = true;
        break;
    }

```

Uses globp 47c, peekc 47a, and pflag 85d.

C.10.5 substitute: s

We are now finally ready to see the code to perform substitutions. `substitute()`^{74d} is the most complex function. For each line in the address range, it calls `match()`^{72d} to find the pattern, then `dosub()`^{75d} to build the replacement in `genbuf`^{75c}, copying everything before `loc1`, the replacement text (expanding `&` and `\1-9` references), and everything after `loc2`. The result is copied back to `linebuf` and written to `tfile` via `putline()`^{54c}, replacing the old `zero` entry. With the suffix `g` flag, the match/dosub loop repeats on the same line via `match(nil)`; the optional skip count (`s3/re/.../`) skips the first `n-1` matches.

```

<commands() switch c cases (ed.c) 74c>+≡ (49b) <63b 78d>
    case 's':
        nonzero();
        substitute(globp != nil);
        continue;

```

Uses globp 47c, nonzero() 60b, and substitute() 74d.

```

<function substitute(ed.c) 74d>≡ (95)
    void
    substitute(int inglob)
    {
        int *a1;
        bool gsubf; // s/.../.../g global subst
        int n = 0;
        <substitute() (ed.c) other locals 78g>

        <substitute() (ed.c) read optional n 78f>

        gsubf = compsuh();

        for(a1 = addr1; a1 <= addr2; a1++) {

```

```

// will internally set linebuf[]
if(match(a1)){

    int *ozero;
    int m = n;

    do {
        int span = loc2-loc1;

        if(--m <= 0) {
            // will modify linebuf[]
            dosub();

            if(!gsubf)
                break;
            // else
            if(span == 0) { /* null RE match */
                if(*loc2 == 0)
                    break;
                loc2++;
            }
        }
    } while(match(nil));

    if(m <= 0) {
        inglob |= 01;
        // will use linebuf[]
        subnewa = putline();
        *a1 &= ~01;
        <substitute() (ed.c) after putline() if anymarks 80b>
        <substitute() (ed.c) after putline() set subolda 78c>
        *a1 = subnewa;

        <substitute() (ed.c) after putline() call append() 79a>
    }
}
}
if(inglob == 0)
    error(Q);
}

```

Uses Q 45c, addr1 49c, addr2 49c, compsub() 73h, dosub() 75d, error() 93b, loc2 73b, match() 72d, putline() 54c, and subnewa 75a.

<globals ed.c 75a>+≡ (95) <73f 75c>
 int subnewa;

<init() (ed.c) initializing globals 75b>+≡ (44a) <44b 79f>
 subnewa = 0;

Uses subnewa 75a.

<globals ed.c 75c>+≡ (95) <75a 78b>
 Rune genbuf[LBSIZE];

Uses LBSIZE-2 41b.

`dosub()` builds the replacement in `genbuf`: it copies `linebuf`^{41c} up to `loc1` (the match start), expands the replacement string from `rhsbuf`^{73f} (handling `&` for the whole match and `\1-\\9` for subgroups via `place()`^{77b}), then appends the rest of `linebuf` after `loc2`. Finally it copies `genbuf` back to `linebuf`.

<function dosub(ed.c) 75d>≡ (95)
 void

```
dosub(void)
{
    Rune *lp, *sp, *rp;
    int c, n;

    lp = linebuf;
    sp = genbuf;
    rp = rhsbuf;
    while(lp < loc1)
        *sp++ = *lp++;
    while(c = *rp++) {
        <dosub() (ed.c) if c == '&' 77a>
        <dosub() (ed.c) if match group 78a>
        // else
        *sp++ = c;
        if(sp >= &genbuf[LBSIZE])
            error(Q);
    }
    lp = loc2;
    loc2 = (sp - genbuf) + linebuf;
    while(*sp++ = *lp++)
        if(sp >= &genbuf[LBSIZE])
            error(Q);
    lp = linebuf;
    sp = genbuf;
    while(*lp++ = *sp++)
        ;
}
```

Uses LBSIZE-2 [41b](#), Q [45c](#), error() [93b](#), genbuf [75c](#), linebuf [41c](#), loc1 [73b](#), loc2 [73b](#), and rhsbuf [73f](#).

The line `loc2 = (sp - genbuf) + linebuf` is the trickiest part: after assembly, `sp` points into `genbuf` where the tail (post-match text) was copied. This arithmetic converts that `genbuf` offset back into a `linebuf` pointer, so that the next call to `match(nil)` in the `g` flag loop will resume searching from the correct position in the updated `linebuf`.

A concrete trace of `s/world/earth/` on `linebuf = "hello world!"` shows the four phases of `dosub()`:

```
initial state (after match() on linebuf):
```

```
linebuf: "hello world!\0"
          ^      ^      ^
          |      loc1 loc2
          lp starts here
rhsbuf:  "earth\0"
genbuf:  (empty, sp at start)
```

```
phase 1: copy linebuf[..loc1) -> genbuf
```

```
genbuf:  "hello "
          ^sp
```

```
phase 2: expand rhsbuf into genbuf
```

```
(no & or \N, so plain copy)
genbuf:  "hello earth"
          ^sp
```

```
phase 3: adjust pointers
```

```
loc2 = (sp - genbuf) + linebuf
```

```

    =    11          + linebuf
    -> points into linebuf at offset 11 (the '!')
This is where the next match(nil) would start.

```

```

phase 4: copy linebuf[loc2..] -> genbuf, then genbuf -> linebuf
genbuf:  "hello earth!\0"
linebuf: "hello earth!\0"

```

Phase 3 is subtle because `loc2` must end up pointing into the *new* `linebuf`, not the old one. At the moment of the assignment, `linebuf` still holds the pre-substitution text, but we know the new prefix (“hello earth”) will occupy bytes `linebuf[0..11]` once phase 4 runs. So `sp-genbuf` (the length of that prefix) plus `linebuf` (the base) is the correct future location. It is a deliberate compute-first, copy-later trick—if the assignment were reordered after the final copy, `sp` would already have advanced and the arithmetic would break.

C.10.6 Advanced substitutions

The code of `dosub()`^{75d}, `compsub()`^{73h}, and `substitute()`^{74d} was simplified before to show just the core code path. Many additional search and replace features require additional code as shown in the following sections.

Matched string reference: `s/.../...&.../`

```

⟨dosub() (ed.c) if c == '&' 77a⟩≡ (75d)
    if(c == '&'){
        sp = place(sp, loc1, loc2);
        continue;
    }

```

Uses `loc1` 73b, `loc2` 73b, and `place()` 77b.

```

⟨function place(ed.c) 77b⟩≡ (95)
    Rune*
    place(Rune *sp, Rune *l1, Rune *l2)
    {
        while(l1 < l2) {
            *sp++ = *l1++;
            if(sp >= &genbuf[LBSIZE])
                error(Q);
        }
        return sp;
    }

```

Uses `LBSIZE-2` 41b, `Q` 45c, `error()` 93b, and `genbuf` 75c.

Matched groups: `s/.../...\1.../`

```

⟨compsub() (ed.c) if match group 77c⟩≡ (73h)
    if(c == '\\') {
        c = getch();
        *p++ = ESCFLG;
        ⟨compsub() (ed.c) sanity check p inside rhsbuf 74a⟩
    }

```

Uses `ESCFLG-6` 73g and `getchr()` 47d.

```

<dosub() (ed.c) if match group 78a>≡ (75d)
    if(c == ESCFLG && (c = *rp++) >= '1' && c < MAXSUB+'0') {
        n = c-'0';
        if(subexp[n].s.rsp && subexp[n].e.rep) {
            sp = place(sp, subexp[n].s.rsp, subexp[n].e.rep);
            continue;
        }
        error(Q);
    }
}

```

Uses ESCFLG-6 73g, MAXSUB-5 72b, Q 45c, error() 93b, place() 77b, and subexp 72c.

undo substitution: u

The u command provides a minimal undo: it remembers the previous **zero** entry (subolda^{78b}) before the last substitution replaced it with **subnewa**^{75a}, and swaps it back. This only undoes the most recent substitution on a single line—far from a general undo facility.

```

<globals ed.c 78b>+≡ (95) <75c 79d>
    int subolda;

```

```

<substitute() (ed.c) after putline() set subolda 78c>≡ (74d)
    subolda = *a1;

```

Uses subolda 78b.

```

<commands() switch c cases (ed.c) 78d>+≡ (49b) <74c 79c>
    case 'u':
        nonzero();
        newline();
        if((*addr2&~01) != subnewa)
            error(Q);
        *addr2 = subolda;
        dot = addr2;
        continue;

```

Uses Q 45c, addr2 49c, dot 40, error() 93b, newline() 60a, nonzero() 60b, subnewa 75a, and subolda 78b.

global subst: s/.../.../g

```

<compsub() (ed.c) peek to check for 'g' 78e>≡ (73h)
    peekc = getch();
    if(peekc == 'g') {
        peekc = 0;
        newline();
        return true;
    }

```

Uses getch() 47d, newline() 60a, and peekc 47a.

Skipped matches: s3/.../.../

```

<substitute() (ed.c) read optional n 78f>≡ (74d)
    n = getnum(); /* OK even if n==0 */

```

Uses getnum() 69a.

Append and subst

```

<substitute() (ed.c) other locals 78g>≡ (74d)
    int *mp, nl;

```

`<substitute() (ed.c) after putline() call append() 79a>≡` (74d)

```
ozer = zero;
nl = append(getsub, a1);
addr2 += nl;
nl += zero-ozer;
a1 += nl;
```

Uses `addr2 49c`, `append() 52b`, `getsub() 79b`, and `zero 39f`.

`<function getsub(ed.c) 79b>≡` (95)

```
int
getsub(void)
{
    Rune *p1, *p2;

    p1 = linebuf;
    if((p2 = linebp) == 0)
        return EOF;
    while(*p1++ = *p2++)
        ;
    linebp = 0;
    return 0;
}
```

Uses `EOF-3 48a`, `linebp 88d`, and `linebuf 41c`.

mark: k

The **k** command *marks* a line with a lowercase letter (**ka** through **kz**). The mark is stored in `names[c-'a']` as the line's **zero** entry (with the low bit cleared—since that bit is used by `global()`^{81a}). The ' (apostrophe) address retrieves a marked line by scanning **zero** for the matching offset.

`<commands() switch c cases (ed.c) 79c>+≡` (49b) <78d 80e>

```
case 'k':
    nonzero();
    c = getchr();
    if(c < 'a' || c > 'z')
        error(Q);
    newline();
    names[c-'a'] = *addr2 & ~01;
    anymarks |= 01;
    continue;
```

Uses `Q 45c`, `addr2 49c`, `anymarks 79e`, `error() 93b`, `getchr() 47d`, `names 79d`, `newline() 60a`, and `nonzero() 60b`.

`<globals ed.c 79d>+≡` (95) <78b 79e>

```
int names[26];
```

`<globals ed.c 79e>+≡` (95) <79d 83b>

```
int anymarks;
```

`<init() (ed.c) initializing globals 79f>+≡` (44a) <75b 80d>

```
anymarks = 0;
```

Uses `anymarks 79e`.


```

⟨address() (ed.c) switch c cases 80a⟩+≡ (67d) <70
case '\':
    c = getchar();
    if(opcnt || c < 'a' || c > 'z')
        error(Q);
    a = zero;
    do {
        a++;
    } while(a <= dol && names[c-'a'] != (*a & ~01));
    break;

```

Uses Q 45c, dol 40, error() 93b, getchar() 47d, names 79d, and zero 39f.

```

⟨substitute() (ed.c) after putline() if anymarks 80b⟩≡ (74d)
if(anymarks) {
    for(mp=names; mp<&names[26]; mp++)
        if(*mp == *a1)
            *mp = subnewa;
}

```

Uses anymarks 79e, names 79d, and subnewa 75a.

```

⟨init() (ed.c) locals 80c⟩≡ (44a)
int *markp;

```

```

⟨init() (ed.c) initializing globals 80d⟩+≡ (44a) <79f 89c>
for(markp = names; markp < &names[26]; )
    *markp++ = 0;

```

Uses names 79d.

C.10.7 global commands: g/re/cmd and v/re/cmd

The **g/re/cmd** command (and its inverse **v/re/cmd**) works in two phases. In phase 1, it scans all lines in the address range and *marks* those that match (or don't match, for **v**) the pattern by setting the low bit of their **zero** entry (***a1** != 01—this is the “bit-stealing trick” that **tline** starting at 2 makes possible). In phase 2, it iterates over **zero**, and for each marked line, clears the mark, sets **globp**^{47c} to the command string (stored in the local **globuf**), and calls **commands()**^{49b} recursively. The two-phase design is necessary because executing commands can change the buffer (e.g., **g/re/d** deletes lines), so we cannot match and execute at the same time.

```

⟨commands() switch c cases (ed.c) 80e⟩+≡ (49b) <79c 80f>
case 'g':
    global(1);
    continue;

```

Uses global() 81a.

```

⟨commands() switch c cases (ed.c) 80f⟩+≡ (49b) <80e 83a>
case 'v':
    global(0);
    continue;

```

Uses global() 81a.

```

⟨constants ed.c 80g⟩+≡ (95) <73g 88e>
GBSIZE = 256, /* max size of global command */

```

<function global(ed.c) 81a>≡ (95)

```
void
global(int k)
{
    Rune *gp, globuf[GBSIZE];
    int c, *a1;

    <global() sanity check not already in global command 82>

    setwide();
    squeeze(dol > zero);

    // readding a '/' or '?'
    c = getchr();
    if(c == '\n')
        error(Q);

    // reading the whole pattern until corresponding ending '/' or '?'
    compile(c);

    gp = globuf;
    <global() read the global command in globuf 81b>
    if(gp == globuf)
        *gp++ = 'p';
    *gp++ = '\n';
    *gp = 0;

    // phase 1
    for(a1=zero; a1<=dol; a1++) {
        *a1 &= ~01;
        if(a1 >= addr1 && a1 <= addr2 && match(a1) == k)
            *a1 |= 01;
    }
    <global() (ed.c) if g/.../d command, call optimized gdelete() 92a>

    // phase 2
    for(a1=zero; a1<=dol; a1++) {
        if(*a1 & 01) {
            *a1 &= ~01;
            dot = a1;
            globp = globuf;
            // recurse!
            commands();
            a1 = zero; // zero may have grown and move, need restart from zero
        }
    }
}
```

Uses Q 45c, addr1 49c, addr2 49c, commands() 49b, compile() 71c, dol 40, dot 40, error() 93b, getchr() 47d, globp 47c, match() 72d, setwide() 51d, squeeze() 52a, and zero 39f.

<global() read the global command in globuf 81b>≡ (81a)

```
while((c=getchr()) != '\n') {
    if(c == EOF)
        error(Q);
    if(c == '\\') {
        c = getchr();
        if(c != '\n')
            *gp++ = '\\';
    }
    *gp++ = c;
}
```

```

    if(gp >= &globuf[GBSIZE-2])
        error(Q);
}

```

Uses EOF-3 48a, GBSIZE-7 80g, Q 45c, error() 93b, and getch() 47d.

There is an important subtlety in the phase 2 loop: after each `commands()` call, `a1` is reset to `zero` rather than just incrementing. This is because `commands()` during that recursive call may call `append()`^{52b}, which may `realloc` the `zero` array, moving it to a new address. All saved pointers into the old array become dangling. Resetting `a1 = zero` picks up the (possibly new) base, and the loop re-scans from the beginning—this is safe because each executed line has its mark cleared, so it won't be processed again.

Visually, a `g/foo/d` on a four-line buffer goes through the following two phases. Bit 0 of each `zero` entry is stolen as the “matched” mark; I show it as `M` when set, and `'.'` when clear:

initial:	after phase 1 (mark):
zero[1] = 0x40	zero[1] = 0x40 . "alpha"
zero[2] = 0x48	zero[2] = 0x49 M "foobar" <- matched
zero[3] = 0x56	zero[3] = 0x56 . "beta"
zero[4] = 0x62	zero[4] = 0x63 M "foo" <- matched


```

phase 2 iteration:
    a1 = zero+1    .    skip
    a1 = zero+2    M    clear mark; dot=a1;
                     globp = "d\n"; commands();
                     (runs 'd' -> rdelete shifts entries down)
                     a1 = zero (restart from base)

    a1 = zero+1    .    skip (old alpha)
    a1 = zero+2    .    skip (was beta, shifted down)
    a1 = zero+3    M    clear mark; delete; a1 = zero
    a1 = zero+1    .    skip
    a1 = zero+2    .    skip
    dol is now zero+2    loop exits

```

Three consequences follow from this design. (1) The marks must live *inside zero* rather than in a parallel bitmap, because `rdelete()`^{62c} for example shifts `zero` entries around and a parallel bitmap would go out of sync instantly. Putting the mark in the low bit of the offset means delete, move, and substitute all carry marks along for free. (2) The low bit is safe to steal because `tline` starts at 2 and grows in multiples of 2 (aligned Rune boundaries), so legitimate offsets never have bit 0 set. (3) The restart-from-zero loop is $O(n^2)$ in the worst case (one full pass per matched line), which is why `gdelete()`^{92b} exists as a special case for `g/.../d` (see Section C.12.4).

With this in hand, the origin of `grep` is now obvious. The classic `ed` one-liner `g/re/p` means “for every line matching the regular expression `re`, execute the `p` command (print)”—i.e., global / regular expression / print. This was such a common idiom that Ken Thompson and the early UNIX folks pulled it out into its own standalone tool

```

(global() sanity check not already in global command 82)≡ (81a)
    if(globp)
        error(Q);

```

Uses Q 45c, error() 93b, and globp 47c.

C.11 Advanced features

The previous sections covered the core editing commands, the addressing system, and search-and-replace. This section collects the remaining commands of `ed`. Most of those commands do not introduce new mechanisms—

they are mostly small additions to the command dispatcher in `commands()`^{49b}.

C.11.1 Running a shell command: !

The `!` command forks a child process, executes the rest of the line via `/bin/rc -c` (see the SHELL book [Pad18]), and waits for it to finish. In verbose mode it prints `!` when the command completes. The `!` command is less useful on Plan 9 where you typically have multiple windows, but was essential back in the old UNIX days on single-terminal systems.

```
<commands() switch c cases (ed.c) 83a>+≡ (49b) <80f 85b>▷
  case '!':
    callunix();
    continue;
```

Uses `callunix()` 83c.

```
<globals ed.c 83b>+≡ (95) <79e 84a>▷
  bool waiting;
```

```
<function callunix(ed.c) 83c>≡ (95)
  void
  callunix(void)
  {
    int c, pid;
    Rune rune;
    char buf[512];
    char *p;

    setnoaddr();

    p = buf;
    while((c=getchr()) != EOF && c != '\n')
      if(p < &buf[sizeof(buf) - 6]) {
        rune = c;
        p += runetochar(p, &rune);
      }
    *p = '\0';

    pid = fork();
    if(pid == 0) {
      // child
      execl("/bin/rc", "rc", "-c", buf, nil);
      // should not be reached
      exits("execl failed");
    }
    // else, parent
    waiting = true;
    while(waitpid() != pid)
      ;
    waiting = false;
    if(vflag)
      putst("!");
  }
```

Uses `EOF-3` 48a, `getchr()` 47d, `putst()` 45d, `setnoaddr()` 60e, `vflag` 42a, and `waiting` 83b.

C.11.2 Advanced listing: l

The `l` (list) command is like `p` but makes non-printing characters visible: tabs become `\t`, backspaces `\b`, and other control or high characters are shown as `\xNNNN` hex escapes. Long lines are folded at column 72. The

listf flag is checked inside putchar() ^{46b} to trigger this special rendering.

```
<globals ed.c 84a>+≡ (95) <83b 84d>
// for displaying special chars, 'l' list flag
bool listf;
```

```
<commands before 'p' case 84b>≡ (59d)
case 'l':
    listf = true;
    // fallthrough:
```

Uses listf 84a.

```
<putchr() if listf 84c>≡ (46b)
if(listf) {
    if(c == '\n') {
        if(linp != line && linp[-1] == ' ') {
            *lp++ = '\\';
            *lp++ = 'n';
        }
    } else {
        if(col > (72-6-2)) {
            col = 8;
            *lp++ = '\\';
            *lp++ = 'n';
            *lp++ = 't';
        }
        col++;
        if(c=='\b' || c=='\t' || c=='\\') {
            *lp++ = '\\';
            if(c == '\b')
                c = 'b';
            else
                if(c == '\t')
                    c = 't';
            col++;
        } else
            if(c<' ' || c>='\177') {
                *lp++ = '\\';
                *lp++ = 'x';
                *lp++ = hex[c>>12];
                *lp++ = hex[c>>8&0xF];
                *lp++ = hex[c>>4&0xF];
                c = hex[c&0xF];
                col += 5;
            }
    }
}
```

Uses col 41e, hex 84d, line 41a, linp 41a, and listf 84a.

```
<globals ed.c 84d>+≡ (95) <84a 85a>
char hex[] = "0123456789abcdef";
```

Uses hex 84d.

C.11.3 Advanced listing: n

The n suffix prepends the line number before each printed line. It is useful when the user is working with addresses and wants to remember which line is which without typing = repeatedly. Like l, n is implemented as

a flag set in `newline()`^{60a} and consumed by `printcom()`^{59e}.

```
<globals ed.c 85a>+≡ (95) <84d 85d>
// 'n' flag
bool listn;

<commands() switch c cases (ed.c) 85b>+≡ (49b) <83a 86a>
case 'n':
    listn = true;
    newline();
    printcom();
    continue;
```

Uses `listn` 85a, `newline()` 60a, and `printcom()` 59e.

```
<printcom() if listn 85c>≡ (59e)
if(listn) {
    count = al-zero;
    putd();
    putchar(L'\t');
}
```

Uses `count` 41d, `listn` 85a, `putchr()` 46b, `putd()` 56b, and `zero` 39f.

C.11.4 Automatic listing: the ...pln suffix flag

```
<globals ed.c 85d>+≡ (95) <85a 86d>
bool pflag;
```

The `pflag` mechanism lets commands request an automatic print after execution. For example, `s/foo/bar/p` sets `pflag`, and at the top of the `commands()`^{49b} loop, if `pflag` is set, `dot` is printed before reading the next command. Actually the suffixes `p`, `l`, and `n` can be appended to many commands (handled in `newline()`^{60a}).

```
<commands() in for loop, if pflag 85e>≡ (49b)
if(pflag) {
    pflag = false;
    addr1 = addr2 = dot;
    printcom();
}
```

Uses `addr1` 49c, `addr2` 49c, `dot` 40, `pflag` 85d, and `printcom()` 59e.

```
<newline() if special chars pln 85f>≡ (60a)
if(c == 'p' || c == 'l' || c == 'n') {
    pflag = true;
    if(c == 'l')
        listf = true;
    else
        if(c == 'n')
            listn = true;
    c = getch();
    if(c == '\n')
        return;
}
```

Uses `getchr()` 47d, `listf` 84a, `listn` 85a, and `pflag` 85d.

Note that `pflag` is set for commands that internally calls `newline()`, e.g., `s/foo/p`, `3dp`, but not for commands such as `g/foo/p` which use a different mechanism (`globp` and recursive calls to `commands()`).

```
<printcom() reset flags at the end 85g>≡ (59e)
listf = false;
listn = false;
pflag = false;
```

Uses `listf` 84a, `listn` 85a, and `pflag` 85d.

C.11.5 joining lines: j

The `j` command concatenates multiple lines into one. It copies all lines in the range into `genbuf` end-to-end, writes the result back as a single line via `putline()`^{54c}, then deletes the extra lines with `rdelete()`^{62c}.

```
<commands() switch c cases (ed.c) 86a>+≡ (49b) <85b 86c>
case 'j':
    if(!given)
        addr2++;
    newline();
    join();
    continue;
```

Uses `addr2` 49c, `given` 66a, `join()` 86b, and `newline()` 60a.

```
<function join(ed.c) 86b>≡ (95)
void
join(void)
{
    Rune *gp, *lp;
    int *a1;

    nonzero();
    gp = genbuf;
    for(a1=addr1; a1<=addr2; a1++) {
        lp = getline(*a1);
        while(*gp = *lp++)
            if(gp++ >= &genbuf[LBSIZE-sizeof(Rune)])
                error(Q);
    }
    lp = linebuf;
    gp = genbuf;
    while(*lp++ = *gp++)
        ;
    *addr1 = putline();
    if(addr1 < addr2)
        rdelete(addr1+1, addr2);
    dot = addr1;
}
```

Uses `LBSIZE-2` 41b, `Q` 45c, `addr1` 49c, `addr2` 49c, `dot` 40, `error()` 93b, `genbuf` 75c, `getline()` 58, `linebuf` 41c, `nonzero()` 60b, `putline()` 54c, and `rdelete()` 62c.

C.11.6 browse: b

The `b` (browse) command is a paging facility: it prints `bpagesize` lines (default 20) starting from the current address. You can go forward or backward with `b+` or `b-`, and change the page size with `b30`.

```
<commands() switch c cases (ed.c) 86c>+≡ (49b) <86a 87b>
case 'b':
    nonzero();
    browse();
    continue;
```

Uses `browse()` 87a and `nonzero()` 60b.

```
<globals ed.c 86d>+≡ (95) <85d 88c>
int bpagesize = 20;
```

Uses `bpagesize` 86d.

```

⟨function browse(ed.c) 87a)≡ (95)
void
browse(void)
{
    int forward;
    int n;
    static int bformat, bnum; /* 0 */

    forward = 1;
    peekc = getchr();
    if(peekc != '\n'){
        if(peekc == '-' || peekc == '+') {
            if(peekc == '-')
                forward = 0;
            getchr();
        }
        n = getnum();
        if(n > 0)
            bpagesize = n;
    }
    newline();
    if(pflag) {
        bformat = listf;
        bnum = listn;
    } else {
        listf = bformat;
        listn = bnum;
    }
    if(forward) {
        addr1 = addr2;
        addr2 += bpagesize;
        if(addr2 > dol)
            addr2 = dol;
    } else {
        addr1 = addr2-bpagesize;
        if(addr1 <= zero)
            addr1 = zero+1;
    }
    printcom();
}

```

Uses addr1 49c, addr2 49c, bpagesize 86d, dol 40, getchr() 47d, getnum() 69a, listf 84a, listn 85a, newline() 60a, peekc 47a, pflag 85d, printcom() 59e, and zero 39f.

C.11.7 edit remembered file: e

The **e** command re-initializes the editor (calling `init()`^{44a} to reset `tfile`^{39b} and `zero`^{39f}) and reads a new file, effectively starting a fresh editing session. **E** is the force variant that skips the unsaved-changes check.

```

⟨commands() switch c cases (ed.c) 87b)+≡ (49b) <86c
case 'E':
    fchange = false;
    c = 'e';
    // Fallthrough
case 'e':
    setnoaddr();
    if(vflag && fchange) {
        fchange = false;
        error(Q);
    }

```



```

filename(c);
init();
addr2 = zero;
goto caseread;

```

Uses Q 45c, addr2 49c, error() 93b, fchange 45a, filename() 50e, init() 44a, setnoaddr() 60e, vflag 42a, and zero 39f.

C.11.8 Append-only files

Plan 9 supports append-only files (the DMAPPEND mode bit, see the KERNEL book [Pad14]), used for log files that can only grow. ed checks for this flag when opening a file for reading and prints a warning, since writing back to such a file would fail.

```

⟨commands() other locals 88a⟩+≡ (49b) <66c
    Dir *d;

```

```

⟨commands() in r case, if append only file 88b⟩≡ (50c)
    if((d = dirfstat(io)) != nil){
        if(d->mode & DMAPPEND)
            print("warning: %s is append only\n", file);
        free(d);
    }

```

Uses file 50a and io 50b.

C.12 Optimizations

The simplified putline() ^{54c} and getline() ⁵⁸ shown earlier do a seek+write/read for every single line. The original ed versions use a (userspace) block cache: getblock() ^{89d} maintains two BLKSIZE-byte buffers (ibuff for reads, obuff for writes) and only performs actual I/O when the requested block is not already cached. This was essential on early UNIX systems that lacked a kernel buffer cache, and it remains a significant optimization when editing large files.

C.12.1 Optimized putline()

The real putline() ^{54c} writes linebuf to tfile through getblock() ^{89d}, handling block boundaries: when the current block is full (nleft == 0), it advances to the next block. The offset arithmetic with tline uses block-aligned addresses.

```

⟨globals ed.c 88c⟩+≡ (95) <86d 88d>
    int nleft;

```

```

⟨globals ed.c 88d⟩+≡ (95) <88c 89b>
    Rune* linebp;

```

```

⟨constants ed.c 88e⟩+≡ (95) <80g 90>
    BLKSIZE = 4096, /* block size in temp file */

```

```

⟨putline() if opti 88f⟩≡ (54c)
    if(opti) return putline_opti();

```

Uses opti 95 and putline_opti() 89a.

```

⟨function putline_opti(ed.c) 89a⟩≡ (95)
  /// main -> commands('r') -> append -> putline -> <>
  int
  putline_opti(void)
  {
    Rune *lp, *bp;
    int nl, tl;

    fchange = true;
    lp = linebuf;
    tl = tline;

    bp = getblock(tl, OWRITE);
    nl = nleft;
    tl &= ~((BLKSIZE/sizeof(Rune))-1);
    while(*bp = *lp++) {
      if(*bp++ == '\n') {
        bp[-1] = 0;
        linebp = lp;
        break;
      }
      nl -= sizeof(Rune);
      if(nl == 0) {
        tl += BLKSIZE/sizeof(Rune);
        bp = getblock(tl, OWRITE);
        nl = nleft;
      }
    }

    nl = tline;
    tline += ((lp-linebuf) + 03) & 077776;
    return nl;
  }

```

Uses BLKSIZE-8 88e, fchange 45a, getblock() 89d, linebp 88d, linebuf 41c, nleft 88c, and tline 39c.

C.12.2 getblock()

getblock() ^{89d} is the heart of the block cache. It converts a line offset `atl` into a block number and an offset within that block.

```

⟨globals ed.c 89b⟩+≡ (95) <88d 92c>
  // index in ibuff
  int iblock;
  // index in obuff
  int oblock;

  int ichanged;

⟨init()(ed.c) initializing globals 89c⟩+≡ (44a) <80d
  iblock = -1;
  oblock = -1;
  ichanged = 0;

```

Uses iblock 89b, ichanged 89b, and oblock 89b.

```

⟨function getblock(ed.c) 89d⟩≡ (95)
  /// putline_opti | getline_opti -> <>
  Rune*
  getblock(int atl, int iof)
  {

```

```

int bno; // block number
int off; // offset

static uchar ibuff[BLKSIZE];
static uchar obuff[BLKSIZE];

bno = atl / (BLKSIZE/sizeof(Rune));
/* &~3 so the ptr is aligned to 4 (?) */
off = (atl*sizeof(Rune)) & (BLKSIZE-1) & ~3;
if(bno >= NBLK) {
    lastc = '\n';
    error(T);
}
nleft = BLKSIZE - off;
if(bno == iblock) {
    ichanged |= iof;
    return (Rune*)(ibuff+off);
}
if(bno == oblock)
    return (Rune*)(obuff+off);
if(iof == OREAD) {
    if(ichanged)
        blkio(iblock, ibuff, (long (*)(int, void *, long))write);
    ichanged = 0;
    iblock = bno;
    blkio(bno, ibuff, (long (*)(int, void *, long))read);
    return (Rune*)(ibuff+off);
}
if(oblock >= 0)
    blkio(oblock, obuff, (long (*)(int, void *, long))write);
oblock = bno;
return (Rune*)(obuff+off);
}

```

Uses BLKSIZE-8 [88e](#), NBLK-9 [90](#), T [44c](#), blkio() [91a](#), error() [93b](#), iblock [89b](#), ichanged [89b](#), lastc [47b](#), nleft [88c](#), and oblock [89b](#).

```

⟨constants ed.c 90⟩+≡ (95) <88e
NBLK    = 8191,      /* max size of temp file */

```

If the requested block is already in `ibuff` (the read cache) or `obuff` (the write cache), it returns a pointer directly into the cached buffer—no I/O needed. Otherwise, it evicts the oldest cached block (flushing `ibuff` to disk via `blkio()` [91a](#) if it was modified), loads the new block, and returns a pointer into it.

The `ichanged` flag is the dirty bit for `ibuff`: when a write request (`iof == OWRITE`) hits the `ibuff` cache, `ichanged` is set via `ichanged |= iof`. Later, if `ibuff` must be evicted for a read, it is flushed first. The `obuff` path has no such flag—it is always flushed on eviction because it is only used by writes, so it is always dirty.

tfile (on disk):

```

+-----+-----+-----+-----+-----...
| block 0 | block 1 | block 2 | block 3 |
| 4096 B  | 4096 B  | 4096 B  | 4096 B  |
+-----+-----+-----+-----+-----...

```

getblock(atl, iof):

```

    bno = atl / (BLKSIZE/sizeof(Rune))    -> which block
    off = (atl*sizeof(Rune)) & (BLKSIZE-1) -> byte offset within block

```

In-memory caches (ed.c globals):

```

ibuff[4096]  <- iblock (currently cached read block)
obuff[4096]  <- oblock (currently cached write block)
ichanged      (dirty bit for ibuff)

```

Three cases:

1. bno == iblock -> cache hit on read buffer
 return &ibuff[off]
 if iofoff==OWRITE, set ichanged
2. bno == oblock -> cache hit on write buffer
 return &obuff[off]
3. miss -> evict the appropriate buffer
 (flush if ichanged or oblock>=0),
 read or allocate the new block,
 return its address

The split between `ibuff` and `obuff` avoids ping-pong: a typical `ed` session keeps reading lines from one part of `tfile` (cached in `ibuff`) while writing modified lines to the growing tail of `tfile` (cached in `obuff`). With a single buffer the cache would constantly be evicted as the program alternates reads and writes; with two, both stay warm.

```

⟨function blkio(ed.c) 91a⟩≡ (95)
void
blkio(int b, uchar *buf, long (*iofcn)(int, void *, long))
{
    seek(tfile, b*BLKSIZE, SEEK__START);
    if((*iofcn)(tfile, buf, BLKSIZE) != BLKSIZE) {
        error(T);
    }
}

```

Uses `BLKSIZE-8` 88e, `T` 44c, `error()` 93b, and `tfile` 39b.

C.12.3 Optimized `getline()`

```

⟨getline() if opti 91b⟩≡ (58)
if(opti) return getline_opti(tl);

```

Uses `getline_opti()` 91c and `opti` 95.

```

⟨function getline_opti(ed.c) 91c⟩≡ (95)
/// printcom | putfile -> <>
Rune*
getline_opti(int tl)
{
    Rune *lp, *bp;
    int nl;

    lp = linebuf;
    bp = getblock(tl, OREAD);
    nl = nleft;
    tl &= ~((BLKSIZE/sizeof(Rune)) - 1);
    while(*lp++ = *bp++) {
        nl -= sizeof(Rune);
    }
}

```

```

        if(nl == 0) {
            t1 += BLKSIZE/sizeof(Rune);
            bp = getblock(t1, OREAD);
            nl = nleft;
        }
    }
    return linebuf;
}

```

Uses BLKSIZE-8 88e, getblock() 89d, linebuf 41c, and nleft 88c.

C.12.4 gdelete()

The generic `g/re/cmd` loop calls `commands()`^{49b} once per marked line, which for `g/re/d` would result in $O(n^2)$ shifting in `rdelete()`^{62c}. `gdelete()`^{92b} below special-cases this: it makes a single pass over zero, copying unmarked entries down and skipping marked ones, achieving $O(n)$ deletion.

<global() (ed.c) if g/.../d command, call optimized gdelete() 92a>≡ (81a)

```

/*
 * Special case: g/.../d (avoid n^2 algorithm)
 */
if(globuf[0] == 'd' && globuf[1] == '\n' && globuf[2] == 0) {
    gdelete();
    return;
}

```

Uses `gdelete()` 92b.

<function gdelete(ed.c) 92b>≡ (95)

```

void
gdelete(void)
{
    int *a1, *a2, *a3;

    a3 = dol;
    for(a1=zero; (*a1&01)==0; a1++)
        if(a1>=a3)
            return;
    for(a2=a1+1; a2<=a3;) {
        if(*a2 & 01) {
            a2++;
            dot = a1;
        } else
            *a1++ = *a2++;
    }
    dol = a1-1;
    if(dot > dol)
        dot = dol;
    fchange = true;
}

```

Uses `dol` 40, `dot` 40, `fchange` 45a, and `zero` 39f.

C.13 Error management

`ed`'s error handling is based on `setjmp/longjmp` (see the `LIBCORE` book [Pad16b]), which act as a poor man's exception mechanism.

<globals ed.c 92c>+≡ (95) <89b 94a>

```

jmp_buf savej;

```

C.13.1 error()

Before calling `commands()`^{49b}, `main()` calls `setjmp(savej)` to save the execution context.

```
⟨main() (ed.c) before commands() 93a⟩≡ (41g)
    setjmp(savej);
```

Uses `savej` 92c.

When `error()`^{93b} is called anywhere in the program, it prints ? followed by a short message, resets global state, and calls `longjmp(savej, 1)` to jump back to just before `commands()`, ready to read the next command.

```
⟨function error(ed.c) 93b⟩≡ (95)
void
error(char *s)
{
    error_1(s);
    longjmp(savej, 1);
}
```

Uses `error_1()` 93c and `savej` 92c.

```
⟨function error_1(ed.c) 93c⟩≡ (95)
void
error_1(char *s)
{
    int c;

    ⟨error_1() (ed.c) reset globals 93d⟩
    putchar(L'?');
    putst(s);
}
```

Uses `putchr()` 46b and `putst()` 45d.

```
⟨error_1() (ed.c) reset globals 93d⟩≡ (93c)
wrapp = false;
listf = false;
listn = false;
count = 0;

seek(STDIN, 0, SEEK__END);
pflag = false;

if(globp)
    lastc = '\n';
globp = nil;

peekc = lastc;
if(lastc)
    for(;;) {
        c = getchr();
        if(c == '\n' || c == EOF)
            break;
    }

if(io > 0) {
    close(io);
    io = -1;
}
```

Uses EOF-3 48a, `count` 41d, `getchr()` 47d, `globp` 47c, `io` 50b, `lastc` 47b, `listf` 84a, `listn` 85a, `peekc` 47a, `pflag` 85d, and `wrapp` 57a.

C.13.2 Notes/signals management

Plan 9 uses *notes* instead of UNIX *signals* (see the KERNEL book [Pad14]). On an “interrupt” note, `ed` prints a newline and jumps back to the command loop via `notejmp` (the Plan 9 equivalent of `siglongjmp`). On a “hangup” note (terminal disconnected), it calls `rescue()`^{94c}, which saves the buffer to `ed.hup` before exiting—a safety net so you do not lose unsaved work.

```
<globals ed.c 94a>+≡ (95) <92c
    bool rescuing;
```

```
<function notifyf(ed.c) 94b>≡ (95)
void
notifyf(void *a, char *s)
{
    if(strcmp(s, "interrupt") == 0){
        if(rescuing || waiting)
            noted(NCONT);
        putchar(L'\n');
        lastc = '\n';
        error_1(Q);
        notejmp(a, savej, 0);
    }
    if(strcmp(s, "hangup") == 0){
        if(rescuing)
            noted(NDFLT);
        rescue();
    }
    fprintf(STDERR, "ed: note: %s\n", s);
    abort();
}
```

Uses Q 45c, `error_1()` 93c, `lastc` 47b, `putchr()` 46b, `rescue()` 94c, `rescuing` 94a, `savej` 92c, and `waiting` 83b.

```
<function rescue(ed.c) 94c>≡ (95)
void
rescue(void)
{
    rescuing = true;
    if(dol > zero) {
        addr1 = zero+1;
        addr2 = dol;
        io = create("ed.hup", OWRITE, 0666);
        if(io > 0){
            Binit(&iobuf, io, OWRITE);
            putfile();
        }
    }
    fchange = false;
    quit();
}
```

Uses `addr1` 49c, `addr2` 49c, `dol` 40, `fchange` 45a, `io` 50b, `iobuf` 50b, `putfile()` 57c, `quit()` 45b, `rescuing` 94a, and `zero` 39f.

C.14 Conclusion

You now know how the Plan 9 editor `ed` works, to the smallest details, and more generally how many line editors work and to some extent screen editors such as `vi`. Hopefully, you can now understand every commands in the `mkenam` script of Section C.1.2, and understand how each of those commands work internally.

C.15 Extra code

```
<ed/ed.c 95>≡
/*
 * Editor
 */
#include <u.h>
#include <libc.h>

#include <bio.h>
#include <regex.h>

enum
{
    <constants ed.c 39d>
};

<globals ed.c 39a>
bool opti = false;

// forward declarations
void    add(int);
int*    address(void);
int     append(int(*) (void), int*);
void    browse(void);
void    callunix(void);
void    commands(void);
void    compile(int);
bool    compsub(void);
void    dosub(void);
void    error(char*);
bool    match(int*);
void    exfile(int);
void    filename(int);
Rune*   getblock(int, int);
int     getchr(void);
int     getcopy(void);
int     getfile(void);
Rune*   getline(int);
int     getnum(void);
int     getsub(void);
int     gettty(void);
void    global(int);
void    init(void);
void    join(void);
void    move(int);
void    newline(void);
void    nonzero(void);
void    notifyf(void*, char*);
Rune*   place(Rune*, Rune*, Rune*);
void    printcom(void);
void    putchar(int);
void    putd(void);
void    putfile(void);
int     putline(void);
void    putshst(Rune*);
void    putst(char*);
void    quit(void);
void    rdelete(int*, int*);
void    regerror(char *);
```



```

void    reverse(int*, int*);
void    setnoaddr(void);
void    setwide(void);
void    squeeze(int);
void    substitute(int);

⟨function main(ed.c) 41g⟩

⟨function commands(ed.c) 49b⟩

// Command helpers
⟨function printcom(ed.c) 59e⟩

// address parsing
⟨function address(ed.c) 67d⟩
⟨function getnum(ed.c) 69a⟩

// addr1/addr2 set and checks
⟨function setwide(ed.c) 51d⟩
⟨function setnoaddr(ed.c) 60e⟩
⟨function nonzero(ed.c) 60b⟩
⟨function squeeze(ed.c) 52a⟩

⟨function newline(ed.c) 60a⟩
⟨function filename(ed.c) 50e⟩
⟨function exfile(ed.c) 56a⟩

// Error management
⟨function error_1(ed.c) 93c⟩
⟨function error(ed.c) 93b⟩
⟨function rescue(ed.c) 94c⟩

// Note management
⟨function notifyf(ed.c) 94b⟩

// Reading characters
⟨function getchr(ed.c) 47d⟩
⟨function gety(ed.c) 48b⟩
⟨function gettty(ed.c) 47e⟩

// Reading and writing files
⟨function getfile(ed.c) 53c⟩
⟨function putfile(ed.c) 57c⟩

⟨function append(ed.c) 52b⟩
⟨function add(ed.c) 61d⟩

⟨function browse(ed.c) 87a⟩

⟨function callunix(ed.c) 83c⟩

⟨function quit(ed.c) 45b⟩
⟨function onquit(ed.c) 97b⟩

// Delete
⟨function rdelete(ed.c) 62c⟩
⟨function gdelete(ed.c) 92b⟩

// Get/Put lines
⟨function getline_opti(ed.c) 91c⟩

```

```

⟨function putline_opti(ed.c) 89a⟩
⟨function blkio(ed.c) 91a⟩
⟨function getblock(ed.c) 89d⟩

⟨function init(ed.c) 44a⟩

⟨function global(ed.c) 81a⟩

⟨function join(ed.c) 86b⟩

// Get/Put lines simplified versions (not in original ed.c)
⟨function getline 58⟩
⟨function putline 54c⟩

// Search and replace
⟨function substitute(ed.c) 74d⟩
⟨function compsub(ed.c) 73h⟩
⟨function getsub(ed.c) 79b⟩
⟨function dosub(ed.c) 75d⟩
⟨function place(ed.c) 77b⟩

⟨function move(ed.c) 63c⟩
⟨function reverse(ed.c) 64⟩
⟨function getcopy(ed.c) 65c⟩

⟨function compile(ed.c) 71c⟩
⟨function match(ed.c) 72d⟩

// Printing text
⟨function putd(ed.c) 56b⟩
⟨function putstr(ed.c) 45d⟩
⟨function putshst(ed.c) 46a⟩
⟨function putchar(ed.c) 46b⟩

⟨function mktemp(ed.c) 43e⟩

```

⟨function regerror(ed.c) 97a⟩

Uses opti 95.

```

⟨function regerror(ed.c) 97a⟩≡ (95)
void
regerror(char *s)
{
    USED(s);
    error(Q);
}

```

Uses Q 45c and error() 93b.

```

⟨function onquit(ed.c) 97b⟩≡ (95)
void
onquit(int sig)
{
    USED(sig);
    quit();
}

```

Uses quit() 45b.

C.16 Index

Here is a list of the identifiers used, and where they appear. Underlined entries indicate the place of definition. This index is generated automatically.

add(): [61b](#), [61c](#), [61d](#)
addr1: [49c](#), [51d](#), [52a](#), [53b](#), [57b](#), [57c](#), [59e](#), [60c](#), [61d](#), [62b](#), [62d](#), [63c](#), [65c](#), [65d](#), [67a](#), [67c](#), [74d](#), [81a](#), [85e](#), [86b](#), [87a](#), [94c](#)
addr2: [49c](#), [50c](#), [51d](#), [52a](#), [53b](#), [57b](#), [57c](#), [59e](#), [60c](#), [61a](#), [61d](#), [62b](#), [62d](#), [63c](#), [65c](#), [67c](#), [74d](#), [78d](#), [79a](#), [79c](#), [81a](#), [85e](#), [86a](#), [86b](#), [87a](#), [87b](#), [94c](#)
address(): [63c](#), [67a](#), [67d](#)
anymarks: [79c](#), [79e](#), [79f](#), [80b](#)
append(): [50c](#), [52b](#), [61d](#), [62d](#), [65b](#), [79a](#)
bcons: [41f](#), [41g](#), [47d](#)
blkio(): [89d](#), [91a](#)
BLKSIZE-8: [54c](#), [88e](#), [89a](#), [89d](#), [91a](#), [91c](#)
bpagesize: [86d](#), [86d](#), [87a](#)
browse(): [86c](#), [87a](#)
callunix(): [83a](#), [83c](#)
col: [41e](#), [45d](#), [46a](#), [84c](#)
commands(): [41g](#), [49b](#), [81a](#)
compile(): [70](#), [71c](#), [73h](#), [81a](#)
compsub(): [73h](#), [74d](#)
count: [41d](#), [50e](#), [53c](#), [56b](#), [57c](#), [61a](#), [85c](#), [93d](#)
dol: [40](#), [44a](#), [45b](#), [50c](#), [51d](#), [52a](#), [52b](#), [53b](#), [57b](#), [61d](#), [62c](#), [65b](#), [67a](#), [67c](#), [67d](#), [68](#), [69b](#), [70](#), [80a](#), [81a](#), [87a](#), [92b](#), [94c](#)
dosub(): [74d](#), [75d](#)
dot: [40](#), [44a](#), [52b](#), [53b](#), [59e](#), [60c](#), [62c](#), [63c](#), [67b](#), [67c](#), [67d](#), [78d](#), [81a](#), [85e](#), [86b](#), [92b](#)
EOF-3: [47d](#), [47e](#), [48a](#), [48b](#), [49d](#), [50e](#), [51b](#), [54a](#), [60a](#), [65c](#), [79b](#), [81b](#), [83c](#), [93d](#)
error(): [45b](#), [48b](#), [49b](#), [50d](#), [50e](#), [51a](#), [51b](#), [52a](#), [53b](#), [54b](#), [54c](#), [56a](#), [57b](#), [57c](#), [60a](#), [60e](#), [63c](#), [65a](#), [67a](#), [67d](#), [69b](#), [70](#), [71c](#), [72a](#), [73h](#), [74a](#), [74d](#), [75d](#), [77b](#), [78a](#), [78d](#), [79c](#), [80a](#), [81a](#), [81b](#), [82](#), [86b](#), [87b](#), [89d](#), [91a](#), [93b](#), [97a](#)
error_1(): [44a](#), [93b](#), [93c](#), [94b](#)
ESCFLG-6: [73g](#), [77c](#), [78a](#)
ESIZE-4: [71b](#)
exfile(): [50c](#), [56a](#), [57b](#)
fchange: [45a](#), [45b](#), [50c](#), [54c](#), [57b](#), [59c](#), [62a](#), [62c](#), [63c](#), [87b](#), [89a](#), [92b](#), [94c](#)
file: [50a](#), [50c](#), [50d](#), [50e](#), [51b](#), [51c](#), [57b](#), [88b](#)
filename(): [50c](#), [50e](#), [57b](#), [60d](#), [87b](#)
FNSIZE-1: [39d](#), [39e](#), [50a](#)
GBSIZE-7: [80g](#), [81b](#)
gdelete(): [92a](#), [92b](#)
genbuf: [75c](#), [75d](#), [77b](#), [86b](#)
getblock(): [89a](#), [89d](#), [91c](#)
getchr(): [47d](#), [48b](#), [50e](#), [51a](#), [59b](#), [60a](#), [67a](#), [67d](#), [69a](#), [71c](#), [73h](#), [77c](#), [78e](#), [79c](#), [80a](#), [81a](#), [81b](#), [83c](#), [85f](#), [87a](#), [93d](#)
getcopy(): [65b](#), [65c](#)
getfile(): [50c](#), [53c](#)
getline(): [57c](#), [58](#), [59e](#), [65c](#), [72d](#), [86b](#)
getline_opti(): [91b](#), [91c](#)
getnum(): [67d](#), [69a](#), [78f](#), [87a](#)
getsub(): [79a](#), [79b](#)
gettty(): [47e](#), [61d](#), [62d](#)

gety(): [47e](#), [48b](#)
 given: [51d](#), [60e](#), [61d](#), [66a](#), [67c](#), [86a](#)
 global(): [80e](#), [80f](#), [81a](#)
 globbp: [41g](#), [43c](#), [47c](#), [47d](#), [74b](#), [74c](#), [81a](#), [82](#), [93d](#)
 hex: [84c](#), [84d](#), [84d](#)
 iblock: [89b](#), [89c](#), [89d](#)
 ichanged: [89b](#), [89c](#), [89d](#)
 init(): [41g](#), [44a](#), [87b](#)
 io: [50b](#), [50c](#), [50d](#), [56a](#), [57b](#), [88b](#), [93d](#), [94c](#)
 iobuf: [50b](#), [50c](#), [53c](#), [56a](#), [57b](#), [57c](#), [94c](#)
 join(): [86a](#), [86b](#)
 lastc: [47b](#), [47d](#), [50d](#), [54b](#), [89d](#), [93d](#), [94b](#)
 LBSIZE-2: [41b](#), [41c](#), [48b](#), [54b](#), [73f](#), [74a](#), [75c](#), [75d](#), [77b](#), [86b](#)
 line: [41a](#), [41a](#), [46b](#), [84c](#)
 linebp: [55](#), [79b](#), [88d](#), [89a](#)
 linebuf: [41c](#), [47e](#), [48b](#), [53c](#), [54a](#), [54b](#), [54c](#), [58](#), [72d](#), [75d](#), [79b](#), [86b](#), [89a](#), [91c](#)
 linp: [41a](#), [41a](#), [46b](#), [84c](#)
 listf: [84a](#), [84b](#), [84c](#), [85f](#), [85g](#), [87a](#), [93d](#)
 listn: [85a](#), [85b](#), [85c](#), [85f](#), [85g](#), [87a](#), [93d](#)
 loc1: [73b](#), [73c](#), [73d](#), [75d](#), [77a](#)
 loc2: [73b](#), [73c](#), [73d](#), [73e](#), [74d](#), [75d](#), [77a](#)
 main-10(): [41g](#)
 match(): [70](#), [72d](#), [74d](#), [81a](#)
 MAXSUB-5: [72b](#), [72c](#), [72d](#), [78a](#)
 mktemp(): [41g](#), [43e](#)
 move(): [63a](#), [63b](#), [63c](#)
 names: [79c](#), [79d](#), [80a](#), [80b](#), [80d](#)
 NBLK-9: [54c](#), [89d](#), [90](#)
 newline(): [59d](#), [60a](#), [61a](#), [61d](#), [62a](#), [62b](#), [62d](#), [63c](#), [73h](#), [78d](#), [78e](#), [79c](#), [85b](#), [86a](#), [87a](#)
 nlall: [39f](#), [39f](#), [41g](#), [53b](#)
 nleft: [88c](#), [89a](#), [89d](#), [91c](#)
 nonzero(): [59e](#), [60b](#), [62b](#), [62d](#), [63c](#), [74c](#), [78d](#), [79c](#), [86b](#), [86c](#)
 notifyf(): [41g](#), [94b](#)
 oblock: [89b](#), [89c](#), [89d](#)
 oflag: [43a](#), [43b](#), [43c](#), [46b](#)
 onquit(): [97b](#)
 opti: [88f](#), [91b](#), [95](#), [95](#)
 pattern: [71a](#), [71c](#), [72d](#), [72e](#)
 peekc: [47a](#), [47d](#), [49a](#), [59b](#), [67d](#), [68](#), [69a](#), [71c](#), [74b](#), [78e](#), [87a](#), [93d](#)
 pflag: [74b](#), [85d](#), [85e](#), [85f](#), [85g](#), [87a](#), [93d](#)
 place(): [77a](#), [77b](#), [78a](#)
 printcom(): [59d](#), [59e](#), [60c](#), [85b](#), [85e](#), [87a](#)
 putchar(): [45d](#), [46a](#), [46b](#), [56a](#), [56b](#), [61a](#), [85c](#), [93c](#), [94b](#)
 putd(): [56a](#), [56b](#), [56b](#), [61a](#), [85c](#)
 putfile(): [57b](#), [57c](#), [94c](#)
 putline(): [52b](#), [54c](#), [74d](#), [86b](#)
 putline_opti(): [88f](#), [89a](#)
 putshst(): [46a](#), [59e](#)
 putst(): [45d](#), [54a](#), [60d](#), [83c](#), [93c](#)

Q: [45b](#), [45c](#), [45c](#), [48b](#), [49b](#), [50e](#), [51a](#), [51b](#), [52a](#), [54b](#), [56a](#), [57c](#), [60a](#), [60e](#), [63c](#), [65a](#), [67a](#), [67d](#), [69b](#), [70](#), [71c](#), [72a](#),
[73h](#), [74a](#), [74d](#), [75d](#), [77b](#), [78a](#), [78d](#), [79c](#), [80a](#), [81a](#), [81b](#), [82](#), [86b](#), [87b](#), [94b](#), [97a](#)
quit(): [41g](#), [45b](#), [59c](#), [62a](#), [94c](#), [97b](#)
rdelete(): [62b](#), [62c](#), [62d](#), [86b](#)
regerror(): [97a](#)
rescue(): [53b](#), [94b](#), [94c](#)
rescuing: [94a](#), [94b](#), [94c](#)
reverse(): [63c](#), [64](#)
rhsbuf: [73f](#), [73h](#), [74a](#), [75d](#)
savedfile: [39e](#), [41g](#), [43c](#), [51b](#), [51c](#), [60d](#)
savej: [92c](#), [93a](#), [93b](#), [94b](#)
setnoaddr(): [60d](#), [60e](#), [62a](#), [83c](#), [87b](#)
setwide(): [50c](#), [51d](#), [57b](#), [61a](#), [81a](#)
squeeze(): [50c](#), [52a](#), [57b](#), [60b](#), [61a](#), [61d](#), [81a](#)
subexp: [72c](#), [72d](#), [73c](#), [73e](#), [78a](#)
subnewa: [74d](#), [75a](#), [75b](#), [78d](#), [80b](#)
subolda: [78b](#), [78c](#), [78d](#)
substitute(): [74c](#), [74d](#)
T: [44a](#), [44c](#), [44c](#), [54c](#), [89d](#), [91a](#)
template: [41g](#), [43d](#), [43d](#)
tfile: [39b](#), [39b](#), [44a](#), [54c](#), [58](#), [91a](#)
tfname: [39a](#), [41g](#), [44a](#), [45b](#)
tline: [39c](#), [44b](#), [54c](#), [89a](#)
vflag: [42a](#), [42a](#), [42b](#), [43b](#), [45b](#), [56a](#), [83c](#), [87b](#)
waiting: [83b](#), [83c](#), [94b](#)
wrapp: [57a](#), [57b](#), [93d](#)
zero: [39f](#), [41g](#), [44a](#), [45b](#), [50c](#), [51d](#), [52a](#), [53b](#), [57b](#), [61a](#), [61d](#), [65b](#), [67a](#), [67d](#), [68](#), [70](#), [73a](#), [79a](#), [80a](#), [81a](#), [85c](#), [87a](#),
[87b](#), [92b](#), [94c](#)
__anon_enum_1: [95](#)

Bibliography

- [BB18] J. Glenn Brookshear and Dennis Brylow. *Computer Science: An Overview*. Pearson, 13th edition, 2018. cited page(s) 11
- [Ben88] Jon Bentley. *More Programming Pearls*. Addison-Wesley, 1988. cited page(s) 19
- [Ben99] Jon Bentley. *Programming Pearls*. Addison-Wesley, 2nd edition, 1999. cited page(s) 64
- [BO10] Randal E. Bryant and David R. O'Hallaron. *Computer Systems: A Programmer's Perspective*. Prentice Hall, 2010. cited page(s) 6
- [Com84] Douglas Comer. *Operating System Design: The XINU Approach*. Prentice Hall, 1984. cited page(s) 5, 9, 24
- [Com87] Douglas Comer. *Operating System Design: Volume II Internetworking with XINU*. Prentice Hall, 1987. cited page(s) 24
- [Dar14] Lewis Dartnell. *The Knowledge: How to Rebuild Our World from Scratch*. Penguin Press, 2014. cited page(s) 26
- [DPP+97] Sean Dorward, Rob Pike, David Leo Presotto, Dennis M. Ritchie, Howard W. Trickey, and Philip Winterbottom. The inferno operating system. *Bell Labs Technical Journal*, 2(1):5–18, 1997. cited page(s) 25
- [EucBC] Euclid. *The Elements*. 300 BC. cited page(s) 9
- [FFFK18] Matthias Felleisen, Robert Bruce Findler, Matthew Flatt, and Shriram Krishnamurthi. *How to Design Programs: An Introduction to Programming and Computing*. MIT Press, 2nd edition, 2018. Available at <https://htdp.org/>. cited page(s) 11
- [FH95] Christopher Fraser and David Hanson. *A Retargetable C Compiler: Design and Implementation*. Addison-Wesley, 1995. cited page(s) 5
- [Int86] Intel. *Intel 80386 Programmer's Reference Manual*. Intel Corporation, 1986. cited page(s) 7
- [Knu73] Donald E. Knuth. *The Art of Computer Programming, Volume 1, 2, 3*. Addison-Wesley, 1973. cited page(s) 9, 24
- [Knu92] Donald E. Knuth. *Literate Programming*. Center for the Study of Language and Information, 1992. cited page(s) 5, 9
- [Knu99] Donald E. Knuth. *MMIXWare, a RISC Computer for the Third Millenium*. Springer, 1999. cited page(s) 24
- [KP84] Brian W. Kernighan and Rob Pike. *The UNIX Programming Environment*. Prentice Hall, 1984. cited page(s) 11, 36

- [KR88] Brian W. Kernighan and Dennis M. Ritchie. *The C Programming Language*. Prentice Hall, 1988. cited page(s) 9, 11
- [Lio77] John Lions. *Lions Commentary on UNIX 6th Edition, with Source Code*. Peer to Peer Communications, 1977. cited page(s) 24
- [NS05] Noam Nisan and Shimon Shoken. *The Elements of Computing Systems*. MIT Press, 2005. cited page(s) 9, 24, 27
- [Pad14] Yoann Padioleau. *Principia Softwarica: The Plan 9 Kernel 9*. 2014. cited page(s) 10, 19, 20, 88, 94
- [Pad16a] Yoann Padioleau. *Principia Softwarica: The Plan 9 C Compiler 5c*. 2016. cited page(s) 42
- [Pad16b] Yoann Padioleau. *Principia Softwarica: The Plan 9 Core Libraries*. 2016. cited page(s) 37, 41, 47, 69, 71, 92
- [Pad18] Yoann Padioleau. *Principia Softwarica: The Plan 9 Shell rc*. 2018. cited page(s) 37, 83
- [Pad25] Yoann Padioleau. *Principia Softwarica: The Plan 9 Utilities*. 2025. cited page(s) 43
- [PH24] Pavel Panchekha and Chris Harrelson. *Web Browser Engineering*. 2024. <https://browser.engineering/>. cited page(s) 5
- [PPD⁺95] Rob Pike, Dave Presotto, Sean Dorward, Bob Flandrena, Ken Thompson, Howard Trickey, and Phil Winterbottom. Plan 9 from bell labs. In *Computing Systems*, pages 221–254, 1995. Also available at [docs/articles/9.ps](#). cited page(s) 11
- [PPT⁺93] Rob Pike, Dave Presotto, Ken Thompson, Howard Trickey, and Phil Winterbottom. The use of name spaces in plan 9. In *Operating Systems Review*, pages 72–76, 1993. Also available at [docs/articles/names.ps](#). cited page(s) 11
- [Sea01] David Seal. *ARM, Architecture Reference Manual*. Addison-Wesley, 2001. cited page(s) 7, 9
- [Sei09] Peter Seibel. *Coders at Work, Reflections on the Craft of Programming*. Apress, 2009. cited page(s) 19
- [SGG18] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. John Wiley & Sons, 10th edition, 2018. cited page(s) 11
- [SM03] Phillip Stanley-Marbell. *Inferno Programming with Limbo*. John Wiley & Sons, 2003. cited page(s) 25
- [Stu08] Brian L. Stuart. *Principles of Operating Systems: Design and Applications*. Cengage, 2008. cited page(s) 25
- [Tan87] Andrew S. Tanenbaum. *Operating Systems Design and Implementation*. Prentice Hall, 1987. cited page(s) 5, 9, 24
- [TML⁺79] C. P. Thacker, E. M. McCreight, B. W. Lampson, R. F. Sproull, and D. R. Boggs. Alto: A personal computer. Technical report, Xerox PARC, 1979. CSL-79-11. cited page(s) 19, 20
- [WG92] Niklaus Wirth and Jurg Gutknecht. *Project Oberon: The Design of an Operating System and Compiler*. Addison-Wesley, 1992. cited page(s) 24
- [WP97] Phil Winterbottom and Rob Pike. The design of the inferno virtual machine. Technical report, Bell Labs, 1997. cited page(s) 25
- [WR13] Alfred N. Whitehead and Bertrand Russell. *Principia Mathematica*. Cambridge University Press, 1910, 1912, 1913. cited page(s) 5, 9